

Security Review — Checkmarx False-Positive Responses

Managed package **Pledgivo** (namespace `pledgivo`) · responses prepared 2026-10-08 against source commit `32ec92a2`

1. Scan

Scanner	Force.com Source Scanner (Checkmarx)
Scan Id	a0OKX000001JTAA2A4
Engine / preset	CxEngine 9.7 / PortalAll
Job type	ZIP_UPLOAD
Scan start / end	2026-10-06 21:57:40 / 2026-10-07 03:41:59
Security issues reported	273 paths, 91 distinct code locations
Quality issues reported	100 paths (section 5)

2. How to read this document

Checkmarx reports one *path* per source-to-sink dataflow, so a single line of code is often reported many times (one path per entry point that reaches it). Section 4 answers each **code location** once, with a response ID (**R-*nnn***). The **Location** column lists the query and the path numbers exactly as they appear in the scan report, the file and line at scan time, and the file and line in the submitted source. The **Response** column has a fixed shape: **Scanner's claim** restates what the query asserts about the flagged statement (the query description from the scan report, and the code it points at); **Why the claim is false** states, in one paragraph, the specific fact that makes the assertion untrue for this statement; **Supporting detail** traces every path that reaches the statement and the control that governs it; **Evidence** lists what was checked, including measurements taken in a scratch org; and **Sources** cites the Salesforce or vendor documentation each reason rests on. Where one location is flagged by more than one query, its full response appears under the first query and later queries refer back to it. Appendix A lists all 273 paths individually with their source, sink and response ID. The scan also reports 100 code-quality paths under two rules; they are not security findings, and section 5 gives each group of them one disposition, with every path listed in Appendix B.

Recurring platform facts the responses rely on:

- **Explicit user mode.** Queries and DML the scanner flags as missing `USER_MODE` often pass `AccessLevel.USER_MODE` as a method argument (the FFLib selector pattern); the query does not read that argument.
- **Deliberate system mode.** The `SYSTEM_MODE` / `as system` operations reported here fall into these families: (a) the Experience Cloud guest licence cannot hold the grant; (b) the object is the detail side of a master-detail relationship, where field permissions are inert and a user-mode query throws, so access is checked on the master; (c) asynchronous or install context (batch, queueable, platform-event subscriber, install handler) with no interactive user; (d) a standard object a second-generation package permission set cannot grant; (e) administrator-only setup-console operations on the package's own configuration and on setup objects such as permission-set assignments.
- **Sharing.** Every top-level class declares a sharing keyword. `without sharing` is used for scheduled/batch/queueable jobs, guest controllers, narrow inner wrappers and logging infrastructure, and each top-level declaration states its reason in the code. The package is built at API 67.0, where a class with no keyword runs `with sharing`, or in its parent class's mode when it extends one.

- **Email sends.** Every `Messaging.sendEmail` the scan reports sets `setSaveAsActivity(false)`, so the send creates no activity record, and its recipient is read from the donation record (the donor, or the tribute honoree the donor named at checkout), never from a request parameter.

3. Summary by query

Query	Paths	False positive	Fixed since scan	Hardening before submission	Confirmed — fix
Apex CRUD Violation	28	28	0	0	0
Apex CRUD ContentDistribution	7	7	0	0	0
Client DOM Stored XSS	1	1	0	0	0
CRUD Delete	1	1	0	0	0
FLS Create	109	109	0	0	0
FLS Update	69	69	0	0	0
Apex SOQL SOSL USER MODE MISSING	29	29	0	0	0
Sharing	27	27	0	0	0
Apex VF Dynamic JS Resource	2	2	0	0	0
Total	273	273	0	0	0

Query descriptions (from the scan report)

Apex CRUD Violation	This query looks for CRUD operations that are performed without checking for <code>isAccessible()</code> , <code>isUpdateable()</code> or <code>isDeletable()</code> . When an object uses <code>Whold</code> , <code>WhatId</code> , <code>ParentId</code> , <code>LinkedEntityId</code> fields an additional CRUD check should also be performed on the parent object along with the CRUD and FLS checks on the child object and fields.
Apex CRUD ContentDistribution	USER_MODE is recommended for DML operations involving "ContentDistribution" object
Client DOM Stored XSS	This query detects external data flowing into a javascript execution context when that data comes from the database or other stored parameters influced by external users. This may be a false positive if the datatype is safe to render unescaped (such as an integer or rich text field).
CRUD Delete	This query looks for CRUD Delete operations that are performed without checking for <code>isDeletable()</code> permisison before performing the delete operation.
FLS Create	This query looks for CRUD Delete operations that are performed without checking for <code>isCreatable()</code> permisison before performing the Create operation on the Object and its fields.

FLS Update	This query looks for CRUD Update operations that are performed without checking for isUpdateable() permisison before performing the Update operation on the Object and its fields.
Apex SOQL SOSL USER MODE MISSING	USER_MODE is recommended for SOQL or SOSL operations
Sharing	All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.
Apex VF Dynamic JS Resource	Static resources must be used load third-party javascript code

4. Responses

Apex CRUD Violation — 28 paths, 13 locations

What this query claims: *This query looks for CRUD operations that are performed without checking for isAccessible(), isUpdateable() or isDeletable(). When an object uses Whold, WhatId, ParentId, LinkedEntityId fields an additional CRUD check should also be performed on the parent object along with the CRUD and FLS checks on the child object and fields.*

Reported vulnerability	Location	Response
R-001 Apex CRUD Violation False positive	Paths 1, 9, 14 At scan: classes/PaymentSetupGuideService.cls:1918 Submitted source: classes/payment/service/PaymentSetupGuideService.cls:1918	SCANNER'S CLAIM This query looks for CRUD operations that are performed without checking for <code>isAccessible()</code> , <code>isUpdateable()</code> or <code>isDeletable()</code> . When an object uses <code>Whold</code> , <code>WhatId</code> , <code>ParentId</code> , <code>LinkedEntityId</code> fields an additional CRUD check should also be performed on the parent object along with the CRUD and FLS checks on the child object and fields. Flagged statement: <code>if (m.find()) {</code> WHY THE CLAIM IS FALSE The claim is false because the flagged line is a regular-expression match on a string already in memory, not a query or DML statement, so no CRUD operation exists to check; the string was read with <code>WITH USER_MODE</code> , so the running administrator's permissions were already enforced. SUPPORTING DETAIL The flagged line is a regular-expression match on a string already in memory; it is not a query or DML statement. The data it examines was read with <code>WITH USER_MODE</code> , so the running administrator's object and field permissions were already enforced, and only credential and parameter names are retained for a setup checklist. The path is reachable only by the administrator permission set; the fake-Id generator source is test-only and the other is the SOQL builder. EVIDENCE - base/main/default/classes/payment/service/PaymentSetupGuideService.cls:1917 <code>Matcher m = MERGE_FORMULA.matcher(...); if (m.find())</code> <code>queryNamedCredentials (:1583)</code> and <code>queryCustomHeaders (:1607)</code> <code>WITH USER_MODE</code> - base/main/default/classes/setup/controller/SetupController.cls:479; <code>SetupController</code> granted only in <code>Fundraising_Admin</code> <code>fflib_ApexMocks.mock/verify</code> and <code>fflib_IDGenerator.generate</code> are vendored test-support framework code: they are only ever invoked from <code>*_Test</code> classes to build stubs and fake Ids, and no production entry point reaches this sink through them. <code>fflib_QueryFactory.setCondition/selectField(s)/addOrdering</code> is the SOQL builder the selectors use; it is a query-construction helper, not an entry point, and appears as a source only because the engine starts the trace at the builder's string parameters. SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm (system mode ignores the running user's FLS and object permissions; record sharing is governed by the class sharing keyword. Verified via search excerpt only; page body not fetchable)

Reported vulnerability	Location	Response
R-002 Apex CRUD Violation False positive	Paths 2, 18 At scan: classes/ReceiptService.cls:518 Submitted source: classes/donation/service/ReceiptService.cls:518	SCANNER'S CLAIM This query looks for CRUD operations that are performed without checking for isAccessible(), isUpdateable() or isDeletable(). When an object uses Whold, WhatId, ParentId, LinkedEntityId fields an additional CRUD check should also be performed on the parent object along with the CRUD and FLS checks on the child object and fields. Flagged statement: <code>if (!msgs.isEmpty()) { Messaging.sendEmail(msgs); }</code> WHY THE CLAIM IS FALSE The claim is false because this is an internal alert from a scheduled job using explicit addresses, no target record and setSaveAsActivity(false); it performs no DML and no user- or guest-reachable path leads to it. SUPPORTING DETAIL This is an internal alert sent by a scheduled reconciliation job to the owner of a disputed gift and the configured reply-to address; it uses explicit addresses, no target record, and setSaveAsActivity(false), so it performs no DML. No user- or guest-reachable code path reaches it; the reported sources are a test-only fake-Id generator and the SOQL builder's parameter methods. The send was reviewed for a code change on 2026-10-01 and none applies: Messaging.sendEmail takes no access-level argument, and with setSaveAsActivity(false) there is no record operation for a CRUD check to guard. EVIDENCE - base/main/default/classes/donation/service/ReceiptService.cls:575-579 setToAddresses + setSaveAsActivity(false) - base/main/default/classes/donation/async/RefundReconciliationBatch.cls:726 sole caller - base/main/default/classes/donation/selector/OpportunitiesSelector.cls:940 selectOwnerEmailById SYSTEM_MODE - fplib_ApexMocks.mock/verify and fplib_IDGenerator.generate are vendored test-support framework code: they are only ever invoked from *_Test classes to build stubs and fake Ids, and no production entry point reaches this sink through them. - fplib_QueryFactory.setCondition/selectField(s)/addOrdering is the SOQL builder the selectors use; it is a query-construction helper, not an entry point, and appears as a source only because the engine starts the trace at the builder's string parameters. SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_classes_email_outbound_base.htm (Email base class: setSaveAsActivity is true by default; saving as activity applies to the targetObjectId record. Verified via search excerpt; the page body returned 403 to direct fetch and via r.jina.ai only a cookie shell, so the exact wording is not quoted. Whether saveAsActivity=false also suppresses an Enhanced Email EmailMessage row: unverified)

Reported vulnerability	Location	Response
R-003 Apex CRUD Violation False positive	Paths 3, 10, 22 At scan: classes/ReceiptService.cls:103 Submitted source: classes/donation/service/ReceiptService.cls:103	SCANNER'S CLAIM <i>This query looks for CRUD operations that are performed without checking for isAccessible(), isUpdateable() or isDeletable(). When an object uses Whold, WhatId, ParentId, LinkedEntityId fields an additional CRUD check should also be performed on the parent object along with the CRUD and FLS checks on the child object and fields.</i> Flagged statement: Messaging.sendEmail(new List<Messaging.SingleEmailMessage>{ msg }); WHY THE CLAIM IS FALSE The claim that a CRUD check is missing before a create/update/delete is false because Messaging.sendEmail is not a DML statement: with setSaveAsActivity(false) it writes no record at all, so there is nothing for isCreateable() to guard. The recipient and merge-field source are the just-finalized gift's own donor Contact, not a request parameter. SUPPORTING DETAIL This sink sends a template receipt to the donor Contact of the gift that was just finalized; setTargetObjectId and setWhatId select the recipient and the merge-field sources, and setSaveAsActivity(false) is set, so the send writes no Task/activity record and performs no DML the user could be denied. The recipient is derived from the Opportunity, never from caller input. The only interactive reach is a staff retry of a stuck donation (a permission-set-granted console action), which re-runs the same scheduled finalization and sends the receipt only on first creation of the gift. The QueryFactory 'source' is the SOQL builder, not an entry point. The send was reviewed for a code change on 2026-10-01 and none applies: Messaging.sendEmail takes no access-level argument, and with setSaveAsActivity(false) there is no record operation for a CRUD check to guard. EVIDENCE - base/main/default/classes/donation/service/ReceiptService.cls:94-103 — setTemplateId, setTargetObjectId(targetContactId), setWhatId(opportunityId), setSaveAsActivity(false), applySender, sendEmail - base/main/default/classes/donation/service/DonationService.cls:399 — sendDonationReceipt called on first creation only - base/main/default/classes/donation/controller/DonationMonitorController.cls:57 retryDonationStagingNow -> StagingReconciliationBatch.reconcileOne (without sharing) - base/main/default/classes/donation/selector/OpportunitiesSelector.cls:333 selectDonorContactById SYSTEM_MODE with reason comment - fflib_QueryFactory.setCondition/selectField(s)/addOrdering is the SOQL builder the selectors use; it is a query-construction helper, not an entry point, and appears as a source only because the engine starts the trace at the builder's string parameters. SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_classes_email_outbound_base.htm (Email base class: setSaveAsActivity is true by default; saving as activity applies to the targetObjectId record. Verify

Reported vulnerability	Location	Response
		d via search excerpt; the page body returned 403 to direct fetch and via r.jina.ai only a cookie shell, so the exact wording is not quoted. Whether saveAsActivity=false also suppresses an Enhanced Email EmailMessage row: unverified) • https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm (system mode ignores the running user's FLS and object permissions; record sharing is governed by the class sharing keyword. Verified via search excerpt only; page body not fetchable)

Reported vulnerability	Location	Response
R-004 Apex CRUD Violation False positive	Paths 4, 20, 24 At scan: classes/DonorPortalService.cls:548 Submitted source: classes/donor/service/DonorPortalService.cls:548	SCANNER'S CLAIM <i>This query looks for CRUD operations that are performed without checking for isAccessible(), isUpdateable() or isDeletable(). When an object uses Whold, WhatId, ParentId, LinkedEntityId fields an additional CRUD check should also be performed on the parent object along with the CRUD and FLS checks on the child object and fields.</i> Flagged statement: <code>Messaging.sendEmail(new List<Messaging.SingleEmailMessage>{ code.message });</code> WHY THE CLAIM IS FALSE The claim is false because the send itself performs no DML (<code>setSaveAsActivity(false)</code>); the staff actions that reach it re-read the donor Contact in user mode, so the running user's CRUD, FLS and sharing have already been enforced on the only record involved. SUPPORTING DETAIL The CRUD finding on the send itself is a false positive: <code>setSaveAsActivity(false)</code> is set and no record is written by <code>Messaging.sendEmail</code>. The staff 'send portal link' and 'show portal code' actions accept a Contact Id, so both staff twins re-read the donor in the staff user's own mode (<code>resolveStaffDonorEmail</code>, <code>USER_MODE</code> selectors for Contact and Person Account) and refuse a donor that user cannot see, before any code is minted, any email is sent or any address is returned. The guest path, which is authorized by possession of the code it produces, is unchanged. The send was reviewed for a code change on 2026-10-01 and none applies: <code>Messaging.sendEmail</code> takes no access-level argument, and with <code>setSaveAsActivity(false)</code> there is no record operation for a CRUD check to guard. EVIDENCE - base/main/default/classes/donor/service/DonorPortalService.cls:30-47 - class header: the staff twins re-read the donor in the STAFF user's mode (<code>resolveStaffDonorEmail</code>) and refuse a donor that user cannot see - base/main/default/classes/donor/controller/DonorPortalAdminController.cls:72 <code>SendPortalLinkAction</code>, :93 <code>CreatePortalLinkAction</code>, :156 <code>ContactsSelector.selectById (USER_MODE)</code>, :175 <code>AccountsSelector.selectPersonDonorById (USER_MODE)</code> - DonorPortalService_Test / DonorPortalAdminController_Test: a Fundraising_User who cannot see the Contact gets 'not found' and no <code>Portal_Access_Token__c</code> row is created - docs/security/appexchange-security-review-readiness.md F-10 (donor-portal IDOR closed 2026-09-25) SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_classes_email_outbound_base.htm (Email base class: <code>setSaveAsActivity</code> is true by default; saving as activity applies to the <code>targetObjectId</code> record. Verified via search excerpt; the page body returned 403 to direct fetch and via <code>r.jina.ai</code> only a cookie shell, so the exact wording is not quoted. Whether <code>saveAsActivity=false</code> also suppresses an Enhanced Email <code>EmailMessage</code> row: unverified) https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm (system mode ignores the running user's FLS and object permissions; record sharing is governed by the class sharing keyword. Verified via search excerpt only; page body not fetchable)

Reported vulnerability	Location	Response
		https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm (sharing mode of the class where a method is DEFINED applies, not the caller's; inherited sharing takes the caller's. Page body not fetchable; the 'defined, not called' rule confirmed only via secondary sources, e.g. https://salesforcegeek.in/with-sharing-and-without-sharing-in-apex/ — treat as partially verified)
R-005 Apex CRUD Violation False positive	Paths 5, 21, 25 At scan: classes/DonorPortalService.cls:483 Submitted source: classes/donor/service/DonorPortalService.cls:483	SCANNER'S CLAIM <i>This query looks for CRUD operations that are performed without checking for isAccessible(), isUpdateable() or isDeletable(). When an object uses Whold, WhatId, ParentId, LinkedEntityId fields an additional CRUD check should also be performed on the parent object along with the CRUD and FLS checks on the child object and fields.</i> Flagged statement: <code>Messaging.sendEmail(new List<Messaging.SingleEmailMessage>{</code> WHY THE CLAIM IS FALSE The claim is false because <code>Messaging.sendEmail</code> with explicit addresses and <code>setSaveAsActivity(false)</code> writes no record, so there is no DML to guard; the address must already belong to an existing donor, the request is rate-limited, and the response is identical whether or not it matched. SUPPORTING DETAIL This sends a one-time sign-in code to a donor's own address after a guest asks for it; the address must match an existing donor, the request is rate-limited, and the response is the same whether or not it matched. The message uses explicit addresses with no target record and <code>setSaveAsActivity(false)</code> , so no record is created. The staff controller methods listed as sources reach a different send site (the staff twin) and share only the message-building helper; the fake-Id generator source is test-only. The send was reviewed for a code change on 2026-10-01 and none applies: <code>Messaging.sendEmail</code> takes no access-level argument, and with <code>setSaveAsActivity(false)</code> there is no record operation for a CRUD check to guard. EVIDENCE - base/main/default/classes/donor/service/DonorPortalService.cls:481-490 <code>sendCodeEmail</code> <code>buildCodeEmail</code>: <code>setToAddresses</code>, <code>setSaveAsActivity(false)</code> <code>fflib_ApexMocks.mock/verify</code> and <code>fflib_IDGenerator.generate</code> are vendored test-support framework code: they are only ever invoked from <code>*_Test</code> classes to build stubs and fake Ids, and no production entry point reaches this sink through them. SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_classes_email_outbound_base.htm (Email base class: <code>setSaveAsActivity</code> is true by default; saving as activity applies to the <code>targetObjectId</code> record. Verified via search excerpt; the page body returned 403 to direct fetch and via <code>r.jina.ai</code> only a cookie shell, so the exact wording is not quoted. Whether <code>saveAsActivity=false</code> also suppresses an Enhanced Email <code>EmailMessage</code> row: unverified)

Reported vulnerability	Location	Response
R-006 Apex CRUD Violation False positive	Paths 6, 11 At scan: classes/ReceiptService.cls:149 Submitted source: classes/donation/service/ReceiptService.cls:149	SCANNER'S CLAIM <i>This query looks for CRUD operations that are performed without checking for isAccessible(), isUpdateable() or isDeletable(). When an object uses WhatId, WhatId, ParentId, LinkedEntityId fields an additional CRUD check should also be performed on the parent object along with the CRUD and FLS checks on the child object and fields.</i> Flagged statement: Messaging.sendEmail(new List<Messaging.SingleEmailMessage>{ msg }); WHY THE CLAIM IS FALSE The claim is false because Messaging.sendEmail with setSaveAsActivity(false) performs no create, read, update or delete on any sObject, so no CRUD check applies; the recipient Contact Id is read from the finalized ticket order, not from a caller. SUPPORTING DETAIL The event-ticket receipt is sent to the purchaser's own Contact after the purchase is finalized; setSaveAsActivity(false) means the send creates no activity record, so there is no create/read/update/delete for a CRUD check to guard. The recipient Id comes from the purchase record, not from any request parameter. The only reported source is the SOQL builder class, which is a helper rather than an entry point. The send was reviewed for a code change on 2026-10-01 and none applies: Messaging.sendEmail takes no access-level argument, and with setSaveAsActivity(false) there is no record operation for a CRUD check to guard. EVIDENCE - base/main/default/classes/donation/service/ReceiptService.cls:143-149 — setTargetObjectId, setWhatId, setSaveAsActivity(false), sendEmail - base/main/default/classes/event/service/EventPurchaseService.cls:533 caller (sendTicketReceiptQuietly, :1281) - fflib_QueryFactory.setCondition/selectField(s)/addOrdering is the SOQL builder the selectors use; it is a query-construction helper, not an entry point, and appears as a source only because the engine starts the trace at the builder's string parameters. SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_classes_email_outbound_base.htm (Email base class: setSaveAsActivity is true by default; saving as activity applies to the targetObjectId record. Verified via search excerpt; the page body returned 403 to direct fetch and via r.jina.ai only a cookie shell, so the exact wording is not quoted. Whether saveAsActivity=false also suppresses an Enhanced Email EmailMessage row: unverified)

Reported vulnerability	Location	Response
R-007 Apex CRUD Violation False positive	Paths 7, 16, 26 At scan: classes/ReceiptService.cls:329 Submitted source: classes/donation/service/ReceiptService.cls:329	SCANNER'S CLAIM This query looks for CRUD operations that are performed without checking for isAccessible(), isUpdateable() or isDeletable(). When an object uses Whold, WhatId, ParentId, LinkedEntityId fields an additional CRUD check should also be performed on the parent object along with the CRUD and FLS checks on the child object and fields. Flagged statement: <code>if (!msgs.isEmpty()) { Messaging.sendEmail(msgs); }</code> WHY THE CLAIM IS FALSE The claim is false because every reported dataflow source is test-support framework code reachable only from test classes, and the sink is an email send with setSaveAsActivity(false) that creates no record; in production the method runs from the platform-event subscriber after a gift is finalized, to that gift's own donor. SUPPORTING DETAIL Every source reported for this sink is test-support framework code (the mocking library's mock/verify methods and its fake-Id generator), which is only called from test classes. In production the method runs from a platform-event subscriber after a gift is finalized and emails matching-gift instructions to that gift's own donor Contact. setSaveAsActivity(false) is set, so no activity record is written and there is no DML to check. The send was reviewed for a code change on 2026-10-01 and none applies: Messaging.sendEmail takes no access-level argument, and with setSaveAsActivity(false) there is no record operation for a CRUD check to guard. EVIDENCE - base/main/default/classes/donation/service/ReceiptService.cls:313-327 — loop builds messages, setSaveAsActivity(false) - base/main/default/classes/donation/service/CompanyMatchService.cls:65 sole production caller - fflib_ApexMocks.mock/verify and fflib_IDGenerator.generate are vendored test-support framework code: they are only ever invoked from *_Test classes to build stubs and fake Ids, and no production entry point reaches this sink through them. SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_classes_email_outbound_base.htm (Email base class: setSaveAsActivity is true by default; saving as activity applies to the targetObjectId record. Verified via search excerpt; the page body returned 403 to direct fetch and via r.jina.ai only a cookie shell, so the exact wording is not quoted. Whether saveAsActivity=false also suppresses an Enhanced Email EmailMessage row: unverified)

Reported vulnerability	Location	Response
R-008 Apex CRUD Violation False positive	Paths 8, 13 At scan: classes/ReceiptService.cls:603 Submitted source: classes/donation/service/ReceiptService.cls:603	SCANNER'S CLAIM This query looks for CRUD operations that are performed without checking for isAccessible(), isUpdateable() or isDeletable(). When an object uses Whold, WhatId, ParentId, LinkedEntityId fields an additional CRUD check should also be performed on the parent object along with the CRUD and FLS checks on the child object and fields. Flagged statement: Messaging.sendEmail(new List<Messaging.SingleEmailMessage>{ msg }); WHY THE CLAIM IS FALSE The claim is false because the send has no target record (setTargetObjectId(null)) and setSaveAsActivity(false), so it writes nothing; it fires once during server-side finalization, to the honoree address the donor supplied with their own gift, and only when the donor opted in. SUPPORTING DETAIL The tribute notice is a one-time email to the honoree address the donor entered with their gift, sent during server-side finalization on first creation and only when the donor opted in. It has no target record (setTargetObjectId(null)) and setSaveAsActivity(false), so it writes nothing. The reported sources are a test-only fake-Id generator and the SOQL builder. The send was reviewed for a code change on 2026-10-01 and none applies: Messaging.sendEmail takes no access-level argument, and with setSaveAsActivity(false) there is no record operation for a CRUD check to guard. EVIDENCE - base/main/default/classes/donation/service/ReceiptService.cls:702-704 setTargetObjectId(null), setToAddresses(honoree), setSaveAsActivity(false) - base/main/default/classes/donation/service/DonationService.cls:404 caller - fflib_ApexMocks.mock/verify and fflib_IDGenerator.generate are vendored test-support framework code: they are only ever invoked from *_Test classes to build stubs and fake Ids, and no production entry point reaches this sink through them. - fflib_QueryFactory.setCondition/selectField(s)/addOrdering is the SOQL builder the selectors use; it is a query-construction helper, not an entry point, and appears as a source only because the engine starts the trace at the builder's string parameters. SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_classes_email_outbound_base.htm (Email base class: setSaveAsActivity is true by default; saving as activity applies to the targetObjectId record. Verified via search excerpt; the page body returned 403 to direct fetch and via r.jina.ai only a cookie shell, so the exact wording is not quoted. Whether saveAsActivity=false also suppresses an Enhanced Email EmailMessage row: unverified)

Reported vulnerability	Location	Response
R-009 Apex CRUD Violation False positive	Paths 12 At scan: classes/ReceiptService.cls:734 Submitted source: classes/donation/service/ReceiptService.cls:734	SCANNER'S CLAIM This query looks for CRUD operations that are performed without checking for isAccessible(), isUpdateable() or isDeletable(). When an object uses Whold, WhatId, ParentId, LinkedEntityId fields an additional CRUD check should also be performed on the parent object along with the CRUD and FLS checks on the child object and fields. Flagged statement: Messaging.sendEmail(new List<Messaging.SingleEmailMessage>{ msg }); WHY THE CLAIM IS FALSE The claim is false because the send writes no record (setSaveAsActivity(false)), is reached only from the finalize path and a scheduled renewal job, and addresses the recurring schedule's own donor Contact; no user- or guest-callable entry point selects the recipient. SUPPORTING DETAIL Recurring-gift notices (confirmation, failed payment, dunning, re-authorization) are sent by the finalize path and a scheduled renewal job to the schedule's own donor Contact, with setSaveAsActivity(false), so the send writes no record. No user- or guest-callable entry point reaches this method. The reported sources are the SOQL builder's parameter methods. The send was reviewed for a code change on 2026-10-01 and none applies: Messaging.sendEmail takes no access-level argument, and with setSaveAsActivity(false) there is no record operation for a CRUD check to guard. EVIDENCE - base/main/default/classes/donation/service/ReceiptService.cls:709-734 - base/main/default/classes/recurring/async/RecurringRenewalBatch.cls:197-202; base/main/default/classes/recurring/service/RecurringDonationService.cls:956 - fflib_QueryFactory.setCondition/selectField(s)/addOrdering is the SOQL builder the selectors use; it is a query-construction helper, not an entry point, and appears as a source only because the engine starts the trace at the builder's string parameters. SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_classes_email_outbound_base.htm (Email base class: setSaveAsActivity is true by default; saving as activity applies to the targetObjectId record. Verified via search excerpt; the page body returned 403 to direct fetch and via r.jina.ai only a cookie shell, so the exact wording is not quoted. Whether saveAsActivity=false also suppresses an Enhanced Email EmailMessage row: unverified)

Reported vulnerability	Location	Response
R-010 Apex CRUD Violation False positive	Paths 15 At scan: classes/CommunityService.cls:513 Submitted source: classes/setup/service/CommunityService.cls:513	SCANNER'S CLAIM This query looks for CRUD operations that are performed without checking for <code>isAccessible()</code> , <code>isUpdateable()</code> or <code>isDeletable()</code> . When an object uses <code>Whold</code> , <code>WhatId</code> , <code>ParentId</code> , <code>LinkedEntityId</code> fields an additional CRUD check should also be performed on the parent object along with the CRUD and FLS checks on the child object and fields. Flagged statement: <code>Integer assignedCount = [</code> WHY THE CLAIM IS FALSE The claim is false because the statement is a read-only <code>COUNT()</code> of <code>PermissionSetAssignment</code> , a setup object with no sharing model, returning a yes/no verdict and no record data; it is reachable only from the administrator-only setup console, and a user-mode read of setup objects requires View Setup and Configuration, which the package's admin permission set cannot assume every administrator holds. SUPPORTING DETAIL This is a read-only count of the site guest user's permission-set assignments, run from the administrator-only setup console to report whether the payment credential is granted to that guest user; it returns a yes/no verdict and no record data. It runs in system mode because <code>PermissionSetAssignment</code> is setup data that the running administrator can read only with 'View Setup and Configuration', which the package's admin permission set does not grant, and without it the console would falsely report the grant as missing. Only users holding the administrator permission set can reach it. The site carries its own <code>SYSTEM_MODE</code> comment saying so, as does every other setup-object read in the class. EVIDENCE - base/main/default/classes/setup/service/CommunityService.cls:511-516 - <code>// SYSTEM_MODE: the guest user's assignments ... a count only, for the Fundraising_Admin-only Settings console.</code> above the query - python3 scripts/lint/system-mode-inventory.py -> 0 unjustified sites (header-only count now excludes this file's reads) - SetupController granted only in Fundraising_Admin; Fundraising_Admin carries no ViewSetup userPermission SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm (system mode ignores the running user's FLS and object permissions; record sharing is governed by the class sharing keyword. Verified via search excerpt only; page body not fetchable)

Reported vulnerability	Location	Response
R-011 Apex CRUD Violation False positive	Paths 17, 23, 27 At scan: classes/ReceiptService.cls:408 Submitted source: classes/donation/service/ReceiptService.cls:408	SCANNER'S CLAIM <i>This query looks for CRUD operations that are performed without checking for isAccessible(), isUpdateable() or isDeletable(). When an object uses Whold, WhatId, ParentId, LinkedEntityId fields an additional CRUD check should also be performed on the parent object along with the CRUD and FLS checks on the child object and fields.</i> Flagged statement: <code>if (!msgs.isEmpty()) { Messaging.sendEmail(msgs); }</code> WHY THE CLAIM IS FALSE The claim is false because the send writes no record (setSaveAsActivity(false)), it runs only after a refund has already been processed under the Process_Refunds custom permission or by the reconciliation job, and the recipient is the refunded gift's own donor Contact. SUPPORTING DETAIL The refund notice is sent only after a refund has been processed, to the refunded gift's own donor Contact, with setSaveAsActivity(false), so the send itself writes no record. The interactive path requires a dedicated custom permission checked in the controller before any work, and the controller declares with sharing, which the service inherits. Two of the reported sources are test-only mocking framework methods; the third is that permission-gated controller. The send was reviewed for a code change on 2026-10-01 and none applies: Messaging.sendEmail takes no access-level argument, and with setSaveAsActivity(false) there is no record operation for a CRUD check to guard. EVIDENCE - base/main/default/classes/donation/controller/RefundController.cls:22 refundDonation; doValidate checks FeatureManagement.checkPermission('Process_Refunds') - base/main/default/classes/donation/service/RefundService.cls:15 public inherited sharing class RefundService; :328-334 sendRefundNotification after commit - base/main/default/classes/donation/service/ReceiptService.cls:404-408 setSaveAsActivity(false) - base/main/default/classes/donation/selector/OpportunitiesSelector.cls:865 selectByIdForRefund SYSTEM_MODE FOR UPDATE - fplib_ApexMocks.mock/verify and fplib_IDGenerator.generate are vendored test-support framework code: they are only ever invoked from *_Test classes to build stubs and fake Ids, and no production entry point reaches this sink through them. - base/main/default/classes/donation/async/RefundReconciliationBatch.cls:734 batch caller (sendRefundNotifications, ReceiptService:380) SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_classes_email_outbound_base.htm (Email base class: setSaveAsActivity is true by default; saving as activity applies to the targetObjectId record. Verified via search excerpt; the page body returned 403 to direct fetch and via r.jina.ai only a cookie shell, so the exact wording is not quoted. Whether saveAsActivity=false also suppresses an Enhanced Email EmailMessage row: unverified)

Reported vulnerability	Location	Response
		• https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm (system mode ignores the running user's FLS and object permissions; record sharing is governed by the class sharing keyword. Verified via search excerpt only; page body not fetchable) • https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm (sharing mode of the class where a method is DEFINED applies, not the caller's; inherited sharing takes the caller's. Page body not fetchable; the 'defined, not called' rule confirmed only via secondary sources, e.g. https://salesforcegeek.in/with-sharing-and-without-sharing-in-apex/ — treat as partially verified)

Reported vulnerability	Location	Response
R-012 Apex CRUD Violation False positive	Paths 19 At scan: classes/RefundRequestHandler.cls:138 Submitted source: classes/guestrequest/handler/RefundRequestHandler.cls:138	SCANNER'S CLAIM This query looks for CRUD operations that are performed without checking for <code>isAccessible()</code> , <code>isUpdateable()</code> or <code>isDeletable()</code> . When an object uses <code>Whold</code> , <code>WhatId</code> , <code>ParentId</code> , <code>LinkedEntityId</code> fields an additional CRUD check should also be performed on the parent object along with the CRUD and FLS checks on the child object and fields. Flagged statement: <code>Messaging.sendEmail(new List<Messaging.SingleEmailMessage>{ msg });</code> WHY THE CLAIM IS FALSE The claim is false because the send performs no DML (<code>setSaveAsActivity(false)</code> , explicit addresses); the gift Id it refers to was validated against the authenticated donor's own giving history when the request was recorded, and the message goes only to that gift's owner and the configured reply-to address. SUPPORTING DETAIL This notifies staff that a donor asked for a refund through the donor portal; the gift Id was checked against the authenticated donor's own giving history when the request was recorded, and the message goes to that gift's owner and the configured reply-to address, never to a caller-supplied address. It is plain text with explicit addresses and <code>setSaveAsActivity(false)</code> , so it writes no record. The reported sources are the SOQL builder's parameter methods, not entry points. The send was reviewed for a code change on 2026-10-01 and none applies: <code>Messaging.sendEmail</code> takes no access-level argument, and with <code>setSaveAsActivity(false)</code> there is no record operation for a CRUD check to guard. EVIDENCE - base/main/default/classes/guestrequest/handler/RefundRequestHandler.cls:194-199 <code>setToAddresses + setSaveAsActivity(false)</code> - DonorPortalService.submitRefundRequest -> <code>requireOwnedDonation</code> <code>fflib_QueryFactory.setCondition/selectField(s)/addOrdering</code> is the SOQL builder the selectors use; it is a query-construction helper, not an entry point, and appears as a source only because the engine starts the trace at the builder's string parameters. SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_classes_email_outbound_base.htm (Email base class: <code>setSaveAsActivity</code> is true by default; saving as activity applies to the <code>targetObjectId</code> record. Verified via search excerpt; the page body returned 403 to direct fetch and via <code>r.jina.ai</code> only a cookie shell, so the exact wording is not quoted. Whether <code>saveAsActivity=false</code> also suppresses an Enhanced Email <code>EmailMessage</code> row: unverified)

Reported vulnerability	Location	Response
R-013 Apex CRUD Violation False positive	Paths 28 At scan: classes/ReceiptService.cls:173 Submitted source: classes/donation/service/ReceiptService.cls:173	SCANNER'S CLAIM <i>This query looks for CRUD operations that are performed without checking for isAccessible(), isUpdateable() or isDeletable(). When an object uses Whold, WhatId, ParentId, LinkedEntityId fields an additional CRUD check should also be performed on the parent object along with the CRUD and FLS checks on the child object and fields.</i> Flagged statement: Messaging.sendEmail(new List<Messaging.SingleEmailMessage>{ msg }); WHY THE CLAIM IS FALSE The claim is false because the send writes no record (setSaveAsActivity(false)), and the only records involved, the gift and its donor Contact, are read in user mode before the message is built, so a staff user who cannot see them gets nothing to send. SUPPORTING DETAIL Resending a receipt is a staff action whose record access is enforced before the message is built: the gift and the donor Contact are read in user mode (object, field and sharing checks of the running user), so a user who cannot see either gets nothing to send. The recipient is always the gift's own donor Contact, not a caller-supplied address, and setSaveAsActivity(false) means the send writes no record. The other two sources reported are the SOQL builder's parameters, not entry points. The send was reviewed for a code change on 2026-10-01 and none applies: Messaging.sendEmail takes no access-level argument, and with setSaveAsActivity(false) there is no record operation for a CRUD check to guard. EVIDENCE - base/main/default/classes/donation/service/ReceiptService.cls:196 oppSel.selectById (USER_MODE), :211 conSel.selectById (USER_MODE) - base/main/default/classes/donation/service/ReceiptService.cls:223-227 setTargetObjectId + setSaveAsActivity(false) - base/main/default/classes/donation/controller/ReceiptAdminController.cls:38 resendReceipt, class declared with sharing - Fundraising_User / Fundraising_Admin permission sets grant ReceiptAdminController - fflib_QueryFactory.setCondition/selectField(s)/addOrdering is the SOQL builder the selectors use; it is a query-construction helper, not an entry point, and appears as a source only because the engine starts the trace at the builder's string parameters. SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_classes_email_outbound_base.htm (Email base class: setSaveAsActivity is true by default; saving as activity applies to the targetObjectId record. Verified via search excerpt; the page body returned 403 to direct fetch and via r.jina.ai only a cookie shell, so the exact wording is not quoted. Whether saveAsActivity=false also suppresses an Enhanced Email EmailMessage row: unverified) https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm (system mode ignores the running user's FLS and object permissions; record sharing is governed by the class sharing)

Reported vulnerability	Location	Response
		ng keyword. Verified via search excerpt only; page body not fetchable) • https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm (sharing mode of the class where a method is DEFINED applies, not the caller's; inherited sharing takes the caller's. Page body not fetchable; the 'defined, not called' rule confirmed only via secondary sources, e.g. https://salesforcegeek.in/with-sharing-and-without-sharing-in-apex/ — treat as partially verified)

Apex CRUD ContentDistribution — 7 paths, 3 locations

What this query claims: *USER_MODE is recommended for DML operations involving "ContentDistribution" object*

Reported vulnerability	Location	Response
R-014 Apex CRUD ContentDistribution False positive	Paths 1–2 At scan: classes/CampaignPostService.cls:555 Submitted source: classes/campaign/service/CampaignPostService.cls:555	SCANNER'S CLAIM <i>USER_MODE is recommended for DML operations involving "ContentDistribution" object</i> Flagged statement: <code>delete as system revokeDistributions;</code> WHY THE CLAIM IS FALSE The claim that this delete should run in user mode is false because the user was already authorized for the action that triggers it (deleting or replacing a campaign post passes the controller's deletable/editable check on the post in user mode), and ContentDistribution is a standard object a 2GP permission set cannot grant: a user-mode delete of a public link created by a different staff member fails with <code>INSUFFICIENT_ACCESS_OR_READONLY</code> (measured), which would leave a revoked file publicly downloadable. SUPPORTING DETAIL This delete revokes the public ContentDistribution link a campaign post served, after staff have deleted the post or replaced its file. It runs only after an action the user was already authorised for: deleting a post first passes <code>requireDeletable</code> on <code>Campaign_Post__c</code> and commits the delete through a user-mode Unit of Work, so the platform refuses it without delete access; replacing a file first filters the posts through <code>RecordAccess.editableIds</code>. The revocation set is narrow: <code>planRevocations</code> keeps any link that another post still serves, so only the link the released post exposed is removed. ContentDistribution is a standard object a second-generation package permission set cannot grant, so a user-mode delete would depend on a manual admin step. The path is never reachable by a guest, and revoking a public link reduces exposure; it never widens it. Converting this delete to user mode was tested and rejected because it breaks the feature for the users it is built for. Measured 2026-10-01 in a scratch org, running as a Standard User who holds the <code>Fundraising_Admin</code> permission set: deleting a ContentDistribution created by another staff member with <code>Database.delete(rows, AccessLevel.USER_MODE)</code> fails with <code>INSUFFICIENT_ACCESS_OR_READONLY</code>. In user mode the public link of a post one colleague published could not be revoked when another colleague deletes that post, and the file would stay publicly reachable. System mode is what guarantees the link is withdrawn. EVIDENCE - base/main/default/classes/campaign/controller/CampaignPostController.cls:156 - <code>requireDeletable(Campaign_Post__c.SObjectType, 'campaign posts')</code> before :166 <code>deletePost(postId)</code> - base/main/default/classes/campaign/service/CampaignPostService.cls:163-187 - <code>deletePost</code>: the <code>UserModeDML</code> delete commits at :179-183 before <code>retirement.execute(false)</code> at :186 - base/main/default/classes/campaign/service/CampaignPostService.cls:379-381 - <code>stampablePosts</code> filters through <code>RecordAccess.editableIds</code> before a file replace; :438 runs the retirement - base/main/default/classes/campaign/service/CampaignPostService.cls:515-545 - <code>planRevocations</code> keeps any link another post still serves (<code>selectByFileUrlsSystemMode</code>, :530)

Reported vulnerability	Location	Response
		- base/main/default/classes/campaign/service/CampaignPostService.cls:547-555 - execute() rationale and the SYSTEM_MODE comment at the site - base/main/default/classes/campaign/service/CampaignPostService.cls:453 - private with sharing class FileRetirement; outer class with sharing at:28 SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm (user-mode DML enforces the running user's object permissions and sharing) - AGENTS.md Change Management: 2GP strips standard-object grants (project finding 2026-08-16)

Reported vulnerability	Location	Response
R-015 Apex CRUD ContentDistribution False positive	Paths 3–4, 6 At scan: <code>classes/AppLogger.cls:125</code> Submitted source: <code>classes/shared/AppLogger.cls:125</code>	SCANNER'S CLAIM <i>USER_MODE is recommended for DML operations involving "ContentDistribution" object</i> Flagged statement: <code>insert as system new Log__c(</code> WHY THE CLAIM IS FALSE The claim is false because the insert is the package's diagnostic log, which must succeed from catch blocks in every context including guest controllers; no permission set grants Create on Log__c by design, so a user-mode insert fails for every user (measured: <code>insert as user new Log__c</code> throws <code>System.TypeException "DML operation INSERT not allowed on Log__c"</code>), and the row holds only a class name, level and message with no caller-controlled routing. SUPPORTING DETAIL AppLogger persists diagnostic entries from catch blocks in every context, including guest controllers whose users cannot hold Create on any internal object; no permission set grants Create on Log__c by design, so logs are writable only through this class. Values are truncated to field length, Log__c is Private with read granted only to internal staff sets, and no client-callable method takes a log payload - the message text comes from server-side exceptions and developer-written strings. The class-level comment documents why it is <code>without sharing</code> and bypasses the Unit of Work. Converting this insert to user mode was tested and rejected. Measured 2026-10-01 in a scratch org, running as a Standard User who holds the Fundraising_Admin permission set: <code>insert as user new Log__c(...)</code> throws <code>System.TypeException: DML operation INSERT not allowed on Log__c</code>, because no permission set grants Create on the log object - by design, so that no user can author or forge diagnostic rows. A user-mode insert would therefore lose every log entry, including the ones that record security-relevant failures. EVIDENCE - base/main/default/classes/shared/AppLogger.cls:1-9 - <code>without sharing</code> with rationale - base/main/default/classes/shared/AppLogger.cls:117-135 - DML-limit guard; left() truncation; failures swallowed - Log__c OWD Private; Fundraising_Admin/User/ReadOnly: Read only, no Create; GuestDonor: no grant SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm (user-mode DML throws when the running user lacks object Create) - Measured in a scratch org (2026-09-30): <code>`insert as user new Log__c(...)`</code> throws <code>System.TypeException "DML operation INSERT not allowed on Log__c"</code>

Reported vulnerability	Location	Response
R-016 Apex CRUD ContentDistribution False positive	Paths 5, 7 At scan: classes/SettingsService.cls:345 Submitted source: classes/setup/service/SettingsService.cls:345	SCANNER'S CLAIM <i>USER_MODE is recommended for DML operations involving "ContentDistribution" object</i> Flagged statement: <code>upsert as system settings;</code> WHY THE CLAIM IS FALSE The claim is false on both counts: the upsert is shared by the InstallHandler (package system user, no permission set) and the admin console (gated by the Manage_Fundraising_Settings custom permission), and in both contexts a user-mode write on the package's hierarchy custom setting requires Customize Application, which neither the install user nor the package's admin permission set can hold (measured: <code>upsert as user</code> throws <code>SecurityException</code>). The ContentDistribution query reads only a Content Delivery the same code created from a packaged static resource. SUPPORTING DETAIL This upsert is shared by the package install handler, which runs as the package's system user with no permission set, and by the settings console controller, which is granted only in the Fundraising_Admin permission set. System mode is kept because the install path has no FLS to enforce and because DML on a Hierarchy Custom Setting in user mode requires Customize Application, a system permission no packaged permission set can carry. The console path does not rely on Apex class access alone: every controller action that reaches this write (SaveGeneralSettings, DetectOrgFeatures, ResetToDefaults, SaveExperienceSite) first checks the packaged custom permission Manage_Fundraising_Settings, granted only in Fundraising_Admin, and refuses the save with a generic error when the running user does not hold it. The site carries a SYSTEM_MODE comment naming that gate. The user-mode alternative was tested and does not work for the people who use this screen. Measured 2026-10-01 in a scratch org, running as a Standard User who holds the Fundraising_Admin permission set: <code>Schema.sObjectType.Settings__c</code> reports <code>isCreateable</code> , <code>isUpdateable</code> and <code>isAccessible</code> all false, and <code>upsert as user</code> on the settings row throws <code>System.SecurityException: Access to entity 'Settings__c' denied</code> . The platform ties custom-setting DML to Customize Application, which is a system permission the fundraising administrator is not expected to hold, so a user-mode write would lock every non-system-administrator out of the settings console. The custom-permission check before the write is the access control; it is enforced in Apex on every action that reaches this line. EVIDENCE - base/main/default/classes/setup/service/SettingsService.cls:338-345 - <code>// SYSTEM_MODE: reason naming the Manage_Fundraising_Settings gate, then upsert as system settings</code> - base/main/default/classes/setup/controller/SetupController.cls:1737-1742 - <code>SettingsWriteAction.requireSettingsPermission(); FeatureManagement.checkPermission('Manage_Fundraising_Settings') else addError(SETTINGS_NOT_AUTHORIZED)</code>

Reported vulnerability	Location	Response
		- base/main/default/classes/setup/controller/SetupController.cls:1745/1754 SaveGeneralSettings, :2666-2667 DetectOrgFeatures, :2702-2703 ResetToDefaults, :2762/2771 SaveExperienceSite - each doValidate() starts with requireSettingsPermission() - base/main/default/customPermissions/Manage_Fundraising_Settings.customPermission-meta.xml; Fundraising_Admin.permissionset-meta.xml grants it; Fundraising_User / Fundraising_ReadOnly / Fundraising_GuestDonor do not - SetupController_Test: five deny tests (bare Standard User), one deny test (Fundraising_User), one allow test (Fundraising_Admin); 312 tests pass, SetupController 96%, SettingsService 98% - PostInstallHandler.onInstall -> seedRecordTypeMappings / ensureDefaultLogoSeeded still reach this write with no permission check: install context, no running user to check - docs/security/apexchange-security-review-readiness.md F-13 SOURCES https://help.salesforce.com/s/articleView?id=sf.cs_schema_settings.htm https://applicationperfection.com/2024/02/29/control-who-gets-read-access-to-custom-settings/ https://github.com/patronmanager/apex-dml-manager/issues/2 https://help.salesforce.com/s/articleView?language=en_US&id=release-notes.rn_apex_default_user_mode.htm&release=262&type=5 https://help.salesforce.com/s/articleView?id=sf.apex_post_install_script.htm&language=en_US&type=5

Client DOM Stored XSS — 1 paths, 1 locations

What this query claims: This query detects external data flowing into a javascript execution context when that data comes from the database or other stored parameters influced by external users. This may be a false positive if the datatype is safe to render unescaped (such as an integer or rich text field).

Reported vulnerability	Location	Response
R-017 Client DOM Stored XSS False positive	Paths 1 At scan: lwc/pfGivingStatement/pfGivingStatement.html:34 Submitted source: lwc/pfGivingStatement/pfGivingStatement.html:34	SCANNER'S CLAIM <i>This query detects external data flowing into a javascript execution context when that data comes from the database or other stored parameters influenced by external users. This may be a false positive if the datatype is safe to render unescaped (such as an integer or rich text field).</i> Flagged statement: <code><p class="gs-err" role="alert">{errorMessage}</p></code> WHY THE CLAIM IS FALSE The claim that stored, externally influenced data reaches a JavaScript execution context is false because the reported source (the donor's portal session token) is only passed as an Apex parameter and never written to the DOM; the value rendered at the flagged line is the response envelope's error text or a packaged constant, bound through an LWC <code>{expression}</code> , which the framework renders as escaped text and never as HTML or script. SUPPORTING DETAIL The flagged source is the donor's own portal session token read from sessionStorage; it is passed as an Apex parameter to DonorPortalController.getGivingStatement and is never written into the DOM. The value rendered at base/main/default/lwc/pfGivingStatement/pfGivingStatement.html:34 is either the error string of the controller's response envelope or a package-authored literal. The controller's action classes extend GuestAction, whose callerSafeMessage returns an AppException message (every one on this path is a constant: the invalid-session message, 'Choose a tax year.', 'That tax year is not available.', 'A donor is required to build a giving statement.') or one fixed generic sentence for any other exception - no record field, request parameter or token is echoed. Finally, the sink is an LWC <code>{expression}</code> text binding, which the framework renders as a text node, not HTML; neither pfGivingStatement nor publicDonorDashboard uses innerHTML, lwc:inner-html or lwc:dom="manual". The flagged source is also shape-checked: the value read from sessionStorage is accepted only when it matches the exact shape of a session token the server issues (64 lowercase hexadecimal characters); anything else is discarded and removed from storage before it reaches component state or an Apex call. The finding is a false positive - the value is never rendered - and storage content that the component did not write cannot enter it at all. EVIDENCE - base/main/default/lwc/publicDonorDashboard/publicDonorDashboard.js:314-331 - stored = window.sessionStorage.getItem(SESSION_KEY) (SESSION_KEY = 'pf-donor-portal-session', :41); assigned to this.token then loadDashboard(); not bound into markup - base/main/default/lwc/publicDonorDashboard/publicDonorDashboard.js:45 and :320-330 - SESSION_TOKEN_SHAPE = /^[0-9a-f]{64}\$/ ; a stored value that fails the test is removed and the sign-in form is shown - base/main/default/classes/donor/service/DonorPortalService.cls:809-811 - generateToken() returns EncodingUtil.convertToHex(Crypto.generateAesKey(256)), i.e. 64 lowercase hex characters

Reported vulnerability	Location	Response
		- base/main/default/lwc/publicDonorDashboard/publicDonorDashboard.html:389 - token={token} passed as a component property only; the only other {token} use is if:false={token} (line 47), a boolean - base/main/default/lwc/pfGivingStatement/pfGivingStatement.js:96-104 and :115-121 - errorMessage = res?.error literal, or the printStatement() return value - base/main/default/lwc/uiStatementPrint/uiStatementPrint.js:249 - printStatement returns only the literal 'Pop-up blocked. Allow pop-ups ...' or null; the statement document itself is built with createElement/textContent (el) helper :31-35), never innerHTML - base/main/default/classes/donor/controller/DonorPortalController.cls:126-128, :383-399, :472-478 - getGivingStatement -> TokenAction.doValidate requireNotBlank(token, 'token') -> validateToken -> service - base/main/default/classes/shared/GuestAction.cls:36-38 - callerSafeMessage returns AppException message verbatim, otherwise GENERIC_FAILURE constant - base/main/default/classes/shared/ControllerAction.cls:45-57 - validation errors joined from addError literals; exceptions routed through callerSafeMessage; full message to AppLogger only - base/main/default/classes/donor/service/DonorPortalService.cls:54-55, :696-711 - validateToken throws AppException(INVALID_MSG) constant for every failure - base/main/default/classes/donation/service/GivingStatementService.cls:298, :303, :310-312 - the only AppExceptions on the statement path are constant strings - grep lwc:dom lwc:inner-html innerHTML in lwc/pfGivingStatement and lwc/publicDonorDashboard - zero hits SOURCES https://lwc.dev/guide/html_templates https://trailhead.salesforce.com/content/learn/modules/secure-clientside-development/mitigate-crosssite-scripting https://developer.salesforce.com/docs/platform/lwc/guide/security-lwsec-sanitize.html

CRUD Delete — 1 paths, 1 locations

What this query claims: *This query looks for CRUD Delete operations that are performed without checking for isDeletable() permisison before performing the delete operation.*

Reported vulnerability	Location	Response
R-018 CRUD Delete False positive	Paths 1 At scan: classes/RetentionPurgeBatch.cls:90 Submitted source: classes/shared/async/RetentionPurgeBatch.cls:90	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isDeletable() permisison before performing the delete operation.</i> Flagged statement: delete as system scope; WHY THE CLAIM IS FALSE The claim that an isDeletable() check is missing is false because the delete runs in batch Apex, which the scan's own rule exempts, over a scope produced only by the retention selectors (diagnostic, staging and request rows past the administrator-configured window); no caller chooses the rows, and a job owner without Delete on an internal object would otherwise silently leave expired data in place. SUPPORTING DETAIL This is the data-retention purge: the scope is produced only by the per-target selector locators (Log__c and Transaction_Log__c older than the admin-configured retention window, terminal Donation_Staging__c and Guest_Request__c older than it, and terminal or stale portal tokens after 7 days). It runs as a scheduled batch or when an administrator runs it from the setup console; no caller supplies record Ids. No packaged permission set grants Delete on these append-only tables by design, so the purge must delete in system mode. EVIDENCE - base/main/default/classes/shared/async/RetentionPurgeBatch.cls:1-16 - rationale; without sharing - base/main/default/classes/shared/async/RetentionPurgeBatch.cls:41-78 - fixed selector locators per Target; else throws - base/main/default/classes/setup/service/SettingsService.cls:927-932 - dataRetentionDays default 90 (no lower floor; admin-controlled) SOURCES - unverified against primary docs (developer.salesforce.com apex_scheduler.htm returned 403 / cookie wall); secondary: https://help.salesforce.com/s/articleView?id=sf.code_schedule_batch_apex.htm (search snippet: scheduled Apex executes under the user who scheduled the job)

FLS Create — 109 paths, 22 locations

What this query claims: *This query looks for CRUD Delete operations that are performed without checking for isCreatable() permisison before performing the Create operation on the Object and its fields.*

Reported vulnerability	Location	Response
R-019 FLS Create False positive	Paths 1 At scan: <code>classes/ReconciliationHeartbeat.cls:59</code> Submitted source: <code>classes/donation/async/ReconciliationHeartbeat.cls:59</code>	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for <code>isCreatable()</code> permission before performing the Create operation on the Object and its fields.</i> Flagged statement: <code>insert as system new Transaction_Log__c(Event_Type__c = 'HEARTBEAT_ALERT', Status__c = 'Rejected',</code> WHY THE CLAIM IS FALSE The claim is false because the insert is an operations alarm written by a scheduled job with no user input, every field a constant or a count of stuck rows; the job runs as whoever scheduled it, including the install context that holds no permission set, and Salesforce's own guidance accepts system-mode DML in scheduled context. SUPPORTING DETAIL This is an operations alarm written by a scheduled job with no user input: every field is a constant or a message built from a count of stuck staging rows. The job is scheduled at package install (post-install context, which holds no permission sets) or by an administrator from the setup console, so the insert must not depend on the scheduling user's <code>Transaction_Log__c</code> grants; <code>Transaction_Log__c</code> is an internal audit object not granted to guests. No record Id or value from a caller reaches the insert. EVIDENCE - base/main/default/classes/donation/async/ReconciliationHeartbeat.cls:8-11 - <code>without sharing</code> with rationale (scheduled job; runs as whoever scheduled it) - base/main/default/classes/donation/async/ReconciliationHeartbeat.cls:26-62 - <code>execute()</code>: selector count -> constant-valued log insert -> admin email - base/main/default/classes/setup/controller/PostInstallHandler.cls:495 - <code>scheduleIfAbsent</code> schedules it at install SOURCES - unverified against primary docs (developer.salesforce.com apex_scheduler.htm returned 403 / cookie wall); secondary: https://help.salesforce.com/s/articleView?id=sf.code_schedule_batch_apex.htm (search snippet: scheduled Apex executes under the user who scheduled the job)
R-015 FLS Create False positive	Paths 2-7 At scan: <code>classes/AppLogger.cls:125</code> Submitted source: <code>classes/shared/AppLogger.cls:125</code>	See R-015 under “Apex CRUD ContentDistribution” — same code location, same response.

Reported vulnerability	Location	Response
R-020 FLS Create False positive	Paths 8–12 At scan: <code>classes/StripePaymentSweep.cls:203</code> Submitted source: <code>classes/donation/async/StripePaymentSweep.cls:203</code>	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permission before performing the Create operation on the Object and its fields.</i> Flagged statement: <code>insert as system new Transaction_Log__c(</code> WHY THE CLAIM IS FALSE The claim is false because the insert records the outcome of a scheduled repair sweep with constant values, the staging Id being repaired and a caught exception message; it runs in batch context, including when scheduled at install by a user with no permission set, and no caller-supplied value reaches it. SUPPORTING DETAIL logSweep records the outcome of the scheduled Stripe-to-Salesforce repair sweep; its arguments are constants chosen by the sweep, the staging Id it is repairing, and a caught exception message. The sweep is scheduled at install or started by an administrator from the setup console, and the install-context owner holds no permission sets, so the audit insert runs in system mode. No caller-supplied value reaches it. EVIDENCE - base/main/default/classes/donation/async/StripePaymentSweep.cls:23-26 - <code>without sharing</code> with rationale - base/main/default/classes/donation/async/StripePaymentSweep.cls:200-210 - private logSweep, all values from sweep internals SOURCES - unverified against primary docs (developer.salesforce.com apex_scheduler.htm returned 403 / cookie wall); secondary: https://help.salesforce.com/s/articleView?id=sf.code_schedule_batch_apex.htm (search snippet: scheduled Apex executes under the user who scheduled the job)

Reported vulnerability	Location	Response
R-021 FLS Create False positive	Paths 13–14, 24, 41, 48–49 At scan: classes/StagingReconciliationBatch.cls:281 Submitted source: classes/donation/async/StagingReconciliationBatch.cls:281	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permission before performing the Create operation on the Object and its fields.</i> Flagged statement: insert as system log; WHY THE CLAIM IS FALSE The claim is false because the insert writes the job's own audit row with every field system-derived (lds created in the same transaction, a constant event type, an exception message, a timestamp), in a scheduled batch context the scan's own rule exempts and that may run as a user with no permission set. SUPPORTING DETAIL This inserts the reconciliation audit row (Transaction_Log__c) for each staging the job processes; every field is system-derived (staging Id, Opportunity Id created by the same transaction, a constant event type/status, an exception message, a timestamp). It runs in the scheduled reconciler and the internal retry action; the scheduled run can be owned by the install context user, which holds no permission sets, so the audit write must not depend on the running user's grants. Transaction_Log__c is internal and not granted to the guest profile. EVIDENCE - base/main/default/classes/donation/async/StagingReconciliationBatch.cls:186-250 - log fields populated from staging Id, oppld, constants, e.getMessage() - base/main/default/classes/donation/async/StagingReconciliationBatch.cls:281 - single insert per staging SOURCES - unverified against primary docs (developer.salesforce.com apex_scheduler.htm returned 403 / cookie wall); secondary: https://help.salesforce.com/s/articleView?id=sf.code_schedule_batch_apex.htm (search snippet: scheduled Apex executes under the user who scheduled the job)
R-016 FLS Create False positive	Paths 15, 28, 50–54, 56–61, 63–65, 67–68, 74–75, 77–78, 81, 88, 90, 92, 94, 96, 98, 100 At scan: classes/SettingsService.cls:345 Submitted source: classes/setup/service/SettingsService.cls:345	See R-016 under “Apex CRUD ContentDistribution” — same code location, same response.

Reported vulnerability	Location	Response
R-022 FLS Create False positive	Paths 16–17, 29 At scan: classes/RefundReconciliationBatch.cls:344 Submitted source: classes/donation/async/RefundReconciliationBatch.cls:344	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permission before performing the Create operation on the Object and its fields.</i> Flagged statement: upsert as system s; WHY THE CLAIM IS FALSE The claim that this write bypasses a required createable/updateable check is false: the row is the package's own job cursor (two timestamps computed from the payment gateway's responses), the statement runs only inside a scheduled batch with no interactive user, and Salesforce's own guidance accepts system-mode DML in batch context as a false positive. A user-mode write would make the job's correctness depend on which administrator happened to schedule it. SUPPORTING DETAIL This write stores two Datetime high-water marks (refund and dispute cursors) computed by the batch from the payment gateway's list responses; no user-supplied value reaches it. It runs only in the scheduled reconciliation job or when an administrator starts that job from the settings console, and the cursor must advance regardless of the scheduling user's field permissions or the next run re-reads the whole window. Enforcing FLS here would not protect any user data; it would only let a missing grant silently stall reconciliation. EVIDENCE - base/main/default/classes/donation/async/RefundReconciliationBatch.cls:334-344 finish(): only when !hadError; sets Refund_Reconcile_Cursor__c / Dispute_Reconcile_Cursor__c from cursorTarget/disputeCursorTarget (system-computed), then upsert as system s - base/main/default/classes/donation/async/RefundReconciliationBatch.cls:170-171 reads the same cursors with a 2h overlap - Scheduled Apex executes in the context of the user who scheduled it (CronTrigger owner), in system mode - not the Automated Process user - Fundraising_Admin holds edit on both cursor fields; the only interactive trigger (runJobNow) is Fundraising_Admin-only - base/main/default/classes/donation/async/RefundReconciliationBatch.cls:155 and :336 use Settings__c.getOrgDefaults() (the org-default row only) SOURCES https://help.salesforce.com/s/articleView?id=platform.code_schedule_batch_apex.htm&language=en_US&type=5

Reported vulnerability	Location	Response
R-023 FLS Create False positive	Paths 18–19, 43 At scan: classes/SettingsService.cls:1026 Submitted source: classes/setup/service/SettingsService.cls:1026	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permission before performing the Create operation on the Object and its fields. Flagged statement:</i> <pre>upsert as system s;</pre> WHY THE CLAIM IS FALSE The claim is false because the write is to the org-default row of the package's own custom setting, reached only through an administrator console gated by the Manage_Fundraising_Settings custom permission, with both the field name and the value validated against fixed allow-lists; and a user-mode upsert on a hierarchy custom setting requires Customize Application, which the package's own admin permission set cannot grant (measured: <code>upsert as user</code> throws <code>SecurityException</code> for a <code>Fundraising_Admin</code> user). SUPPORTING DETAIL This writes one of two scheduling-interval fields on the org-default <code>Settings__c</code> row; the family is checked against two fixed names and the interval against a fixed allow-list before the write. It is reached only from the settings console controller's <code>ScheduleJobFamily</code> action, which is granted only in the <code>Fundraising_Admin</code> permission set. System mode is retained because DML on a Hierarchy Custom Setting in user mode requires Customize Application, which a packaged permission set cannot grant. <code>ScheduleJobFamily.doValidate()</code> first checks the packaged custom permission <code>Manage_Fundraising_Settings</code> (granted only in <code>Fundraising_Admin</code>) and refuses the save when the running user does not hold it; the site carries a <code>SYSTEM_MODE</code> comment naming that gate. The user-mode alternative was tested and does not work for the people who use this screen. Measured 2026-10-01 in a scratch org, running as a Standard User who holds the <code>Fundraising_Admin</code> permission set: <code>Schema.sObjectType.Settings__c</code> reports <code>isCreateable</code>, <code>isUpdateable</code> and <code>isAccessible</code> all false, and <code>upsert as user</code> on the settings row throws <code>System.SecurityException: Access to entity 'Settings__c' denied</code>. The platform ties custom-setting DML to Customize Application, which is a system permission the fundraising administrator is not expected to hold, so a user-mode write would lock every non-system-administrator out of the settings console. The custom-permission check before the write is the access control; it is enforced in Apex on every action that reaches this line. EVIDENCE - base/main/default/classes/setup/service/SettingsService.cls:1022-1026 - <code>// SYSTEM_MODE:</code> reason naming the <code>ScheduleJobFamily</code> gate, then <code>upsert as system s</code> - base/main/default/classes/setup/controller/SetupController.cls:2192-2201 - <code>ScheduleJobFamily</code> extends <code>SettingsWriteAction</code>; <code>doValidate()</code> calls <code>requireSettingsPermission()</code> before the family / interval checks <code>SetupController_Test.test_scheduleJobFamily_userWithoutSettingsPermission_isRefused</code> <code>Fundraising_Admin</code> is the only permission set with <code>SetupController</code> classAccess and the only one carrying <code>Manage_Fundraising_Settings</code>

Reported vulnerability	Location	Response
		- docs/security/appexchange-security-review-readiness.md F-13 SOURCES https://help.salesforce.com/s/articleView?id=sf.cs_schema_settings.htm https://github.com/patronmanager/apex-dml-manager/issues/2 https://help.salesforce.com/s/articleView?language=en_US&id=release-notes.rn_apex_default_user_mode.htm&release=262&type=5

Reported vulnerability	Location	Response
R-024 FLS Create False positive	Paths 20, 44 At scan: classes/PostInstallHandler.cls:1515 Submitted source: classes/setup/controller/PostInstallHandler.cls:1515	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permission before performing the Create operation on the Object and its fields.</i> Flagged statement: <code>upsert as system s;</code> WHY THE CLAIM IS FALSE The claim is false because the write targets the org-default row of the package's own hierarchy custom setting, runs only inside the InstallHandler as the package's system user (no permission set), and writes an Id the same install step just produced; custom-setting DML in user mode would additionally require Customize Application, which no install user holds. SUPPORTING DETAIL This stores the Id of the event pricebook that the same install step just found or created on the org-default custom setting row, only when that field is blank. It runs only from the InstallHandler as the package's system user, which has no permission set to enforce, and never overwrites an administrator's value. No user input reaches the write. EVIDENCE - base/main/default/classes/setup/controller/PostInstallHandler.cls:18 <code>public without sharing class PostInstallHandler implements InstallHandler;</code> lines 1-4 header: 'runs in package install/upgrade context' - base/main/default/classes/setup/controller/PostInstallHandler.cls:169-173 doc: 'the script runs as a per-package system user that does not exist before runtime' - Salesforce: a post-install script 'runs as a special system user that represents your package' (search-result excerpt of the help page; page body itself not fetchable, so the quote is from the indexed snippet) - base/main/default/classes/setup/controller/PostInstallHandler.cls:183-186 <code>onInstall</code> calls each <code>seed*</code> step in its own try/catch; only <code>logStepFailure</code> on error - grep of non-test classes: <code>seedDefaultSettings</code>, <code>seedDefaultImages</code>, <code>seedDefaultPageDesign</code>, <code>seedEventPricebook</code>, <code>seedEventDonationProduct</code>, <code>seedEventFeeProduct</code>, <code>seedContentTemplates</code> have NO callers outside <code>PostInstallHandler</code> (mentions in <code>PricebookService/SettingsService</code> are comments/strings only) - Class is <code>public</code>, not <code>global</code> (changed 2026-09-05), so subscriber anonymous Apex / other namespaces cannot invoke these methods - Only backfills when the field is blank ('only backfills the Settings lookup when blank, so an admin's own configuration is never overwritten on upgrade' - method Javadoc) - Value is the Id of a record this same method just found or inserted; no user input SOURCES https://help.salesforce.com/s/articleView?id=sf.apex_post_install_script.htm&language=en_US&type=5

Reported vulnerability	Location	Response
R-025 FLS Create False positive	Paths 21, 45 At scan: classes/PostInstallHandler.cls:1557 Submitted source: classes/setup/controller/PostInstallHandler.cls:1557	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permission before performing the Create operation on the Object and its fields.</i> Flagged statement: upsert as system s; WHY THE CLAIM IS FALSE The claim is false because the write is to the org-default row of the package's own custom setting, from the InstallHandler's system user with no permission set, storing an Id the install step itself created; there is no running user whose field permissions could meaningfully be checked. SUPPORTING DETAIL This records the Id of the event donation product, found or created by the same install step, on the org-default custom setting row when that field is blank. It runs only from the InstallHandler as the package's system user and does not overwrite an administrator's value. No user input reaches the write. EVIDENCE - base/main/default/classes/setup/controller/PostInstallHandler.cls:18 public without sharing class PostInstallHandler implements InstallHandler; lines 1-4 header: 'runs in package install/upgrade context' - base/main/default/classes/setup/controller/PostInstallHandler.cls:169-173 doc: 'the script runs as a per-package system user that does not exist before runtime' - Salesforce: a post-install script 'runs as a special system user that represents your package' (search-result excerpt of the help page; page body itself not fetchable, so the quote is from the indexed snippet) - base/main/default/classes/setup/controller/PostInstallHandler.cls:183-186 onInstall calls each seed* step in its own try/catch; only logStepFailure on error - grep of non-test classes: seedDefaultSettings, seedDefaultImages, seedDefaultPageDesign, seedEventPricebook, seedEventDonationProduct, seedEventFeeProduct, seedContentTemplates have NO callers outside PostInstallHandler (mentions in PricebookService/SettingsService are comments/strings only) - Class is public, not global (changed 2026-09-05), so subscriber anonymous Apex / other namespaces cannot invoke these methods - Only backfills when the field is blank ('only backfills the Settings lookup when blank, so an admin's own configuration is never overwritten on upgrade' - method Javadoc) - Value is the Id of a record this same method just found or inserted; no user input SOURCES https://help.salesforce.com/s/articleView?id=sf.apex_post_install_script.htm&language=en_US&type=5

Reported vulnerability	Location	Response
R-026 FLS Create False positive	Paths 22, 46 At scan: classes/PostInstallHandler.cls:1601 Submitted source: classes/setup/controller/PostInstallHandler.cls:1601	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permission before performing the Create operation on the Object and its fields. Flagged statement:</i> <pre>upsert as system s;</pre> WHY THE CLAIM IS FALSE The claim is false because the write is to the org-default row of the package's own custom setting, from the InstallHandler's system user with no permission set, storing an Id the install step itself created; the administrator's existing value is never overwritten. SUPPORTING DETAIL This records the Id of the event processing-fee product, found or created by the same install step, on the org-default custom setting row when that field is blank. It runs only from the InstallHandler as the package's system user and never overwrites an administrator's value. No user-supplied data reaches it. EVIDENCE - base/main/default/classes/setup/controller/PostInstallHandler.cls:18 public without sharing class PostInstallHandler implements InstallHandler; lines 1-4 header: 'runs in package install/upgrade context' - base/main/default/classes/setup/controller/PostInstallHandler.cls:169-173 doc: 'the script runs as a per-package system user that does not exist before runtime' - Salesforce: a post-install script 'runs as a special system user that represents your package' (search-result excerpt of the help page; page body itself not fetchable, so the quote is from the indexed snippet) - base/main/default/classes/setup/controller/PostInstallHandler.cls:183-186 onInstall calls each seed* step in its own try/catch; only logStepFailure on error - grep of non-test classes: seedDefaultSettings, seedDefaultImages, seedDefaultPageDesign, seedEventPricebook, seedEventDonationProduct, seedEventFeeProduct, seedContentTemplates have NO callers outside PostInstallHandler (mentions in PricebookService/SettingsService are comments/strings only) - Class is public, not global (changed 2026-09-05), so subscriber anonymous Apex / other namespaces cannot invoke these methods - Only backfills when the field is blank ('only backfills the Settings lookup when blank, so an admin's own configuration is never overwritten on upgrade' - method Javadoc) - Value is the Id of a record this same method just found or inserted; no user input SOURCES https://help.salesforce.com/s/articleView?id=sf.apex_post_install_script.htm&language=en_US&type=5

Reported vulnerability	Location	Response
R-027 FLS Create False positive	Paths 23, 25–27 At scan: <code>classes/PostInstallHandler.cls:260</code> Submitted source: <code>classes/setup/controller/PostInstallHandler.cls:260</code>	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permission before performing the Create operation on the Object and its fields.</i> Flagged statement: <code>upsert as system s;</code> WHY THE CLAIM IS FALSE The claim is false because the write seeds the org-default row of the package's own hierarchy custom setting from compiled constants, runs only in the InstallHandler as the package's system user (no permission set), and fills only blank fields; custom-setting DML in user mode requires Customize Application, which the install user does not hold, so a user-mode write would fail the install. SUPPORTING DETAIL This seeds the org-default row of the package's hierarchy custom setting with constant defaults during package install or upgrade, filling only blank fields. It runs only from the InstallHandler, which executes as the package's special system user; that user holds no permission set, so there is no user FLS to enforce. The method is not reachable from any controller, guest page or subscriber Apex. EVIDENCE - base/main/default/classes/setup/controller/PostInstallHandler.cls:18 <code>public without sharing class PostInstallHandler implements InstallHandler</code>; lines 1-4 header: 'runs in package install/upgrade context' - base/main/default/classes/setup/controller/PostInstallHandler.cls:169-173 doc: 'the script runs as a per-package system user that does not exist before runtime' - Salesforce: a post-install script 'runs as a special system user that represents your package' (search-result excerpt of the help page; page body itself not fetchable, so the quote is from the indexed snippet) - base/main/default/classes/setup/controller/PostInstallHandler.cls:183-186 <code>onInstall</code> calls each <code>seed*</code> step in its own try/catch; only <code>logStepFailure</code> on error - grep of non-test classes: <code>seedDefaultSettings</code>, <code>seedDefaultImages</code>, <code>seedDefaultPageDesign</code>, <code>seedEventPricebook</code>, <code>seedEventDonationProduct</code>, <code>seedEventFeeProduct</code>, <code>seedContentTemplates</code> have NO callers outside <code>PostInstallHandler</code> (mentions in <code>PricebookService/SettingsService</code> are comments/strings only) - Class is <code>public</code>, not <code>global</code> (changed 2026-09-05), so subscriber anonymous Apex / other namespaces cannot invoke these methods - base/main/default/classes/setup/controller/PostInstallHandler.cls:206-260 <code>seedDefaultSettings</code>: creates the org-default <code>Settings__c</code> only if absent (<code>SetupOwnerId = orgId</code>) and backfills blank colour/text defaults from constants; <code>adoptTributeMasterSwitch</code> reads Campaign WITH <code>SYSTEM_MODE</code> - All values are package constants or derived from existing org data; no user input SOURCES https://help.salesforce.com/s/articleView?id=sf.apex_post_install_script.htm&language=en_US&type=5

Reported vulnerability	Location	Response
R-028 FLS Create False positive	Paths 30–32 At scan: classes/SettingsService.cls:563 Submitted source: classes/setup/service/SettingsService.cls:563	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permission before performing the Create operation on the Object and its fields.</i> Flagged statement: Database.insert(cv, AccessLevel.SYSTEM_MODE); WHY THE CLAIM IS FALSE The claim is false because the insert runs during install as the package's system user, which holds no permission set, and copies the package's own static-resource image into ContentVersion with constant Title and PathOnClient; ContentVersion is a standard object a 2GP permission set cannot grant, and no subscriber data is involved. SUPPORTING DETAIL PackagedAssetSeeder copies the package's own static-resource images (default logo, hero image, social-share image) into ContentVersion so the public pages have a default; Title/PathOnClient are constants and VersionData is the packaged resource body. It runs in the post-install context, whose user holds no permission sets, and ContentVersion is a standard object a second-generation package permission set cannot grant. The interactive admin upload of a custom logo uses user mode instead. EVIDENCE - base/main/default/classes/setup/service/SettingsService.cls:543-594 - PackagedAssetSeeder, inner without sharing with rationale; content from DefaultOrgLogo/DefaultHeroImage/DefaultSocialShareImage - base/main/default/classes/setup/service/SettingsService.cls createLogoDistribution - USER_MODE for the admin upload path SOURCES https://help.salesforce.com/s/articleView?id=sf.apex_post_install_script.htm (post-install script runs as a package-specific system user) - AGENTS.md Change Management: 2GP strips standard-object grants (project finding 2026-08-16)

Reported vulnerability	Location	Response
R-029 FLS Create False positive	Paths 33–37 At scan: classes/SettingsService.cls:581 Submitted source: classes/setup/service/SettingsService.cls:581	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permission before performing the Create operation on the Object and its fields.</i> Flagged statement: Database.insert(cd, AccessLevel.SYSTEM_MODE); WHY THE CLAIM IS FALSE The claim is false because the insert runs during install as the package's system user, which holds no permission set, and publishes a public link for the file the previous statement created from a packaged static resource; ContentDistribution is a standard object a 2GP permission set cannot grant, and the public link is the feature (default images on unauthenticated donation pages and receipts), not a leak. SUPPORTING DETAIL This publishes a public link for the package's own default image, which must be publicly reachable because it is rendered on unauthenticated donation pages and in receipt emails. The file is created one line earlier from a packaged static resource, so no subscriber or user file can be published through this path. It runs in the post-install context (no permission sets) or from an administrator's reset-to-default action. EVIDENCE - base/main/default/classes/setup/service/SettingsService.cls:570-581 - ContentDistribution on the ContentVersion just inserted from a static resource SOURCES https://help.salesforce.com/s/articleView?id=sf.apex_post_install_script.htm (post-install script runs as a package-specific system user) - AGENTS.md Change Management: 2GP strips standard-object grants (project finding 2026-08-16)

Reported vulnerability	Location	Response
R-030 FLS Create False positive	Paths 38 At scan: classes/PostInstallHandler.cls:1503 Submitted source: classes/setup/controller/PostInstallHandler.cls:1503	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permission before performing the Create operation on the Object and its fields.</i> Flagged statement: <code>insert as system pb;</code> WHY THE CLAIM IS FALSE The claim is false because the insert runs only inside the package InstallHandler as the package's system user, which holds no permission set, and Pricebook2 is a standard object a 2GP permission set cannot grant; every value is a constant, so no user input reaches the write. SUPPORTING DETAIL This creates the package's event pricebook once, at install, with a constant name, only if it does not already exist. Pricebook2 is a standard object that a packaged permission set cannot grant, and the method runs only from the InstallHandler as the package's system user. No controller, guest page or subscriber code can call it. EVIDENCE - base/main/default/classes/setup/controller/PostInstallHandler.cls:18 <code>public without sharing class PostInstallHandler implements InstallHandler</code>; lines 1-4 header: 'runs in package install/upgrade context' - base/main/default/classes/setup/controller/PostInstallHandler.cls:169-173 doc: 'the script runs as a per-package system user that does not exist before runtime' - Salesforce: a post-install script 'runs as a special system user that represents your package' (search-result excerpt of the help page; page body itself not fetchable, so the quote is from the indexed snippet) - base/main/default/classes/setup/controller/PostInstallHandler.cls:183-186 onInstall calls each seed* step in its own try/catch; only logStepFailure on error - grep of non-test classes: seedDefaultSettings, seedDefaultImages, seedDefaultPageDesign, seedEventPricebook, seedEventDonationProduct, seedEventFeeProduct, seedContentTemplates have NO callers outside PostInstallHandler (mentions in PricebookService/SettingsService are comments/strings only) - Class is <code>public</code>, not <code>global</code> (changed 2026-09-05), so subscriber anonymous Apex / other namespaces cannot invoke these methods - Pricebook2 and Product2 are standard objects; AGENTS.md / readiness FP-1 item 2 record that a 2GP permission set cannot ship a standard-object grant (build strips it silently), and the install user holds no permission set anyway - base/main/default/classes/event/service/PricebookService.cls:11-24 doc: install-testing (2026-08-09) showed a <code>with sharing</code> path could not even resolve Pricebook2 as the per-package system user - base/main/default/classes/setup/controller/PostInstallHandler.cls:1501 <code>new Pricebook2(Name = EVENT_PRICEBOOK_NAME, IsActive = true)</code> - constant values; inserted only when neither the current nor the legacy pricebook exists SOURCES

Reported vulnerability	Location	Response
		• https://help.salesforce.com/s/articleView?id=sf.apex_post_install_script.htm&language=en_US&type=5

Reported vulnerability	Location	Response
R-031 FLS Create False positive	Paths 39 At scan: classes/PostInstallHandler.cls:1537 Submitted source: classes/setup/controller/PostInstallHandler.cls:1537	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permission before performing the Create operation on the Object and its fields.</i> Flagged statement: <pre>insert as system p;</pre> WHY THE CLAIM IS FALSE The claim is false because the insert runs only inside the package InstallHandler as the package's system user, which holds no permission set, and Product2 is a standard object a 2GP permission set cannot grant; every field value is a compiled constant. SUPPORTING DETAIL This creates the package's generic event donation product once, at install, with constant field values and only if it does not already exist. Product2 is a standard object that a packaged permission set cannot grant, and the code runs only from the InstallHandler as the package's system user. It is not reachable from any user-facing entry point. EVIDENCE - base/main/default/classes/setup/controller/PostInstallHandler.cls:18 <code>public without sharing class PostInstallHandler implements InstallHandler</code>; lines 1-4 header: 'runs in package install/upgrade context' - base/main/default/classes/setup/controller/PostInstallHandler.cls:169-173 doc: 'the script runs as a per-package system user that does not exist before runtime' - Salesforce: a post-install script 'runs as a special system user that represents your package' (search-result excerpt of the help page; page body itself not fetchable, so the quote is from the indexed snippet) - base/main/default/classes/setup/controller/PostInstallHandler.cls:183-186 onInstall calls each seed* step in its own try/catch; only logStepFailure on error - grep of non-test classes: seedDefaultSettings, seedDefaultImages, seedDefaultPageDesign, seedEventPricebook, seedEventDonationProduct, seedEventFeeProduct, seedContentTemplates have NO callers outside PostInstallHandler (mentions in PricebookService/SettingsService are comments/strings only) - Class is <code>public</code>, not <code>global</code> (changed 2026-09-05), so subscriber anonymous Apex / other namespaces cannot invoke these methods - Pricebook2 and Product2 are standard objects; AGENTS.md / readiness FP-1 item 2 record that a 2GP permission set cannot ship a standard-object grant (build strips it silently), and the install user holds no permission set anyway - base/main/default/classes/event/service/PricebookService.cls:11-24 doc: install-testing (2026-08-09) showed a <code>with sharing</code> path could not even resolve Pricebook2 as the per-package system user - base/main/default/classes/setup/controller/PostInstallHandler.cls:1534 <code>new Product2(Name = EVENT_DONATION_PRODUCT_NAME, IsActive = true)</code> - constants; only when no Product2 with that name exists (query WITH SYSTEM_MODE) SOURCES

Reported vulnerability	Location	Response
		• https://help.salesforce.com/s/articleView?id=sf.apex_post_install_script.htm&language=en_US&type=5

Reported vulnerability	Location	Response
R-032 FLS Create False positive	Paths 40 At scan: classes/PostInstallHandler.cls:1581 Submitted source: classes/setup/controller/PostInstallHandler.cls:1581	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permission before performing the Create operation on the Object and its fields.</i> Flagged statement: <code>insert as system p;</code> WHY THE CLAIM IS FALSE The claim is false because the insert runs only inside the package InstallHandler as the package's system user, which holds no permission set, and Product2 is a standard object a 2GP permission set cannot grant; the values are constants. SUPPORTING DETAIL This creates the package's event processing-free product once, at install, with constant values and only if it does not already exist. Product2 is a standard object a packaged permission set cannot grant, and the method runs only from the InstallHandler as the package's system user. It has no interactive or guest caller. EVIDENCE - base/main/default/classes/setup/controller/PostInstallHandler.cls:18 <code>public without sharing class PostInstallHandler implements InstallHandler</code>; lines 1-4 header: 'runs in package install/upgrade context' - base/main/default/classes/setup/controller/PostInstallHandler.cls:169-173 doc: 'the script runs as a per-package system user that does not exist before runtime' - Salesforce: a post-install script 'runs as a special system user that represents your package' (search-result excerpt of the help page; page body itself not fetchable, so the quote is from the indexed snippet) - base/main/default/classes/setup/controller/PostInstallHandler.cls:183-186 onInstall calls each seed* step in its own try/catch; only logStepFailure on error - grep of non-test classes: seedDefaultSettings, seedDefaultImages, seedDefaultPageDesign, seedEventPricebook, seedEventDonationProduct, seedEventFeeProduct, seedContentTemplates have NO callers outside PostInstallHandler (mentions in PricebookService/SettingsService are comments/strings only) - Class is <code>public</code>, not <code>global</code> (changed 2026-09-05), so subscriber anonymous Apex / other namespaces cannot invoke these methods - Pricebook2 and Product2 are standard objects; AGENTS.md / readiness FP-1 item 2 record that a 2GP permission set cannot ship a standard-object grant (build strips it silently), and the install user holds no permission set anyway - base/main/default/classes/event/service/PricebookService.cls:11-24 doc: install-testing (2026-08-09) showed a <code>with sharing</code> path could not even resolve Pricebook2 as the per-package system user - base/main/default/classes/setup/controller/PostInstallHandler.cls:1578 <code>new Product2(Name = EVENT_FEE_PRODUCT_NAME, IsActive = true)</code> - constants; only when absent SOURCES https://help.salesforce.com/s/articleView?id=sf.apex_post_install_script.htm&language=en_US&type=5

Reported vulnerability	Location	Response
R-033 FLS Create False positive	Paths 42 At scan: classes/EventPurchaseService.cls:590 Submitted source: classes/event/service/EventPurchaseService.cls:590	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permisison before performing the Create operation on the Object and its fields.</i> Flagged statement: insert as system toAdd; // bulk – never per-attendee DML WHY THE CLAIM IS FALSE The claim is false because the insert runs in the reconciliation job that finalizes a paid ticket order, a context the scan's own rule exempts, and CampaignMember is a standard object a 2GP permission set cannot grant; the Contact and Campaign Ids come from the finalized order, not from a caller. SUPPORTING DETAIL After a paid ticket order is finalized by the reconciliation job, attendees with a Contact are added to the event Campaign as CampaignMembers. CampaignMember is a standard object that a second-generation package permission set cannot grant, and the finalizer runs in scheduled context, so the insert runs in system mode. The Campaign comes from the server-side staging record and the Contacts from the attendees created in the same finalization; existing members are skipped, and no browser input chooses either. EVIDENCE - base/main/default/classes/event/service/EventPurchaseService.cls:570-590 - campaign from staging, contacts from attendees, existing members excluded - Comment cites 'CampaignMemberSyncService.syncForOpportunities', which does not exist (method is syncForDonations); 'after a guest order is committed' - it actually runs in the batch/retry finalizer SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_batch_interface.htm (batch Apex runs asynchronously; Salesforce false-positive guidance exempts batch and async context) - AGENTS.md Change Management: 2GP strips standard-object grants (project finding 2026-08-16)

Reported vulnerability	Location	Response
R-034 FLS Create False positive	Paths 47 At scan: classes/CampaignMemberSyncService.cls:80 Submitted source: classes/donation/service/CampaignMemberSyncService.cls:80	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permission before performing the Create operation on the Object and its fields.</i> Flagged statement: <pre>List<Database.SaveResult> results = Database.insert(toAdd, false, AccessLevel.SYSTEM_MODE);</pre> WHY THE CLAIM IS FALSE The claim is false because the insert runs inside the platform-event subscriber trigger as the Automated Process entity, which can hold no permission set, and CampaignMember is a standard object a 2GP permission set cannot grant; the Contact and Campaign Ids come from the finalized donation itself, not from a caller. SUPPORTING DETAIL This adds the donor to the donation's campaign as a CampaignMember after a gift is finalized; it runs in the platform-event subscriber trigger, which executes as the Automated Process entity. CampaignMember is a standard object that a second-generation package permission set cannot grant, and the Automated Process context has no user permissions to enforce. CampaignId and ContactId are read from the finalized Opportunity itself, and existing memberships are skipped, so the insert cannot target arbitrary campaigns or contacts. EVIDENCE - base/main/default/classes/donation/service/CampaignMemberSyncService.cls:68 - CampaignMember built from Opportunity CampaignId + resolved contact - base/main/default/classes/donation/service/CampaignMemberSyncService.cls:80 - comment present and accurate SOURCES https://developer.salesforce.com/docs/atlas.en-us.platform_events.meta/platform_events/platform_events_subscribe_apex.htm - AGENTS.md Change Management: 2GP strips standard-object grants (project finding 2026-08-16)

Reported vulnerability	Location	Response
R-035 FLS Create False positive	Paths 55, 80, 83–87 At scan: classes/PostInstallHandler.cls:1645 Submitted source: classes/setup/controller/PostInstallHandler.cls:1645	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permission before performing the Create operation on the Object and its fields.</i> Flagged statement: <code>insert as system toInsert;</code> WHY THE CLAIM IS FALSE The claim is false because the insert runs only inside the package InstallHandler as the package's system user, which holds no permission set, and the rows are starter templates whose text is literal in the package source; no user supplies any value and no user's permissions exist to check. SUPPORTING DETAIL This inserts a starter set of question and FAQ templates, whose text is literal in the package source, the first time the package is installed and only when the template table is empty. It runs only from the InstallHandler as the package's system user, which holds no permission set. Administrators edit or delete these templates afterwards through the normal, permission-checked console. EVIDENCE - base/main/default/classes/setup/controller/PostInstallHandler.cls:18 <code>public without sharing class PostInstallHandler implements InstallHandler</code>; lines 1-4 header: 'runs in package install/upgrade context' - base/main/default/classes/setup/controller/PostInstallHandler.cls:169-173 doc: 'the script runs as a per-package system user that does not exist before runtime' - Salesforce: a post-install script 'runs as a special system user that represents your package' (search-result excerpt of the help page; page body itself not fetchable, so the quote is from the indexed snippet) - base/main/default/classes/setup/controller/PostInstallHandler.cls:183-186 onInstall calls each seed* step in its own try/catch; only logStepFailure on error - grep of non-test classes: seedDefaultSettings, seedDefaultImages, seedDefaultPageDesign, seedEventPricebook, seedEventDonationProduct, seedEventFeeProduct, seedContentTemplates have NO callers outside PostInstallHandler (mentions in PricebookService/SettingsService are comments/strings only) - Class is <code>public</code>, not <code>global</code> (changed 2026-09-05), so subscriber anonymous Apex / other namespaces cannot invoke these methods - base/main/default/classes/setup/controller/PostInstallHandler.cls:1615-1672 seedContentTemplates inserts starter question/FAQ Content_Template__c rows (RecordTypeId, Name, Prompt__c, Question_Type__c, Is_Active__c, Picklist_Options__c, Answer__c) only when the table is empty; all text is literal in source SOURCES https://help.salesforce.com/s/articleView?id=sf.apex_post_install_script.htm&language=en_US&type=5

Reported vulnerability	Location	Response
R-036 FLS Create False positive	Paths 62, 66, 73, 76, 79, 82, 89, 91, 93, 95, 97, 99, 101–109 At scan: classes/PostInstallHandler.cls:758 Submitted source: classes/setup/controller/PostInstallHandler.cls:758	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permission before performing the Create operation on the Object and its fields.</i> Flagged statement: <code>upsert as system toUpsert;</code> WHY THE CLAIM IS FALSE The claim is false because the upsert creates or refreshes the six packaged theme records from constants compiled into the package, runs only in the InstallHandler as the package's system user with no permission set, and cannot be invoked by any user at runtime. SUPPORTING DETAIL This creates or refreshes the six packaged page-design theme records from constants compiled into the package, during install or upgrade. It runs only from the InstallHandler as the package's system user, which holds no permission set, and every assigned value is a literal from the package source. Packaged themes are read-only to administrators, who clone them to customise, so no administrator data is overwritten. EVIDENCE - base/main/default/classes/setup/controller/PostInstallHandler.cls:18 <code>public without sharing class PostInstallHandler implements InstallHandler</code>; lines 1-4 header: 'runs in package install/upgrade context' - base/main/default/classes/setup/controller/PostInstallHandler.cls:169-173 doc: 'the script runs as a per-package system user that does not exist before runtime' - Salesforce: a post-install script 'runs as a special system user that represents your package' (search-result excerpt of the help page; page body itself not fetchable, so the quote is from the indexed snippet) - base/main/default/classes/setup/controller/PostInstallHandler.cls:183-186 <code>onInstall</code> calls each <code>seed*</code> step in its own <code>try/catch</code>; only <code>logStepFailure</code> on error - grep of non-test classes: <code>seedDefaultSettings</code>, <code>seedDefaultImages</code>, <code>seedDefaultPageDesign</code>, <code>seedEventPricebook</code>, <code>seedEventDonationProduct</code>, <code>seedEventFeeProduct</code>, <code>seedContentTemplates</code> have NO callers outside <code>PostInstallHandler</code> (mentions in <code>PricebookService/SettingsService</code> are comments/strings only) - Class is <code>public</code>, not <code>global</code> (changed 2026-09-05), so subscriber anonymous Apex / other namespaces cannot invoke these methods - base/main/default/classes/setup/controller/PostInstallHandler.cls:687-758 <code>seedDefaultPageDesign</code> upserts the six packaged <code>Campaign_Design__c</code> theme rows; <code>buildTheme</code> (:1196+) assigns colours, fonts, radius, widths from compile-time theme constants; reads <code>WITH SYSTEM_MODE</code>; followed by <code>retireDeletedDesigns</code> - Packaged themes are read-only in the console (clone to edit), so overwriting their style fields on upgrade is the intended contract SOURCES https://help.salesforce.com/s/articleView?id=sf.apex_post_install_script.htm&language=en_US&type=5

Reported vulnerability	Location	Response
R-037 FLS Create False positive	Paths 69, 71 At scan: classes/PricebookService.cls:125 Submitted source: classes/event/service/PricebookService.cls:125	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permisison before performing the Create operation on the Object and its fields.</i> Flagged statement: <code>if (!newStd.isEmpty()) { insert as system newStd; } // MUST precede the custom-pricebook insert below</code> WHY THE CLAIM IS FALSE The claim is false because PricebookEntry is a standard object a 2GP permission set cannot grant, and the insert runs from install (package system user, no permission set), admin ticket-tier saves and guest purchases, where the Guest licence cannot hold Create; every value is fixed by Apex from a Product2 the package itself created. SUPPORTING DETAIL This creates the standard pricebook PricebookEntry a ticket Product2 needs before an OpportunityLineItem can reference it. Values are fixed by Apex (the Product2 the package created for the ticket tier, the pricebook Id, a placeholder unit price, IsActive=true, the org currency) - the real price lives on the line item. PricebookEntry/Pricebook2 are standard objects the package permission sets cannot grant, and the same method runs in the post-install context and the scheduled finalizer; the staff entry point first verifies the user can edit the event Campaign and that the tier belongs to it. EVIDENCE - base/main/default/classes/event/service/PricebookService.cls - class without sharing with doc rationale (Pricebook2 unresolvable under with sharing in post-install context) - base/main/default/classes/event/service/PricebookService.cls:142-143 - PricebookEntry(Product2Id, Pricebook2Id, UnitPrice=PLACEHOLDER_PRICE, IsActive=true) - base/main/default/classes/event/controller/EventTicketAdminController.cls:156-176, 197-263 - requireCampaignWritable + requireTiersBelongToCampaign before ensureEntriesFor; wrapped non-fatal SOURCES - AGENTS.md Change Management: 2GP strips standard-object grants (project finding 2026-08-16)

Reported vulnerability	Location	Response
R-038 FLS Create False positive	Paths 70, 72 At scan: classes/PricebookService.cls:135 Submitted source: classes/event/service/PricebookService.cls:135	SCANNER'S CLAIM <i>This query looks for CRUD Delete operations that are performed without checking for isCreatable() permisison before performing the Create operation on the Object and its fields.</i> Flagged statement: insert as system newEvt; WHY THE CLAIM IS FALSE The claim is false because PricebookEntry is a standard object a 2GP permission set cannot grant, and the insert runs from install (package system user, no permission set), admin ticket-tier saves and guest purchases, where the Guest licence cannot hold Create; every value is fixed by Apex from a Product2 the package itself created. SUPPORTING DETAIL This creates the package event pricebook PricebookEntry a ticket Product2 needs before an OpportunityLineItem can reference it. Values are fixed by Apex (the Product2 the package created for the ticket tier, the pricebook Id, a placeholder unit price, IsActive=true, the org currency) - the real price lives on the line item. PricebookEntry/Pricebook2 are standard objects the package permission sets cannot grant, and the same method runs in the post-install context and the scheduled finalizer; the staff entry point first verifies the user can edit the event Campaign and that the tier belongs to it. EVIDENCE - base/main/default/classes/event/service/PricebookService.cls - class without sharing with doc rationale (Pricebook2 unresolvable under with sharing in post-install context) - base/main/default/classes/event/service/PricebookService.cls:142-143 - PricebookEntry(Product2Id, Pricebook2Id, UnitPrice=PLACEHOLDER_PRICE, IsActive=true) - base/main/default/classes/event/controller/EventTicketAdminController.cls:156-176, 197-263 - requireCampaignWritable + requireTiersBelongToCampaign before ensureEntriesFor; wrapped non-fatal SOURCES - AGENTS.md Change Management: 2GP strips standard-object grants (project finding 2026-08-16)

FLS Update — 69 paths, 14 locations

What this query claims: *This query looks for CRUD Update operations that are performed without checking for isUpdateable() permisison before performing the Update operation on the Object and its fields.*

Reported vulnerability	Location	Response
R-039 FLS Update False positive	Paths 1 At scan: classes/StagingReconciliationBatch.cls:275 Submitted source: classes/donation/async/StagingReconciliationBatch.cls:275	SCANNER'S CLAIM <i>This query looks for CRUD Update operations that are performed without checking for isUpdateable() permission before performing the Update operation on the Object and its fields.</i> Flagged statement: update as system upd; WHY THE CLAIM IS FALSE The claim is false because the update runs in a scheduled batch (and the equivalent staff 'Retry now' path that re-reads the gateway), writing only the caught exception message and the gateway's own payment status onto a guest-created staging row that no running user is granted Edit on; the values never come from the caller. SUPPORTING DETAIL The reconciler writes the finalization error and, for an unpaid staging past its TTL, sets Status__c='Expired'; the values come from the caught exception and Stripe's PaymentIntent status, never from the caller. It runs in a scheduled batch and in an internal 'retry now' action; Fundraising_User can view and retry stagings but deliberately has no Edit on Donation_Staging__c (the row is system-owned state), so the status transition must run in system mode. The only caller input is the staging Id on the retry path, and the action it triggers is the same idempotent reconciliation the scheduler would perform anyway. EVIDENCE - base/main/default/classes/donation/async/StagingReconciliationBatch.cls:260-276 - upd built from e.getMessage() and pi.status; Expired only when past TTL and payment not succeeded - base/main/default/permissionsets/Fundraising_User - Donation_Staging__c Create/Read, no Edit - base/main/default/classes/donation/staging/DonationStagingSelector.cls selectById - SYSTEM_MODE read, so retry has no record-visibility gate (minor; controller granted only to staff sets) SOURCES - unverified against primary docs (developer.salesforce.com apex_scheduler.htm returned 403 / cookie wall); secondary: https://help.salesforce.com/s/articleView?id=sf.code_schedule_batch_apex.htm (search snippet: scheduled Apex executes under the user who scheduled the job)
R-016 FLS Update False positive	Paths 2, 12–21, 26–27, 30–31, 35, 37–38, 41–42, 44, 46, 48, 50, 52, 54, 56, 58, 60 At scan: classes/SettingsService.cls:345 Submitted source: classes/setup/service/SettingsService.cls:345	See R-016 under “Apex CRUD ContentDistribution” — same code location, same response.

Reported vulnerability	Location	Response
R-022 FLS Update False positive	Paths 3–4 At scan: classes/RefundReconciliationBatch.cls:344 Submitted source: classes/donation/async/RefundReconciliationBatch.cls:344	See R-022 under “FLS Create” — same code location, same response.
R-040 FLS Update False positive	Paths 5 At scan: classes/DonorPortalService.cls:383 Submitted source: classes/donor/service/DonorPortalService.cls:383	SCANNER’S CLAIM <i>This query looks for CRUD Update operations that are performed without checking for isUpdateable() permisison before performing the Update operation on the Object and its fields.</i> Flagged statement: Database.update(bump, AccessLevel.SYSTEM_MODE); WHY THE CLAIM IS FALSE The claim is false because the write increments the failed-attempt counter on a portal access code and locks it, a brute-force defence that must run as the Experience Cloud guest user; the Guest User licence cannot hold Edit on any object, and the credential table carries no guest grant at all, so a user-mode update would disable the lockout. SUPPORTING DETAIL This increments the failed-attempt counter on a donor-portal access code and locks it after the maximum attempts - a brute-force defence. The guest user cannot hold Edit on any object, and the credential table carries no guest grant at all, so the write must be system mode. The row is resolved server-side from the submitted email under FOR UPDATE, never from a client-supplied Id, and the only fields written are Attempt_Count__c and Status__c='Locked', both computed by Apex. The worst a caller can do is lock a code, which the legitimate donor replaces by requesting a new one. EVIDENCE - base/main/default/classes/donor/service/DonorPortalService.cls:367-383 - bump built from selector row; attempts computed server-side - base/main/default/classes/donor/service/DonorPortalService.cls class header - without sharing with rationale - Portal_Access_Token__c has no Fundraising_GuestDonor grant (permset doc comment) - ControllerAction.run has no savepoint rollback on caught error, so the increment persists after a failed verify (intended) SOURCES - project memory/platform limit: Guest User licence cannot be granted object Edit/Delete (AGENTS.md guest-user notes) - unverified against a primary URL in this pass

Reported vulnerability	Location	Response
R-023 FLS Update False positive	Paths 6–7 At scan: classes/SettingsService.cls:1026 Submitted source: classes/setup/service/SettingsService.cls:1026	See R-023 under “FLS Create” — same code location, same response.

Reported vulnerability	Location	Response
R-041 FLS Update False positive	Paths 8 At scan: classes/PostInstallHandler.cls:1498 Submitted source: classes/setup/controller/PostInstallHandler.cls:1498	SCANNER'S CLAIM <i>This query looks for CRUD Update operations that are performed without checking for isUpdateable() permission before performing the Update operation on the Object and its fields.</i> Flagged statement: update as system legacy; WHY THE CLAIM IS FALSE The claim is false because the statement runs only inside the package InstallHandler, which executes as a package-specific system user that holds no permission set, so there is no user whose isUpdateable() could be consulted; and Pricebook2 is a standard object that a second-generation package permission set cannot grant, so no permission check could ever be satisfied by package metadata. SUPPORTING DETAIL This renames the package's own event pricebook from its earlier name to the current one during upgrade, so existing price book entries are kept. Pricebook2 is a standard object that a packaged permission set cannot grant, and the code runs only from the InstallHandler as the package's system user. The new name is a constant; nothing user-supplied is written. EVIDENCE - base/main/default/classes/setup/controller/PostInstallHandler.cls:18 public without sharing class PostInstallHandler implements InstallHandler; lines 1-4 header: 'runs in package install/upgrade context' - base/main/default/classes/setup/controller/PostInstallHandler.cls:169-173 doc: 'the script runs as a per-package system user that does not exist before runtime' - Salesforce: a post-install script 'runs as a special system user that represents your package' (search-result excerpt of the help page; page body itself not fetchable, so the quote is from the indexed snippet) - base/main/default/classes/setup/controller/PostInstallHandler.cls:183-186 onInstall calls each seed* step in its own try/catch; only logStepFailure on error - grep of non-test classes: seedDefaultSettings, seedDefaultImages, seedDefaultPageDesign, seedEventPricebook, seedEventDonationProduct, seedEventFeeProduct, seedContentTemplates have NO callers outside PostInstallHandler (mentions in PricebookService/SettingsService are comments/strings only) - Class is public, not global (changed 2026-09-05), so subscriber anonymous Apex / other namespaces cannot invoke these methods - Pricebook2 and Product2 are standard objects; AGENTS.md / readiness FP-1 item 2 record that a 2GP permission set cannot ship a standard-object grant (build strips it silently), and the install user holds no permission set anyway - base/main/default/classes/event/service/PricebookService.cls:11-24 doc: install-testing (2026-08-09) showed a with sharing path could not even resolve Pricebook2 as the per-package system user - base/main/default/classes/setup/controller/PostInstallHandler.cls:1477-1499 seedEventPricebook: if only the pre-rename pricebook exists, renames it in place to EVENT_PRICEBOOK_NAME ('Fundraising Events') so its PricebookEntry rows survive; the new name is a constant

Reported vulnerability	Location	Response
		SOURCES • https://help.salesforce.com/s/articleView?id=sf.apex_post_install_script.htm&language=en_US&type=5
R-024 FLS Update False positive	Paths 9 At scan: classes/PostInstallHandler.cls:1515 Submitted source: classes/setup/controller/PostInstallHandler.cls:1515	See R-024 under “FLS Create” — same code location, same response.
R-025 FLS Update False positive	Paths 10 At scan: classes/PostInstallHandler.cls:1557 Submitted source: classes/setup/controller/PostInstallHandler.cls:1557	See R-025 under “FLS Create” — same code location, same response.
R-026 FLS Update False positive	Paths 11 At scan: classes/PostInstallHandler.cls:1601 Submitted source: classes/setup/controller/PostInstallHandler.cls:1601	See R-026 under “FLS Create” — same code location, same response.

Reported vulnerability	Location	Response
R-042 FLS Update False positive	Paths 22–23, 32 At scan: classes/AttendeeNamingHandler.cls:129 Submitted source: classes/guestrequest/handler/AttendeeNamingHandler.cls:129	SCANNER'S CLAIM <i>This query looks for CRUD Update operations that are performed without checking for isUpdateable() permission before performing the Update operation on the Object and its fields.</i> Flagged statement: Database.SaveResult[] results = Database.update(toUpdate, false, AccessLevel.SYSTEM_MODE); WHY THE CLAIM IS FALSE The claim is false because the write runs asynchronously from the request queue, after ownership of the ticket order was verified synchronously against the donor's portal session, and only the attendee-name fields of that order's own seats are written; the running user is the guest user or the sweep's scheduler, neither of whom can hold Edit on the object. SUPPORTING DETAIL A donor who is signed in to the donor portal with a verified access code may name the blank guest seats on their own ticket order. Ownership is checked synchronously before anything is queued (requireOwnedOrder against the portal session), the verified order Id is stored server-side in Subject_Id__c, and this handler re-reads the seats of that order only - seat Ids in the payload that are not on the order, cancelled seats, and seats already named are ignored. The only fields written are First_Name__c, Last_Name__c and (when blank) Email__c, length-clipped and email-validated, which are exactly the data the donor is entitled to supply; the Guest licence cannot hold Edit, hence system mode. EVIDENCE - base/main/default/classes/donor/service/DonorPortalService.cls:1231-1283 - requireOwnedOrder(ctx, orderId); payload rebuilt from owned seats, never echoed from the browser - base/main/default/classes/guestrequest/handler/AttendeeNamingHandler.cls:83-117 - seats re-read from Subject_Id__c order; unmatched/cancelled/already-named seats skipped; NAME_MAX/EMAIL_MAX - base/main/default/classes/guestrequest/handler/AttendeeNamingHandler.cls:10-14 - without sharing with rationale - base/main/default/classes/guestrequest/handler/AttendeeNamingHandler.cls:19 - MAX_SEATS payload cap SOURCES - unverified in this pass: Guest User licence cannot hold object Edit (project-documented platform limit)

Reported vulnerability	Location	Response
R-043 FLS Update False positive	Paths 24, 33 At scan: classes/GuestRequestService.cls:138 Submitted source: classes/guestrequest/service/GuestRequestService.cls:138	SCANNER'S CLAIM <i>This query looks for CRUD Update operations that are performed without checking for isUpdateable() permission before performing the Update operation on the Object and its fields.</i> Flagged statement: Database.update(claimed, false, AccessLevel.SYSTEM_MODE); WHY THE CLAIM IS FALSE The claim is false because the write claims queued requests (status and attempt counter only) so the queueable and the sweep never process a row twice; it runs in the guest user's own queueable, where the Guest User licence cannot hold Edit on any object, or in the scheduled sweep, so a user-mode update could never succeed. SUPPORTING DETAIL This claims queued public requests (Status__c Pending -> Processing, Attempt_Count__c + 1) so the queueable and the sweep never process the same row twice. It runs in the guest user's own queueable - the Guest licence cannot hold Edit on any object - or in the scheduled sweep; the request Ids come from the row the server just inserted or from the sweep's query, and only rows still Pending are claimed. Both written values are computed by Apex. EVIDENCE - base/main/default/classes/guestrequest/service/GuestRequestService.cls:120-138 - only Pending rows; status/attempts computed - base/main/default/classes/guestrequest/async/GuestRequestQueueable.cls:31, base/main/default/classes/guestrequest/async/GuestRequestSweepBatch.cls:48 - sole callers of process() - Fundraising_GuestDonor has no Edit on Guest_Request__c SOURCES - unverified against primary docs (developer.salesforce.com apex_scheduler.htm returned 403 / cookie wall); secondary: https://help.salesforce.com/s/articleView?id=sf.code_schedule_batch_apex.htm (search snippet: scheduled Apex executes under the user who scheduled the job)

Reported vulnerability	Location	Response
R-044 FLS Update False positive	Paths 25, 34, 39 At scan: classes/GuestRequestService.cls:237 Submitted source: classes/guestrequest/service/GuestRequestService.cls:237	SCANNER'S CLAIM <i>This query looks for CRUD Update operations that are performed without checking for isUpdateable() permission before performing the Update operation on the Object and its fields. Flagged statement:</i> <pre>Database.update(resolving, false, AccessLevel.SYSTEM_MODE);</pre> WHY THE CLAIM IS FALSE The claim is false because the write closes the donor's own refund request only after the refund itself succeeded under the refund controller's Process_Refunds authorization; it selects rows by the refunded gift's Id and writes only a status, and the staff user is not granted Edit on the guest-request object by design, so a user-mode update would leave the portal showing a completed refund as pending. SUPPORTING DETAIL After staff refund a gift, this closes the donor's matching refund request that was parked for review so the donor portal stops showing it as pending. It runs only after the refund itself succeeded under the refund controller's own authorization, it selects requests by the refunded Opportunity Id and only those in Awaiting_Review of the refund type, and it writes a fixed status, a fixed donor-facing sentence and a timestamp. The request rows are created by the guest user and owned by the site's default owner, so an internal user generally has no sharing on them; system mode is what lets the refund close its own request. EVIDENCE - base/main/default/classes/donation/service/RefundService.cls:352-357 - sole caller, constant status/message - base/main/default/classes/guestrequest/service/GuestRequestService.cls:214-238 - filters to Awaiting_Review + type - Fundraising_User has Edit but not View All on Guest_Request__c; Admin has Modify All - so the real blocker for a user-mode update is record sharing, not the object grant SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm (user-mode DML requires the running user's Edit permission on the object) https://help.salesforce.com/s/articleView?id=sf.perm_sets_custom.htm (custom permission Process_Refunds is the authorization the refund controller enforces; unverified exact URL)

Reported vulnerability	Location	Response
R-045 FLS Update False positive	Paths 28 At scan: classes/PostInstallHandler.cls:409 Submitted source: classes/setup/controller/PostInstallHandler.cls:409	SCANNER'S CLAIM <i>This query looks for CRUD Update operations that are performed without checking for isUpdateable() permission before performing the Update operation on the Object and its fields.</i> Flagged statement: <code>if (!toUpdate.isEmpty()) { update as system toUpdate; }</code> WHY THE CLAIM IS FALSE The claim is false because the update runs only inside the InstallHandler as the package's system user, which holds no permission set, touches only the package's own theme records (selected by their packaged names) and writes a URL computed from the org's own site settings; no user input and no user permissions are involved. SUPPORTING DETAIL This backfills the default hero image URL on the package's own theme records during install or upgrade, limited to the packaged theme names. It runs only from the InstallHandler as the package's system user, which has no permission set, and the value is a URL the install step itself generated from a packaged static resource. No interactive or guest caller reaches it. EVIDENCE - base/main/default/classes/setup/controller/PostInstallHandler.cls:18 <code>public without sharing class PostInstallHandler implements InstallHandler</code>; lines 1-4 header: 'runs in package install/upgrade context' - base/main/default/classes/setup/controller/PostInstallHandler.cls:169-173 doc: 'the script runs as a per-package system user that does not exist before runtime' - Salesforce: a post-install script 'runs as a special system user that represents your package' (search-result excerpt of the help page; page body itself not fetchable, so the quote is from the indexed snippet) - base/main/default/classes/setup/controller/PostInstallHandler.cls:183-186 <code>onInstall</code> calls each <code>seed*</code> step in its own <code>try/catch</code>; only <code>logStepFailure</code> on error - grep of non-test classes: <code>seedDefaultSettings</code>, <code>seedDefaultImages</code>, <code>seedDefaultPageDesign</code>, <code>seedEventPricebook</code>, <code>seedEventDonationProduct</code>, <code>seedEventFeeProduct</code>, <code>seedContentTemplates</code> have NO callers outside <code>PostInstallHandler</code> (mentions in <code>PricebookService/SettingsService</code> are comments/strings only) - Class is <code>public</code>, not <code>global</code> (changed 2026-09-05), so subscriber anonymous Apex / other namespaces cannot invoke these methods - base/main/default/classes/setup/controller/PostInstallHandler.cls:383-409 <code>seedDefaultImages</code>: copies the seeded default hero image URL from <code>Settings__c</code> onto <code>Campaign_Design__c.Default_Hero_Image_URL__c</code> for the packaged theme names only, and only when changed; the query is <code>WITH SYSTEM_MODE</code> - Value originates from the packaged static-resource distribution URL seeded by <code>SettingsService.ensureDefaultImagesSeeded</code>, not user input SOURCES https://help.salesforce.com/s/articleView?id=sf.apex_post_install_script.htm&language=en_US&type=5

Reported vulnerability	Location	Response
R-036 FLS Update False positive	Paths 29, 36, 40, 43, 45, 47, 49, 51, 53, 55, 57, 59, 61–69 At scan: classes/PostInstallHandler.cls:758 Submitted source: classes/setup/controller/PostInstallHandler.cls:758	See R-036 under “FLS Create” — same code location, same response.

Apex SOQL SOSL USER MODE MISSING — 29 paths, 20 locations

What this query claims: *USER_MODE* is recommended for SOQL or SOSL operations

Reported vulnerability	Location	Response
R-046 Apex SOQL SOSL USER MODE MISSING False positive	Paths 1 At scan: classes/CurrencyResolver.cls:265 Submitted source: classes/shared/CurrencyResolver.cls:265	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: Database.queryWithBinds(soql, new Map<String, Object>{ 'recordIds' => recordIds }, AccessLevel.SYSTEM_MODE), WHY THE CLAIM IS FALSE The claim is false because the query reads only Id and CurrencyIsoCode for parent records whose Ids a trigger-driven rollup already holds, in order to convert amounts before writing the rollup total; the object name comes from the Id's schema token, not from input, and the read runs in trigger context as the DML writer, where a user without the parent's currency field would otherwise see the rollup silently fail. SUPPORTING DETAIL The query reads only Id and CurrencyIsoCode for parent records whose Ids the trigger-driven rollup already holds, so it can convert gift amounts into the parent's currency before writing the total. The object name comes from the Id's own schema token, not from input, so the dynamic FROM cannot be injected. The result is a map of Id to ISO code that is used in the arithmetic and never returned to a user. Running in user mode would make a rollup fail or mis-convert for a staff user who can save a gift but lacks read FLS on a standard CurrencyIsoCode field. The code catches any exception and falls back to the corporate currency. EVIDENCE - base/main/default/classes/shared/CurrencyResolver.cls:258-269 - dynamic FROM built from Id.getSObjectType().getDescribe().getName(); bind is Set<Id>; SYSTEM_MODE; catch returns empty map - base/main/default/classes/shared/CurrencyResolver.cls - isoCodesByRecordId doc states the reason (currency code is not customer data) - base/main/default/classes/donation/selector/OpportunitiesSelector.cls:189 - upstream SYSTEM_MODE rollup read, itself commented SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm

Reported vulnerability	Location	Response
R-047 Apex SOQL SOSL USER MODE MISSING False positive	Paths 2, 4 At scan: classes/AccountsSelector.cls:311 Submitted source: classes/donor/selector/AccountsSelector.cls:311	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: <code>public List<Account> selectPersonDonorDetailById(Id accountId) {</code> WHY THE CLAIM IS FALSE The claim that USER_MODE is missing is false: the query is executed with Database.queryWithBinds(..., AccessLevel.USER_MODE), which enforces the running user's object permissions, field-level security and sharing. The scanner recognises only the literal WITH USER_MODE clause, and adding that clause to this query is impossible: the platform rejects a dynamic query that carries both (measured: "Cannot use the WITH AccessLevel clause in dynamic queries that also specify an access level"). SUPPORTING DETAIL This query already runs in user mode: it is executed with Database.queryWithBinds(..., AccessLevel.USER_MODE), which enforces object permissions, field-level security and sharing for the running user. The scanner only recognises the literal WITH USER_MODE clause and does not read the AccessLevel argument. The caller, DonorDetailController.requireDonor, uses this user-mode, sharing-enforced read as its IDOR gate: if the record Id supplied by the component is not visible to the user, zero rows come back and the controller throws "Donor not found" before any related data is read. A literal WITH USER_MODE clause cannot be added to satisfy the pattern: the query is built by the FFLib query factory as a dynamic string, and the platform rejects a dynamic query that carries both the clause and an AccessLevel argument. Measured 2026-10-01 in a scratch org, running as a Standard User who holds the Fundraising_Admin permission set: Database.queryWithBinds('SELECT Id FROM Account WITH USER_MODE', binds, AccessLevel.USER_MODE) throws System.SecurityException: Cannot use the WITH AccessLevel clause in dynamic queries that also specify an access level. The AccessLevel argument is the documented way to run a dynamic query in user mode. EVIDENCE - base/main/default/classes/donor/selector/AccountsSelector.cls:311~~322 - Database.queryWithBinds(..., AccessLevel.USER_MODE); class is inherited sharing - base/main/default/classes/donor/controller/DonorDetailController.cls:352 - requireDonor calls the Account/Contact detail selector and throws QueryException("Donor not found") on an empty result - base/main/default/classes/donor/controller/DonorDetailController.cls:102,142,176,205,238,272 - every @AuraEnabled action calls requireDonor before any other read - base/main/default/classes/donor/controller/DonorDetailController.cls header - documents requireDonor as the gate in front of the SYSTEM_MODE reads of Donation_Designation__c and Guest_Request__c - base/main/default/classes/campaign/controller/FundraiserDetailController.cls:39 - getRecurring never calls AccountsSelector; Checkmarx path 4/6 source is spurious SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm

Reported vulnerability	Location	Response
		• https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm

Reported vulnerability	Location	Response
R-048 Apex SOQL SOSL USER MODE MISSING False positive	Paths 3, 5 At scan: classes/ContactsSelector.cls:228 Submitted source: classes/donor/selector/ContactsSelector.cls:228	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: <code>public List<Contact> selectDonorDetailById(Id contactId) {</code> WHY THE CLAIM IS FALSE The claim that USER_MODE is missing is false: the query factory is built with CRUD and FLS enforcement on and the SOQL is executed with Database.queryWithBinds(..., AccessLevel.USER_MODE), which enforces object permissions, FLS and sharing for the running user. The scanner recognises only the literal WITH USER_MODE clause, which the platform forbids in a dynamic query that already specifies an access level (measured). SUPPORTING DETAIL The SOQL string is built by the query factory and executed with Database.queryWithBinds(..., AccessLevel.USER_MODE), so object permissions, FLS and sharing are enforced for the running user. The factory is also constructed with CRUD enforcement on. The scanner does not recognise the AccessLevel argument. The only caller that passes a client-supplied Id is DonorDetailController.requireDonor, which throws "Donor not found" when this user-mode read returns nothing, so a user cannot reach another donor's gifts, recurring plans or requests by guessing an Id. A literal WITH USER_MODE clause cannot be added to satisfy the pattern: the query is built by the FFLib query factory as a dynamic string, and the platform rejects a dynamic query that carries both the clause and an AccessLevel argument. Measured 2026-10-01 in a scratch org, running as a Standard User who holds the Fundraising_Admin permission set: Database.queryWithBinds('SELECT Id FROM Account WITH USER_MODE', binds, AccessLevel.USER_MODE) throws System.SecurityException: Cannot use the WITH AccessLevel clause in dynamic queries that also specify an access level. The AccessLevel argument is the documented way to run a dynamic query in user mode. EVIDENCE - base/main/default/classes/donor/selector/ContactsSelector.cls:228-~242 - newQueryFactory() (fflib defaults enforceCRUD=true) then queryWithBinds with AccessLevel.USER_MODE - base/main/default/classes/donor/controller/DonorDetailController.cls:352 - requireDonor gate; empty result -> QueryException - base/main/default/classes/donor/controller/DonorDetailController.cls:102,142,176,205,238,272 - every action gated by requireDonor first - base/main/default/classes/campaign/controller/FundraiserDetailController.cls:39 - getRecurring uses RecurringDonationsSelector.selectByCampaignPaged, not ContactsSelector SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm

Reported vulnerability	Location	Response
R-049 Apex SOQL SOSL USER MODE MISSING False positive	Paths 6, 8 At scan: classes/CommunityService.cls:269 Submitted source: classes/setup/service/CommunityService.cls:269	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: <code>protected virtual List<NetworkRow> queryNetworkById(String networkId) {</code> WHY THE CLAIM IS FALSE The claim is false because the read returns only the Experience site's Id, name, URL prefix and status, org configuration rather than record data, and is reachable only from the administrator-only setup console; Network is a setup object, and a user-mode read of setup objects requires View Setup and Configuration, which the package's admin permission set cannot assume every administrator holds. SUPPORTING DETAIL This read resolves the Experience Cloud site that the administrator is configuring. It returns only the Network's Id, name, URL prefix and status, which are org configuration and not record data. It is reachable only from the package's setup console, whose controller is granted solely by the admin permission set. Network visibility in user mode is limited to sites the user is a member of, unless the user holds Create and Set Up Experiences, View All Data or Modify All Data. The admin must be able to resolve a site they are about to repoint the package at, including one still in Preview. The Id binds as a variable, and the query sits in a try/catch that reports "site not found" instead of exposing an error. EVIDENCE - base/main/default/classes/setup/service/CommunityService.cls:269-277 - Database.query with AccessLevel.SYSTEM_MODE; bind variable, not concatenation - base/main/default/classes/setup/service/CommunityService.cls:251-266 - resolveByNetworkId wraps it in try/catch and logs via AppLogger.info - base/main/default/classes/setup/controller/SetupController.cls:632,642 - saveExperienceSite / checkGuestPermissionSet are @AuraEnabled on a with-sharing controller - base/main/default/permissionsets - only Fundraising_Admin grants classAccess to SetupController; Fundraising_User/ReadOnly deliberately omit it - base/main/default/classes/setup/service/CommunityService.cls:6-13 - public virtual without sharing; header rationale: User is the only record-shared read; Network and the other setup objects carry per-site SYSTEM_MODE reasons https://developer.salesforce.com/docs/atlas.en-us.object_reference.meta/object_reference/sforce_api_objects_network.htm - special access rules: users with Create and Set Up Experiences / View All Data / Modify All Data see all Network rows; others see only sites they are members of SOURCES https://developer.salesforce.com/docs/atlas.en-us.object_reference.meta/object_reference/sforce_api_objects_network.htm https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm

Reported vulnerability	Location	Response
R-016 Apex SOQL SOSL USER MODE MISSING False positive	Paths 7, 15–16, 27 At scan: classes/SettingsService.cls:345 Submitted source: classes/setup/service/SettingsService.cls:345	See R-016 under “Apex CRUD ContentDistribution” — same code location, same response.
R-010 Apex SOQL SOSL USER MODE MISSING False positive	Paths 9 At scan: classes/CommunityService.cls:513 Submitted source: classes/setup/service/CommunityService.cls:513	See R-010 under “Apex CRUD Violation” — same code location, same response.

Reported vulnerability	Location	Response
R-050 Apex SOQL SOSL USER MODE MISSING False positive	Paths 10 At scan: classes/CommunityService.cls:328 Submitted source: classes/setup/service/CommunityService.cls:328	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: List<PermissionSetAssignment> assignments = [WHY THE CLAIM IS FALSE The claim is false because the read is a diagnostic over PermissionSetAssignment, a setup object with no sharing model, returning only a boolean and a remediation message to the administrator-only setup console; the guest user Id is resolved on the server, and a user-mode read would require View Setup and Configuration, which the package cannot grant. SUPPORTING DETAIL This is a read-only diagnostic in the admin setup console. It checks whether the site's guest user has been assigned the package's guest permission set and returns only a boolean and a remediation message. The guest user Id is resolved on the server from the selected site, not taken from the client. No record data is returned, and the read is wrapped in a try/catch that degrades to 'could not verify'. PermissionSetAssignment is a setup object whose visibility is governed by View Setup and Configuration rather than record sharing, which the package's admin permission set does not grant; the read stays in system mode so an administrator without it gets a true answer. The site carries its own SYSTEM_MODE comment stating this. EVIDENCE - base/main/default/classes/setup/service/CommunityService.cls:325-331 - per-site // SYSTEM_MODE: comment directly above the query (setup data about the site GUEST user; only a Boolean / Set<Id> leaves the method) - python3 scripts/lint/system-mode-inventory.py -> 0 unjustified sites - SetupController.cls saveExperienceSite / checkGuestPermissionSet are @AuraEnabled on a with-sharing controller granted only in Fundraising_Admin - docs/security/appexchange-security-review-readiness.md F-1 (third pass, 2026-09-25) SOURCES https://developer.salesforce.com/docs/atlas.en-us.object_reference.meta/object_reference/sforce_api_objects_permissionsetassignment.htm https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm https://developer.salesforce.com/blogs/2021/05/boost-security-by-auditing-your-privileged-users-with-soql

Reported vulnerability	Location	Response
R-051 Apex SOQL SOSL USER MODE MISSING False positive	Paths 11 At scan: classes/CommunityService.cls:569 Submitted source: classes/setup/service/CommunityService.cls:569	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: <code>for (PermissionSetAssignment psa : [</code> WHY THE CLAIM IS FALSE The claim is false because the read checks the guest user's permission-set assignments for a standard-object grant the administrator must add by hand and returns only a boolean to the administrator-only setup console; the objects are setup objects with no sharing model, and a user-mode read would require View Setup and Configuration, which the package cannot grant. SUPPORTING DETAIL This read-only check verifies that the site guest user has read/create on User External Credential, a standard-object grant the admin must add by hand because a 2GP permission set cannot carry it. The method returns only a boolean for the setup console checklist. The guest user Id is derived on the server, the read never runs as or for a guest request, and failures degrade to a 'check manually' message. The reads are setup data governed by View Setup and Configuration, so they stay in system mode; each carries its own SYSTEM_MODE comment. EVIDENCE - base/main/default/classes/setup/service/CommunityService.cls:566-583 - per-site // SYSTEM_MODE: comment directly above the query (setup data about the site GUEST user; only a Boolean / Set<Id> leaves the method) - python3 scripts/lint/system-mode-inventory.py -> 0 unjustified sites - SetupController.cls saveExperienceSite / checkGuestPermissionSet are @AuraEnabled on a with-sharing controller granted only in Fundraising_Admin - docs/security/appexchange-security-review-readiness.md F-1 (third pass, 2026-09-25) SOURCES https://developer.salesforce.com/docs/atlas.en-us.object_reference.meta/object_reference/sforce_api_objects_permissionsetassignment.htm https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm

Reported vulnerability	Location	Response
R-052 Apex SOQL SOSL USER MODE MISSING False positive	Paths 12 At scan: classes/CommunityService.cls:608 Submitted source: classes/setup/service/CommunityService.cls:608	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: for (SetupEntityAccess a : [WHY THE CLAIM IS FALSE The claim is false because the read is over SetupEntityAccess, a setup object with no sharing model, returning only permission-set Ids used for an in-memory comparison in the administrator-only setup console; a user-mode read would require View Setup and Configuration, which the package's admin permission set cannot grant. SUPPORTING DETAIL This read-only check finds which permission sets grant access to the Stripe external credential's principal, so the setup console can tell the admin whether the guest user can make the payment callout. It returns only permission-set Ids used for an in-memory set intersection, never record data. SetupEntityAccess is a setup object whose visibility is governed by View Setup and Configuration, which the admin permission set does not grant, so the read stays in system mode. The inputs are resolved on the server, the call is reachable only from the admin-only setup controller, and the site carries its own SYSTEM_MODE comment. EVIDENCE - base/main/default/classes/setup/service/CommunityService.cls:605-611 - per-site // SYSTEM_MODE: comment directly above the query (setup data about the site GUEST user; only a Boolean / Set<Id> leaves the method) - python3 scripts/lint/system-mode-inventory.py -> 0 unjustified sites - SetupController.cls saveExperienceSite / checkGuestPermissionSet are @AuraEnabled on a with-sharing controller granted only in Fundraising_Admin - docs/security/appexchange-security-review-readiness.md F-1 (third pass, 2026-09-25) SOURCES https://developer.salesforce.com/docs/atlas.en-us.object_reference.meta/object_reference/sforce_api_objects_setupentityaccess.htm https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm

Reported vulnerability	Location	Response
R-053 Apex SOQL SOSL USER MODE MISSING False positive	Paths 13 At scan: classes/CommunityService.cls:407 Submitted source: classes/setup/service/CommunityService.cls:407	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: for (PermissionSetAssignment psa : [WHY THE CLAIM IS FALSE The claim is false because the read counts ObjectPermissions rows on the guest user's permission sets and returns only a boolean to the administrator-only setup console; both are setup objects with no sharing model, and a user-mode read would require View Setup and Configuration, which the package's admin permission set cannot grant. SUPPORTING DETAIL This read-only check powers the setup console's 'guest user can read Campaign' step. It reads the guest user's permission-set assignments and counts ObjectPermissions rows granting Campaign read, returning only a boolean so the admin knows whether the documented manual grant is done (a managed package cannot ship that grant on a standard object). No record data leaves the method and the user Id is derived on the server. Both reads are setup data governed by View Setup and Configuration, which the admin permission set does not grant, so they stay in system mode; each carries its own SYSTEM_MODE comment. EVIDENCE - base/main/default/classes/setup/service/CommunityService.cls:404-418 - per-site // SYSTEM_MODE: comment directly above the query (setup data about the site GUEST user; only a Boolean / Set<Id> leaves the method) - python3 scripts/lint/system-mode-inventory.py -> 0 unjustified sites - SetupController.cls saveExperienceSite / checkGuestPermissionSet are @AuraEnabled on a with-sharing controller granted only in Fundraising_Admin - docs/security/appexchange-security-review-readiness.md F-1 (third pass, 2026-09-25) SOURCES https://developer.salesforce.com/docs/atlas.en-us.object_reference.meta/object_reference/sforce_api_objects_permissionsetassignment.htm https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm
R-023 Apex SOQL SOSL USER MODE MISSING False positive	Paths 14 At scan: classes/SettingsService.cls:1026 Submitted source: classes/setup/service/SettingsService.cls:1026	See R-023 under “FLS Create” — same code location, same response.

Reported vulnerability	Location	Response
R-054 Apex SOQL SOSL USER MODE MISSING False positive	Paths 17 At scan: classes/EventPurchaseService_Test.cls:906 Submitted source: classes/event/service/EventPurchaseService_Test.cls:906	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: delete as system tier; WHY THE CLAIM IS FALSE The claim is false because the statement is a fixture delete inside an @IsTest class, which no user can invoke at runtime, and must succeed under whatever user the subscriber's org-wide test run executes as; it is also DML, not a query, so the SOQL/SOSL rule does not apply. SUPPORTING DETAIL This is a fixture write inside an @IsTest class. It adjusts a test ticket-tier product (for example, sets it sold out or outside its sale window) to arrange the scenario for the service method under test. The class runs only in Apex test execution and is not reachable by any runtime user. The dataflow source is a schema record-type describe, not external input. The statement stays in system mode on purpose: the package's tests are also run in subscriber orgs (an org-wide test run there includes managed-package tests), where the running user may lack a permission the fixture would need in user mode (edit access to Campaign, the master of the ticket-type object, which the platform ties to the Marketing User flag). A user-mode fixture would fail the subscriber's test run for a reason unrelated to the behaviour under test, while protecting nothing, because a test class has no runtime caller. EVIDENCE - base/main/default/classes/event/service/EventPurchaseService_Test.cls:1 - class annotated @IsTest(SeeAllData=false) - base/main/default/classes/shared/TestDataFactory.cls:743 - the dataflow source is the Product2 record-type describe in TestDataFactory.createTicketProduct (:741), a test fixture helper - base/main/default/classes/event/service/EventPurchaseService_Test.cls:906 - fixture mutation of a test ticket tier (capacity, sold count, or window) before the method under test runs; DML, not SOQL SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_annotation_isTest.htm

Reported vulnerability	Location	Response
R-055 Apex SOQL SOSL USER MODE MISSING False positive	Paths 18 At scan: classes/EventPurchaseService_Test.cls:261 Submitted source: classes/event/service/EventPurchaseService_Test.cls:261	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: update as system tier; WHY THE CLAIM IS FALSE The claim is false because the statement is a fixture write inside an @IsTest class, which no user can invoke at runtime, and must succeed under whatever user the subscriber's org-wide test run executes as; it is also DML, not a query, so the SOQL/SOSL rule does not apply. SUPPORTING DETAIL This is a fixture write inside an @IsTest class. It adjusts a test ticket-tier product (for example, sets it sold out or outside its sale window) to arrange the scenario for the service method under test. The class runs only in Apex test execution and is not reachable by any runtime user. The dataflow source is a schema record-type describe, not external input. The statement stays in system mode on purpose: the package's tests are also run in subscriber orgs (an org-wide test run there includes managed-package tests), where the running user may lack a permission the fixture would need in user mode (edit access to Campaign, the master of the ticket-type object, which the platform ties to the Marketing User flag). A user-mode fixture would fail the subscriber's test run for a reason unrelated to the behaviour under test, while protecting nothing, because a test class has no runtime caller. EVIDENCE - base/main/default/classes/event/service/EventPurchaseService_Test.cls:1 - class annotated @IsTest(SeeAllData=false) - base/main/default/classes/shared/TestDataFactory.cls:743 - the dataflow source is the Product2 record-type describe in TestDataFactory.createTicketProduct (:741), a test fixture helper - base/main/default/classes/event/service/EventPurchaseService_Test.cls:261 - fixture mutation of a test ticket tier (capacity, sold count, or window) before the method under test runs; DML, not SOQL SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_annotation_isTest.htm

Reported vulnerability	Location	Response
R-056 Apex SOQL SOSL USER MODE MISSING False positive	Paths 19 At scan: classes/EventPurchaseService_Test.cls:578 Submitted source: classes/event/service/EventPurchaseService_Test.cls:578	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: update as system tier; WHY THE CLAIM IS FALSE The claim is false because the statement is a fixture write inside an @IsTest class, which no user can invoke at runtime, and must succeed under whatever user the subscriber's org-wide test run executes as; it is also DML, not a query, so the SOQL/SOSL rule does not apply. SUPPORTING DETAIL This is a fixture write inside an @IsTest class. It adjusts a test ticket-tier product (for example, sets it sold out or outside its sale window) to arrange the scenario for the service method under test. The class runs only in Apex test execution and is not reachable by any runtime user. The dataflow source is a schema record-type describe, not external input. The statement stays in system mode on purpose: the package's tests are also run in subscriber orgs (an org-wide test run there includes managed-package tests), where the running user may lack a permission the fixture would need in user mode (edit access to Campaign, the master of the ticket-type object, which the platform ties to the Marketing User flag). A user-mode fixture would fail the subscriber's test run for a reason unrelated to the behaviour under test, while protecting nothing, because a test class has no runtime caller. EVIDENCE - base/main/default/classes/event/service/EventPurchaseService_Test.cls:1 - class annotated @IsTest(SeeAllData=false) - base/main/default/classes/shared/TestDataFactory.cls:743 - the dataflow source is the Product2 record-type describe in TestDataFactory.createTicketProduct (:741), a test fixture helper - base/main/default/classes/event/service/EventPurchaseService_Test.cls:578 - fixture mutation of a test ticket tier (capacity, sold count, or window) before the method under test runs; DML, not SOQL SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_annotation_isTest.htm

Reported vulnerability	Location	Response
R-057 Apex SOQL SOSL USER MODE MISSING False positive	Paths 20 At scan: classes/EventPurchaseService_Test.cls:610 Submitted source: classes/event/service/EventPurchaseService_Test.cls:610	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: update as system tier; WHY THE CLAIM IS FALSE The claim is false because the statement is a fixture write inside an @IsTest class, which no user can invoke at runtime, and must succeed under whatever user the subscriber's org-wide test run executes as; it is also DML, not a query, so the SOQL/SOSL rule does not apply. SUPPORTING DETAIL This is a fixture write inside an @IsTest class. It adjusts a test ticket-tier product (for example, sets it sold out or outside its sale window) to arrange the scenario for the service method under test. The class runs only in Apex test execution and is not reachable by any runtime user. The dataflow source is a schema record-type describe, not external input. The statement stays in system mode on purpose: the package's tests are also run in subscriber orgs (an org-wide test run there includes managed-package tests), where the running user may lack a permission the fixture would need in user mode (edit access to Campaign, the master of the ticket-type object, which the platform ties to the Marketing User flag). A user-mode fixture would fail the subscriber's test run for a reason unrelated to the behaviour under test, while protecting nothing, because a test class has no runtime caller. EVIDENCE - base/main/default/classes/event/service/EventPurchaseService_Test.cls:1 - class annotated @IsTest(SeeAllData=false) - base/main/default/classes/shared/TestDataFactory.cls:743 - the dataflow source is the Product2 record-type describe in TestDataFactory.createTicketProduct (:741), a test fixture helper - base/main/default/classes/event/service/EventPurchaseService_Test.cls:610 - fixture mutation of a test ticket tier (capacity, sold count, or window) before the method under test runs; DML, not SOQL SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_annotation_isTest.htm

Reported vulnerability	Location	Response
R-058 Apex SOQL SOSL USER MODE MISSING False positive	Paths 21 At scan: classes/EventPurchaseService_Test.cls:641 Submitted source: classes/event/service/EventPurchaseService_Test.cls:641	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: update as system tier; WHY THE CLAIM IS FALSE The claim is false because the statement is a fixture write inside an @IsTest class, which no user can invoke at runtime, and must succeed under whatever user the subscriber's org-wide test run executes as; it is also DML, not a query, so the SOQL/SOSL rule does not apply. SUPPORTING DETAIL This is a fixture write inside an @IsTest class. It adjusts a test ticket-tier product (for example, sets it sold out or outside its sale window) to arrange the scenario for the service method under test. The class runs only in Apex test execution and is not reachable by any runtime user. The dataflow source is a schema record-type describe, not external input. The statement stays in system mode on purpose: the package's tests are also run in subscriber orgs (an org-wide test run there includes managed-package tests), where the running user may lack a permission the fixture would need in user mode (edit access to Campaign, the master of the ticket-type object, which the platform ties to the Marketing User flag). A user-mode fixture would fail the subscriber's test run for a reason unrelated to the behaviour under test, while protecting nothing, because a test class has no runtime caller. EVIDENCE - base/main/default/classes/event/service/EventPurchaseService_Test.cls:1 - class annotated @IsTest(SeeAllData=false) - base/main/default/classes/shared/TestDataFactory.cls:743 - the dataflow source is the Product2 record-type describe in TestDataFactory.createTicketProduct (:741), a test fixture helper - base/main/default/classes/event/service/EventPurchaseService_Test.cls:641 - fixture mutation of a test ticket tier (capacity, sold count, or window) before the method under test runs; DML, not SOQL SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_annotation_isTest.htm

Reported vulnerability	Location	Response
R-059 Apex SOQL SOSL USER MODE MISSING False positive	Paths 22 At scan: classes/EventPurchaseService_Test.cls:672 Submitted source: classes/event/service/EventPurchaseService_Test.cls:672	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: update as system tier; WHY THE CLAIM IS FALSE The claim is false because the statement is a fixture write inside an @IsTest class, which no user can invoke at runtime, and must succeed under whatever user the subscriber's org-wide test run executes as; it is also DML, not a query, so the SOQL/SOSL rule does not apply. SUPPORTING DETAIL This is a fixture write inside an @IsTest class. It adjusts a test ticket-tier product (for example, sets it sold out or outside its sale window) to arrange the scenario for the service method under test. The class runs only in Apex test execution and is not reachable by any runtime user. The dataflow source is a schema record-type describe, not external input. The statement stays in system mode on purpose: the package's tests are also run in subscriber orgs (an org-wide test run there includes managed-package tests), where the running user may lack a permission the fixture would need in user mode (edit access to Campaign, the master of the ticket-type object, which the platform ties to the Marketing User flag). A user-mode fixture would fail the subscriber's test run for a reason unrelated to the behaviour under test, while protecting nothing, because a test class has no runtime caller. EVIDENCE - base/main/default/classes/event/service/EventPurchaseService_Test.cls:1 - class annotated @IsTest(SeeAllData=false) - base/main/default/classes/shared/TestDataFactory.cls:743 - the dataflow source is the Product2 record-type describe in TestDataFactory.createTicketProduct (:741), a test fixture helper - base/main/default/classes/event/service/EventPurchaseService_Test.cls:672 - fixture mutation of a test ticket tier (capacity, sold count, or window) before the method under test runs; DML, not SOQL SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_annotation_isTest.htm

Reported vulnerability	Location	Response
R-060 Apex SOQL SOSL USER MODE MISSING False positive	Paths 23 At scan: classes/EventRegistrationController_Test.cls:171 Submitted source: classes/fundraising/controller/EventRegistrationController_Test.cls:171	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: <code>update as system t;</code> WHY THE CLAIM IS FALSE The claim is false because the statement is a fixture update inside an @IsTest class, which cannot run outside Apex test execution and must succeed under whatever user the subscriber's org-wide test run executes as; it is DML, not a query, so the SOQL/SOSL rule does not apply, and the reported dataflow source is a schema describe, not user input. SUPPORTING DETAIL This line is a test-fixture update inside an @IsTest class. It prepares a ticket product so the controller method under test sees a particular state. It cannot run outside Apex test execution. The dataflow source is a schema describe. The statement stays in system mode on purpose: the package's tests are also run in subscriber orgs (an org-wide test run there includes managed-package tests), where the running user may lack a permission the fixture would need in user mode (edit access to Campaign, the master of the ticket-type object, which the platform ties to the Marketing User flag). A user-mode fixture would fail the subscriber's test run for a reason unrelated to the behaviour under test, while protecting nothing, because a test class has no runtime caller. EVIDENCE - base/main/default/classes/fundraising/controller/EventRegistrationController_Test.cls:1 - class annotated @IsTest(SeeAllData=false) - base/main/default/classes/shared/TestDataFactory.cls:743 - the dataflow source is the Product2 record-type describe in TestDataFactory.createTicketProduct (:741), a test fixture helper - base/main/default/classes/fundraising/controller/EventRegistrationController_Test.cls:171 - fixture update of a test ticket product; DML, not SOQL SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_annotation_isTest.htm

Reported vulnerability	Location	Response
R-061 Apex SOQL SOSL USER MODE MISSING False positive	Paths 24–26, 29 At scan: classes/ReceiptSequenceSelector.cls:63 Submitted source: classes/donation/selector/ReceiptSequenceSelector.cls:63	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: <code>return (List<Receipt_Sequence__c>) Database.queryWithBinds(</code> WHY THE CLAIM IS FALSE The claim is false because the row is an internal per-tax-year receipt counter holding only a year and the last number issued, not donor data; the FOR UPDATE lock keeps receipt numbers gapless, and the read must succeed from the guest finalize path and the renewal job, neither of whose running users can be granted the object. SUPPORTING DETAIL Receipt_Sequence__c is an internal per-tax-year counter row, not donor data. It holds only a year and the last receipt number issued. The query locks the counter (FOR UPDATE) so receipt numbers stay gapless and unique. Access must not depend on the running user, because a user without read access on the counter would otherwise break receipt numbering for their own gift. The only bind is a set of tax years derived from the Opportunity close date; no caller-supplied Id or filter reaches the query, and nothing from the counter row is returned to a UI. EVIDENCE - base/main/default/classes/donation/selector/ReceiptSequenceSelector.cls:60-65 - SYSTEM_MODE queryWithBinds, FOR UPDATE, bind is Set<Decimal> of tax years - base/main/default/classes/donation/service/ReceiptNumberService.cls - lockCounters/assignWithoutUnitOfWork are the only callers; class is inherited sharing - base/main/default/classes/donation/domain/Opportunities.cls:284-296 - stampOfflineReceiptNumber (before trigger, with sharing domain) calls assignWithoutUnitOfWork for manually entered Closed Won gifts; this runs as the interactive user, not in batch - base/main/default/objects/Receipt_Sequence__c - sharingModel ReadWrite; Fundraising_Admin CRU+ViewAll, Fundraising_User and ReadOnly Read only (User cannot create a counter, so USER_MODE would break the offline-gift path for staff) https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm - USER_MODE with no object access throws a QueryException rather than returning zero rows SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm

Reported vulnerability	Location	Response
R-062 Apex SOQL SOSL USER MODE MISSING False positive	Paths 28 At scan: classes/CurrencyResolver.cls:264 Submitted source: classes/shared/CurrencyResolver.cls:264	SCANNER'S CLAIM <i>USER_MODE is recommended for SOQL or SOSL operations</i> Flagged statement: return toIsoMap(WHY THE CLAIM IS FALSE The claim is false because the query reads only Id and CurrencyIsoCode for parent records whose Ids a trigger-driven rollup already holds, in order to convert amounts before writing the rollup total; the object name comes from the Id's schema token, not from input, and the read runs in trigger context as the DML writer, where a user without the parent's currency field would otherwise see the rollup silently fail. SUPPORTING DETAIL The query reads only Id and CurrencyIsoCode for parent records whose Ids the trigger-driven rollup already holds, so it can convert gift amounts into the parent's currency before writing the total. The object name comes from the Id's own schema token, not from input, so the dynamic FROM cannot be injected. The result is a map of Id to ISO code that is used in the arithmetic and never returned to a user. Running in user mode would make a rollup fail or mis-convert for a staff user who can save a gift but lacks read FLS on a standard CurrencyIsoCode field. The code catches any exception and falls back to the corporate currency. EVIDENCE - base/main/default/classes/shared/CurrencyResolver.cls:258-269 - dynamic FROM built from Id.getSObjectType().getDescribe().getName(); bind is Set<Id>; SYSTEM_MODE; catch returns empty map - base/main/default/classes/shared/CurrencyResolver.cls - isoCodesByRecordId doc states the reason (currency code is not customer data) - base/main/default/classes/donation/selector/OpportunitiesSelector.cls:189 - upstream SYSTEM_MODE rollup read, itself commented SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_enforce_usermode.htm

Sharing — 27 paths, 27 locations

What this query claims: All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.

Reported vulnerability	Location	Response
R-063 Sharing False positive	Paths 1 At scan: classes/RecurringDonationsSelector.cls:11 Submitted source: classes/recurring/selector/RecurringDonationsSelector.cls:11	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservice or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: <code>public inherited sharing class RecurringDonationsSelector extends fflib_SObjectSelector</code> WHY THE CLAIM IS FALSE The claim is false because <code>inherited sharing</code> is the documented mode for a class that is never an entry point: the selector takes the sharing of its caller, so a staff request is sharing-enforced and only the guest and job callers, which declare <code>without sharing</code> with a stated reason, bypass it. SUPPORTING DETAIL <code>inherited sharing</code> is intentional on this selector. It is never the entry point of a transaction, so its record visibility is that of the caller: staff controllers are <code>with sharing</code>, and the guest controllers and reconciliation jobs are <code>without sharing</code> for their own documented reasons. The Apex Developer Guide recommends <code>inherited sharing</code> for 'service classes that must be flexible and support use cases with different sharing modes'. Each query also sets an explicit <code>AccessLevel</code>, and each <code>SYSTEM_MODE</code> query carries its own reason at the call site. EVIDENCE - base/main/default/classes/recurring/selector/RecurringDonationsSelector.cls:11 - declaration - base/main/default/classes/shared/Application.cls - resolved via the Selector factory only - grep: no entry-point annotation or async interface in the class SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-064 Sharing False positive	Paths 2 At scan: classes/RecurringDesignationsSelector.cls:4 Submitted source: classes/recurring/selector/RecurringDesignationsSelector.cls:4	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservice or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: <code>public inherited sharing class RecurringDesignationsSelector extends fflib_SObjectSelector</code> WHY THE CLAIM IS FALSE The claim is false because <code>inherited sharing</code> is the documented mode for a class that is never an entry point: the selector takes the sharing of its caller, so a staff request is sharing-enforced and only the guest and job callers, which declare <code>without sharing</code> with a stated reason, bypass it. SUPPORTING DETAIL <code>inherited sharing</code> is intentional on this selector. It is never the entry point of a transaction, so its record visibility is that of the caller: staff controllers are <code>with sharing</code>, and the guest controllers and reconciliation jobs are <code>without sharing</code> for their own documented reasons. The Apex Developer Guide recommends <code>inherited sharing</code> for 'service classes that must be flexible and support use cases with different sharing modes'. Each query also sets an explicit <code>AccessLevel</code>, and each <code>SYSTEM_MODE</code> query carries its own reason at the call site. EVIDENCE - base/main/default/classes/recurring/selector/RecurringDesignationsSelector.cls:4 - declaration - base/main/default/classes/shared/Application.cls - resolved via the Selector factory only - grep: no entry-point annotation or <code>async</code> interface in the class SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-065 Sharing False positive	Paths 3 At scan: classes/OpportunitiesSelector.cls:19 Submitted source: classes/donation/selector/OpportunitiesSelector.cls:19	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: <code>public inherited sharing class OpportunitiesSelector extends fflib_SObjectSelector</code> WHY THE CLAIM IS FALSE The claim is false because <code>inherited sharing</code> is the documented mode for a class that is never an entry point: the selector takes the sharing of its caller, so a staff controller's request is sharing-enforced and only the guest and job paths, which declare <code>without sharing</code> with a stated reason, bypass it. SUPPORTING DETAIL <code>inherited sharing</code> is intentional on this selector. It is never the entry point of a transaction, so its record visibility is that of the caller: staff controllers are <code>with sharing</code>, and the guest controllers and reconciliation jobs are <code>without sharing</code> for their own documented reasons. The Apex Developer Guide recommends <code>inherited sharing</code> for 'service classes that must be flexible and support use cases with different sharing modes'. Each query also sets an explicit <code>AccessLevel</code>, and each <code>SYSTEM_MODE</code> query carries its own reason at the call site. EVIDENCE - base/main/default/classes/donation/selector/OpportunitiesSelector.cls:19 - declaration - base/main/default/classes/shared/Application.cls - resolved via the Selector factory only - grep: no entry-point annotation or async interface in the class SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-066 Sharing False positive	Paths 4 At scan: classes/fflib_SObjectUnitOfWork.cls:191 Submitted source: classes/fflib/fflib_SObjectUnitOfWork.cls:191	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: public inherited sharing virtual class SimpleDML implements IDML, IDMLUpsertable WHY THE CLAIM IS FALSE The claim is false because SimpleDML is declared inherited sharing, the documented mode for a non-entry-point class: a commit runs in the sharing mode of the class that called commitWork(), so a staff controller's commit is sharing-enforced by the platform and only callers that explicitly opt into the SystemDML subclass, each with a stated reason, bypass it. SUPPORTING DETAIL SimpleDML is the FFLib Unit of Work's default DML strategy. It is declared inherited sharing, so a commit follows the sharing mode of the class that called commitWork(). Every staff controller and service in the package is with sharing, so a staff commit runs with sharing and the platform enforces the running user's record access on the write. The writes that must bypass sharing (guest entry points, batch jobs, the platform-event subscriber and trigger-fired rollups) do not rely on this class: they construct their Unit of Work with SystemDML, a package-authored without sharing subclass, and each of the 24 construction sites states which of those families it belongs to. Object and field permissions are enforced one layer up, in the FFLib Selectors and by Security.stripInaccessible. SystemDML_Test proves, with a Minimum Access user, that a SimpleDML commit on a record the user is not shared is refused and a SystemDML commit succeeds. EVIDENCE - base/main/default/classes/fflib/fflib_SObjectUnitOfWork.cls:64 - public virtual inherited sharing class fflib_SObjectUnitOfWork, rationale at :54-61 - base/main/default/classes/fflib/fflib_SObjectUnitOfWork.cls:165-191 - rationale and public inherited sharing virtual class SimpleDML implements IDML, IDMLUpsertable - base/main/default/classes/shared/SystemDML.cls:30 - public without sharing class SystemDML extends fflib_SObjectUnitOfWork.SimpleDML, family list at :9-27 - grep: new SystemDML() at 24 production sites, each preceded by a // SystemDML: guest job rollup comment; none in a staff controller

Reported vulnerability	Location	Response
		- base/main/default/classes/shared/SystemDML_Test.cls:33 - test_simpleDml_withSharingCaller_cannotUpdateUnsharedRecord - base/main/default/classes/shared/SystemDML_Test.cls:60 - test_systemDml_withSharingCaller_updatesUnsharedRecord - DEPENDENCIES.md fork log row for fflib_SObjectUnitOfWork.cls https://github.com/apex-enterprise-patterns/fflib-apex-common/issues/530 - upstream proposal for the same inherited sharing declarations SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm https://github.com/apex-enterprise-patterns/fflib-apex-common/issues/530

Reported vulnerability	Location	Response
R-067 Sharing False positive	Paths 5 At scan: classes/EventTicketTypesSelector.cls:6 Submitted source: classes/event/selector/EventTicketTypesSelector.cls:6	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: <code>public inherited sharing class EventTicketTypesSelector extends fflib_SObjectSelector</code> WHY THE CLAIM IS FALSE The claim is false because <code>inherited sharing</code> is the documented mode for a class that is never an entry point: the selector takes the sharing of its caller. Its reads are additionally gated on the master Campaign's visibility by the calling controller, because the object is a master-detail detail whose field permissions are inert. SUPPORTING DETAIL <code>inherited sharing</code> is intentional on this selector. It is never the entry point of a transaction, so its record visibility is that of the caller: staff controllers are <code>with sharing</code>, and the guest controllers and reconciliation jobs are <code>without sharing</code> for their own documented reasons. The Apex Developer Guide recommends <code>inherited sharing</code> for 'service classes that must be flexible and support use cases with different sharing modes'. Each query also sets an explicit <code>AccessLevel</code>, and each <code>SYSTEM_MODE</code> query carries its own reason at the call site. EVIDENCE - base/main/default/classes/event/selector/EventTicketTypesSelector.cls:6 - declaration - base/main/default/classes/shared/Application.cls - resolved via the Selector factory only - grep: no entry-point annotation or async interface in the class SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-068 Sharing False positive	Paths 6 At scan: classes/EventRegistrationController.cls:196 Submitted source: classes/fundraising/controller/EventRegistrationController.cls:196	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app. Flagged statement: private inherited sharing class CreateTicketIntent extends GuestAction {</i> WHY THE CLAIM IS FALSE The claim is false because the inner action class is <code>inherited sharing</code> (matching the <code>GuestAction</code> base it extends) and is invoked only from the public event-registration controller, where the Experience Cloud guest user holds no record sharing; the controller's <code>without sharing</code> carries its reason, and the action's reads are gated on an active, published campaign. SUPPORTING DETAIL The inner class extends <code>GuestAction</code>, which is <code>inherited sharing</code>, and declares the same keyword so its mode is readable at the declaration. Invoked from the <code>without sharing</code> guest controller, it runs without sharing. That is required: an Experience Cloud guest creates a ticket order against a Campaign it is not shared under sharing rules, and the Guest User licence holds no record sharing. Record visibility is decided by explicit predicates instead: <code>requireOpenCampaign</code> refuses the request unless the campaign is active, published and open for sales, and checkout is not blocked. <code>GuestAction</code> also collapses any unexpected error to one generic message, so nothing internal reaches the guest. EVIDENCE - base/main/default/classes/fundraising/controller/EventRegistrationController.cls:194-196 - rationale and <code>private inherited sharing class CreateTicketIntent extends GuestAction</code> - base/main/default/classes/fundraising/controller/EventRegistrationController.cls:9 - <code>public without sharing class EventRegistrationController</code> - base/main/default/classes/fundraising/controller/EventRegistrationController.cls:283 - <code>requireOpenCampaign(campaignId)</code> gates the request - base/main/default/classes/fundraising/controller/EventRegistrationController.cls:871-879 - <code>requireOpenCampaign: requireActiveCampaign, CampaignService.assertAcceptingDonations, SettingsService.assertCheckoutNotBlocked</code> - base/main/default/classes/shared/GuestAction.cls:30 - <code>public abstract inherited sharing class GuestAction extends ControllerAction</code>

Reported vulnerability	Location	Response
		SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm
R-069 Sharing False positive	Paths 7 At scan: classes/EmailSenderService.cls:10 Submitted source: classes/setup/service/EmailSenderService.cls:10	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: <code>public inherited sharing class EmailSenderService {</code> WHY THE CLAIM IS FALSE The claim is false because the class is a stateless helper that is never a transaction entry point and reads only org email configuration; <code>inherited sharing</code> makes it run in the mode of the service that builds the message, which is the documented purpose of that keyword. SUPPORTING DETAIL This is a stateless helper that stamps the org's configured sender identity onto outbound donor email. It is never a transaction entry point, and it takes the sharing mode of the service that builds the message. It reads only org email configuration. The keyword was reviewed for a change to <code>with sharing</code> on 2026-10-01 and kept: the class is called from the guest donation path and from the platform-event subscriber as well as from staff actions, and <code>inherited sharing</code> is the declaration that lets each of those run in its caller's mode instead of forcing one mode on all of them. EVIDENCE - base/main/default/classes/setup/service/EmailSenderService.cls:1-10 - header + declaration - grep: no entry-point annotation or async interface SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-070 Sharing False positive	Paths 8 At scan: classes/DonationService.cls:7 Submitted source: classes/donation/service/DonationService.cls:7	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app. Flagged statement: public inherited sharing class DonationService implements IDonationService {</i> WHY THE CLAIM IS FALSE The claim is false because DonationService is never a transaction entry point; inherited sharing makes it take the mode of its callers, the guest controllers (where visibility is decided by token or campaign predicates) and the reconciliation jobs, each of which declares its own keyword with its reason. SUPPORTING DETAIL DonationService is never an entry point. Its public methods are reached from the guest donation controllers, where visibility is decided by token or campaign predicates, and from the reconciliation jobs. Both of those callers establish without sharing and document why. When a with sharing staff controller calls it, it runs with sharing. That is the documented behavior of inherited sharing and the reason it is used here. EVIDENCE - base/main/default/classes/donation/service/DonationService.cls:7 - declaration - base/main/default/classes/donation/async/StagingReconciliationBatch.cls:216 - finalizeSucceededStaging caller - base/main/default/classes/donation/async/StripePaymentSweep.cls:166 - caller SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-071 Sharing False positive	Paths 9 At scan: classes/DonationDesignationsSelector.cls:8 Submitted source: classes/donation/selector/DonationDesignationsSelector.cls:8	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: <code>public inherited sharing class DonationDesignationsSelector extends fflib_SObjectSelector</code> WHY THE CLAIM IS FALSE The claim is false because <code>inherited sharing</code> is the mode Salesforce documents for non-entry-point classes and is accepted by its own false-positive guidance: the selector is never a transaction entry point, so it runs with the sharing of whichever controller or job called it, which is exactly the behaviour the rule asks for. SUPPORTING DETAIL <code>inherited sharing</code> is intentional on this selector. It is never the entry point of a transaction, so its record visibility is that of the caller: staff controllers are <code>with sharing</code>, and the guest controllers and reconciliation jobs are <code>without sharing</code> for their own documented reasons. The Apex Developer Guide recommends <code>inherited sharing</code> for 'service classes that must be flexible and support use cases with different sharing modes'. Each query also sets an explicit <code>AccessLevel</code>, and each <code>SYSTEM_MODE</code> query carries its own reason at the call site. EVIDENCE - base/main/default/classes/donation/selector/DonationDesignationsSelector.cls:8 - declaration - base/main/default/classes/shared/Application.cls - resolved via the Selector factory only - grep: no entry-point annotation or async interface in the class SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-072 Sharing False positive	Paths 10 At scan: classes/CurrencyResolver.cls:8 Submitted source: classes/shared/CurrencyResolver.cls: 8	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: public inherited sharing class CurrencyResolver { WHY THE CLAIM IS FALSE The claim is false because the helper is never a transaction entry point; inherited sharing makes it run in the mode of the controller or trigger that called it, which is the documented purpose of that keyword. SUPPORTING DETAIL This helper resolves the currency for a campaign or payment account. It is never a transaction entry point and runs in whatever sharing mode its calling controller established, which is the purpose of inherited sharing. EVIDENCE - base/main/default/classes/shared/CurrencyResolver.cls:8 - declaration - grep: no entry-point annotation or async interface SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-073 Sharing False positive	Paths 11 At scan: classes/CampaignMemberSyncService.cls:9 Submitted source: classes/donation/service/CampaignMemberSyncService.cls:9	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app. Flagged statement: public inherited sharing class CampaignMemberSyncService implements ICampaignMemberSyncService {</i> WHY THE CLAIM IS FALSE The claim is false because the class has no @AuraEnabled, REST or asynchronous surface of its own, so it is not an entry point; <code>inherited sharing</code> makes it run in its caller's mode, which is the documented purpose of that keyword. SUPPORTING DETAIL This is a service, never a transaction entry point. It takes the sharing mode of the controller or job that calls it, which is what <code>inherited sharing</code> is documented for. It has no @AuraEnabled, REST, or async surface of its own. EVIDENCE - base/main/default/classes/donation/service/CampaignMemberSyncService.cls:9 - declaration - grep: no entry-point annotation or async interface SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-074 Sharing False positive	Paths 12 At scan: <code>classes/SystemDML.cls:30</code> Submitted source: <code>classes/shared/SystemDML.cls:30</code>	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: <code>public without sharing class SystemDML extends fflib_SObjectUnitOfWork.SimpleDML {</code> WHY THE CLAIM IS FALSE The claim is false because SystemDML is a deliberately narrow wrapper, not an entry point: it is the package's explicit, per-site opt-in for commits where the running user is by design not the authority over the records (guest entry points, batch and queueable jobs, trigger-fired rollups), each opt-in carrying its reason in code; staff commits use the user-mode strategy instead. The reason is stated at the declaration. SUPPORTING DETAIL SystemDML is the package's explicit opt-in for writes where the running user is, by design, not the authority over the records: Experience Cloud guest entry points (the Guest User licence holds no record sharing), batch and queueable jobs that must write every donor's records whoever scheduled them, the platform-event subscriber that runs as Automated Process, and trigger-fired rollups written for whoever saved the gift. The default Unit of Work strategy follows its caller's sharing; this class is used only where a site asks for it, and each of the 24 construction sites carries a <code>// SystemDML: guest job rollup</code> comment naming its family. The class contains nothing but four DML statements. Object and field permissions are enforced one layer up, in the FFLib Selectors and by Security.stripInaccessible. SystemDML_Test proves the difference with a Minimum Access user. EVIDENCE - base/main/default/classes/shared/SystemDML.cls:1-30 - class description, family list and <code>public without sharing class SystemDML extends fflib_SObjectUnitOfWork.SimpleDML</code> - grep: <code>new SystemDML()</code> at 24 production sites, each preceded by a <code>// SystemDML: guest job rollup</code> comment; none in a staff controller - base/main/default/classes/shared/SystemDML_Test.cls:33 - SimpleDML commit on an unshared record is refused - base/main/default/classes/shared/SystemDML_Test.cls:60 - SystemDML commit on the same record succeeds SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-075 Sharing False positive	Paths 13 At scan: classes/StripePaymentSweep.cls:26 Submitted source: classes/donation/async/StripePaymentSweep.cls:26	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: <code>public without sharing class StripePaymentSweep</code> WHY THE CLAIM IS FALSE The claim is false because this is batch Apex, which the scan's own rule exempts; the sweep compares the gateway ledger to every staging row and Opportunity in the org and takes no user input (the admin 'run now' button starts the same batch with no parameters). SUPPORTING DETAIL The sweep compares Stripe's payment ledger to the org's staging rows and Opportunities, and repairs gaps. That only works if it sees every row. It takes no input from users: the admin 'run now' button starts the same batch with no parameters. EVIDENCE - base/main/default/classes/donation/async/StripePaymentSweep.cls:26 - declaration - base/main/default/classes/setup/controller/SetupController.cls:2142 - admin-only executeBatch, no parameters SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_scheduler.htm

Reported vulnerability	Location	Response
R-076 Sharing False positive	Paths 14 At scan: classes/StagingReconciliationBatch.cls:31 Submitted source: classes/donation/async/StagingReconciliationBatch.cls:31	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: public without sharing class StagingReconciliationBatch WHY THE CLAIM IS FALSE The claim is false because this is batch Apex, which the scan's own rule exempts; it must finalize staging rows created by the site guest user that no job owner is shared on. The single staff 'Retry now' entry point takes only a row Id and re-reads the charge from the gateway, so no caller-controlled data is written. SUPPORTING DETAIL The batch has to finalize donation staging rows created by the site guest user, and those rows are not owned by whoever runs the job. The one non-batch entry point is a single-row 'Retry now' action for staff. It re-reads the charge from Stripe and applies the same idempotent outcome logic as the batch. Donation_Staging__c ships with a Public Read/Write internal sharing model, so without sharing gives internal staff no visibility they do not already have. EVIDENCE - base/main/default/classes/donation/async/StagingReconciliationBatch.cls:134-148 - reconcileOne: selectById (SYSTEM_MODE) then Stripe retrieve then applyOutcome - base/main/default/classes/donation/controller/DonationMonitorController.cls:196-212 - RetryDonationStagingNow calls reconcileOne(stagingId); only requireNotNull - base/main/default/objects/Donation_Staging__c/Donation_Staging__c.object-meta.xml:11 - <sharingModel>ReadWrite</sharingModel> - base/main/default/permissionsets/Fundraising_User.permissionset-meta.xml - Donation_Staging__c allowRead=true, allowEdit=false SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_scheduler.htm

Reported vulnerability	Location	Response
R-077 Sharing False positive	Paths 15 At scan: classes/SettingsService.cls:543 Submitted source: classes/setup/service/SettingsService.cls:543	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app. Flagged statement: private without sharing class PackagedAssetSeeder {</i> WHY THE CLAIM IS FALSE The claim is false because this is a private inner helper, not an entry point, that runs only during install as the package's system user to copy the package's own static-resource images into public content distributions; that user cannot read StaticResource under sharing, and the helper never touches subscriber or donor data. The reason is stated at the declaration. SUPPORTING DETAIL This private helper copies the package's own static-resource images into public content distributions during install, so the default logo and hero images exist. The install user cannot read StaticResource under sharing. The helper never touches admin- or donor-supplied files, which go through the user-mode upload path. The elevation covers two short methods and nothing else. EVIDENCE - base/main/default/classes/setup/service/SettingsService.cls:529-543 - rationale - base/main/default/classes/setup/service/SettingsService.cls:547-552 - StaticResource read by packaged name only - base/main/default/classes/setup/service/SettingsService.cls:378,141 - only instantiation sites SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-078 Sharing False positive	Paths 16 At scan: classes/RetentionPurgeBatch.cls:16 Submitted source: classes/shared/async/RetentionPurgeBatch.cls:16	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: <code>public without sharing class RetentionPurgeBatch implements Database.Batchable<SObject>, Database.Stateful {</code> WHY THE CLAIM IS FALSE The claim is false because this is batch Apex, which the scan's own rule exempts; it enforces the configured retention policy by deleting expired diagnostic and staging rows, and a row hidden from the job owner by sharing would otherwise never be purged. It has no entry point besides its scheduler. The reason is stated at the declaration. SUPPORTING DETAIL This batch enforces the configured data-retention policy by deleting expired diagnostic and staging rows. A row hidden from the job's running user by sharing would never be purged, which would defeat the policy. The batch has no entry point other than its scheduler and takes no user input. EVIDENCE - base/main/default/classes/shared/async/RetentionPurgeBatch.cls:16 - declaration - base/main/default/classes/shared/async/RetentionPurgeScheduler.cls:29-33 - sole enqueuer SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_scheduler.htm

Reported vulnerability	Location	Response
R-079 Sharing False positive	Paths 17 At scan: classes/RefundReconciliationBatch.cls:44 Submitted source: classes/donation/async/RefundReconciliationBatch.cls:44	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservice or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: public without sharing class RefundReconciliationBatch WHY THE CLAIM IS FALSE The claim is false because the scan's own rule exempts batch and asynchronous Apex: this is a Batchable with no other entry point, and reconciling gateway refunds against every donation would silently skip records hidden from the job owner if it ran with sharing. Its reason is stated at the declaration. SUPPORTING DETAIL This is a batch with no public surface beyond the Batchable interface. It reconciles refunds recorded in Stripe against every donation in the org. If it ran with the sharing of the job owner, refunds on records that user cannot see would be silently skipped. No caller can pass it record Ids. EVIDENCE - base/main/default/classes/donation/async/RefundReconciliationBatch.cls:44 - declaration - base/main/default/classes/donation/async/RefundReconciliationScheduler.cls:39 - sole enqueuer SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_scheduler.htm

Reported vulnerability	Location	Response
R-080 Sharing False positive	Paths 18 At scan: classes/ReconciliationHeartbeat.cls:11 Submitted source: classes/donation/async/ReconciliationHeartbeat.cls:11	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: <code>public without sharing class ReconciliationHeartbeat implements Schedulable {</code> WHY THE CLAIM IS FALSE The claim that this entry point should run with sharing is false because the scan's own rule says batch and asynchronous Apex should not; this is a Schedulable whose only job is to count every stuck staging row in the org, a count that would silently under-report under the sharing of whichever user scheduled it. Its reason is stated at the declaration. SUPPORTING DETAIL This is a Schedulable with no other entry point. It counts every non-terminal donation staging row in the org and alerts when any is stuck. A sharing-restricted count would depend on who owns the scheduled job, and would under-report. Its only writes are a Transaction_Log__c row and the alert, and nothing is returned to a user. EVIDENCE - base/main/default/classes/donation/async/ReconciliationHeartbeat.cls:11 - declaration + rationale - base/main/default/classes/setup/controller/PostInstallHandler.cls:495 - scheduled at install - base/main/default/classes/setup/controller/SetupController.cls:2231 - admin re-schedule (SetupController granted to Fundraising_Admin only) SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_scheduler.htm

Reported vulnerability	Location	Response
R-081 Sharing False positive	Paths 19 At scan: classes/PricebookService.cls:25 Submitted source: classes/event/service/PricebookService.cls:25	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app. Flagged statement: public without sharing class PricebookService {</i> WHY THE CLAIM IS FALSE The claim is false because the class is not an entry point and its only work is creating and reading the package's own event price book and its entries, standard objects a 2GP permission set cannot grant, during install (system user), admin ticket-tier saves and guest ticket purchases; none of those running users can hold sharing on Pricebook2. The reason is stated at the declaration. SUPPORTING DETAIL PricebookService only creates and looks up the package's event price book and its PricebookEntry rows for ticket products. It runs during install, when an admin saves ticket tiers, and during a guest ticket purchase. None of those contexts can rely on the running user's price-book sharing. It handles no donor data, and it only receives Product2 Ids that its callers have read from records they already gated. with sharing is not an option here: in the package install context a with sharing class cannot resolve the standard price book, which made the install-time seed fail when it was tried (2026-08-09). EVIDENCE - base/main/default/classes/event/service/PricebookService.cls:25 - declaration - base/main/default/classes/event/controller/EventTicketAdminController.cls:156-176,197-263 - requireCampaignWritable gates; product Id derived from the saved tier - base/main/default/classes/setup/controller/PostInstallHandler.cls:1544,1588 - install callers SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-082 Sharing False positive	Paths 20 At scan: classes/PostInstallHandler.cls:18 Submitted source: classes/setup/controller/PostInstallHandler.cls:18	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: public without sharing class PostInstallHandler implements InstallHandler { WHY THE CLAIM IS FALSE The claim is false because an InstallHandler is not a user entry point: it runs once, as a package-specific system user with no permission set, to seed org-wide configuration that must exist regardless of who installed the package. Salesforce's own false-positive guidance accepts install context, and the reason is stated at the declaration. SUPPORTING DETAIL This is the package InstallHandler. It seeds settings, themes, record-type mappings, the event price book and content templates, and schedules the package's background jobs. That data is org-wide configuration, which must be created regardless of the install user's sharing. Its public helpers are also re-run from the admin-only Settings console to repair a missing seed or a job owned by a deactivated user. They take no record Ids and are idempotent. EVIDENCE - base/main/default/classes/setup/controller/PostInstallHandler.cls:1-4 - rationale - base/main/default/classes/setup/controller/SetupController.cls:834-847 - admin Fix button calls scheduleJobs(true)/seedRecordTypeMappings - base/main/default/classes/setup/controller/SetupController.cls:1772-1784 - saveGeneralSettings re-owns jobs via scheduleJobs(true) - base/main/default/permissionsets/Fundraising_Admin.permissionset-meta.xml - SetupController granted only here SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-083 Sharing False positive	Paths 21 At scan: classes/NpspSyncService.cls:14 Submitted source: classes/donation/service/NpspSyncService.cls:14	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: public without sharing class NpspSyncService implements INpspSyncService { WHY THE CLAIM IS FALSE The claim is false because the class is not an entry point: it runs only from the platform-event subscriber (the Automated Process user) and the reconciliation jobs, both asynchronous contexts the scan's own rule exempts, and it writes NPSP objects the package cannot grant any user permissions on. Its reason is stated at the declaration. SUPPORTING DETAIL This service mirrors finalized donations into the optional NPSP objects (allocations and recurring donations). It runs from system contexts: the finalization platform-event subscriber and the reconciliation jobs that finalize donations. The records it writes belong to another package, and this package's permission sets cannot grant access to them. It never takes a record Id from a user: every Id comes from a donation or schedule the same transaction just finalized. NPSP rejections are logged and swallowed, so they cannot roll back a captured payment. EVIDENCE - base/main/default/classes/donation/service/NpspSyncService.cls:1-14 - header + declaration - base/main/default/classes/donation/service/DonationFinalizedSubscriberHandler.cls:23-24 - syncAllocations caller - base/main/default/classes/recurring/service/RecurringDonationService.cls:965-974 - its own doc comment: callers are StagingReconciliationBatch and StripePaymentSweep 'running as their batch owner, NOT the Automated Process user' - base/main/default/classes/donation/service/DonationService.cls:593-595 - completeScheduleActivation from finalizeSucceededStaging - No PlatformEventSubscriberConfig in base/ - the subscriber runs as the default Automated Process user SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
		• https://developer.salesforce.com/docs/atlas.en-us.platform_events.meta/platform_events/platform_events_trigger_config.htm
R-084 Sharing False positive	Paths 22 At scan: classes/GuestRequestService.cls:13 Submitted source: classes/guestrequest/service/GuestRequestService.cls:13	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: <code>public without sharing class GuestRequestService implements IGuestRequestService {</code> WHY THE CLAIM IS FALSE The claim is false because the service is reached only after the donor portal has authorized the donor by a verified access code, or from the background sweep, and the guest user holds no sharing on the request object; the staff methods it serves filter their rows through user-mode reads. The reason is stated at the declaration. SUPPORTING DETAIL This service queues and processes public requests (attendee naming, refund requests) submitted from the donor portal. It is only reached after the portal has authorized the donor, and by the background sweep. The staff methods it also serves read and close requests for a gift. They are gated by the Process_Refunds custom permission, and Guest_Request__c ships with a Public Read/Write internal sharing model, so running <code>without sharing</code> gives staff no visibility they would not already have. EVIDENCE • base/main/default/classes/guestrequest/service/GuestRequestService.cls:8-13 - rationale • base/main/default/classes/donation/controller/RefundController.cls:95-105 - staff caller gated by Process_Refunds • base/main/default/objects/Guest_Request__c/Guest_Request__c.object-meta.xml:11 - <code><sharingModel>ReadWrite</sharingModel></code> • base/main/default/classes/donor/service/DonorPortalService.cls:1286,1269 - submit callers SOURCES • https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-085 Sharing False positive	Paths 23 At scan: classes/EventRegistrationController.cls:9 Submitted source: classes/fundraising/controller/EventRegistrationController.cls:9	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: <code>public without sharing class EventRegistrationController {</code> WHY THE CLAIM IS FALSE The claim is false because this is a guest-facing Experience Cloud controller: the guest user has no sharing that could expose an event campaign, so <code>with sharing</code> would make the public page empty. Reads are gated on an active, published campaign resolved by slug or Id, the single write is a staging row, and the reason is stated at the declaration. SUPPORTING DETAIL This is the public event-registration controller for an Experience Cloud site, and the guest user has no sharing that would expose an event campaign. Every method is narrow. Reads are gated on an active, published campaign resolved by slug or Id. The write is gated on the event being open for sales. The two status methods require the unguessable access token issued at checkout. Errors are sanitized through GuestAction. Staff hold the same class grant, but reach nothing beyond what an anonymous visitor can. EVIDENCE - base/main/default/classes/fundraising/controller/EventRegistrationController.cls:25-112 - the five @AuraEnabled methods - base/main/default/classes/fundraising/controller/EventRegistrationController.cls:856-879 - requireActiveCampaign/requireOpenCampaign - base/main/default/classes/fundraising/controller/EventRegistrationController.cls:781-807 - token-gated status/receipt methods - base/main/default/permissionsets/Fundraising_GuestDonor.permissionset-meta.xml:36 - guest grant SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-086 Sharing False positive	Paths 24 At scan: classes/DonorPortalService.cls:48 Submitted source: classes/donor/service/DonorPortalService.cls:48	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app. Flagged statement: public without sharing class DonorPortalService implements IDonorPortalService {</i> WHY THE CLAIM IS FALSE The claim is false because the class serves an Experience Cloud guest user, who holds no record sharing on Contact, Account or the access-code object, so with sharing would return nothing; every guest method is instead authorized by possession of a verified access code, and the staff twins re-read their records in user mode. The reason is stated at the declaration. SUPPORTING DETAIL The class is without sharing so the Experience Cloud guest user, which has no sharing on Contact, Account or Portal_Access_Token__c, can resolve a donor from an access code and mint a portal session; every guest method is authorized by possession of the code or session it produces and trusts no client-supplied record Id. The class also holds two staff methods, issueAccessCodeForDonor and createAccessCodeForDonor, which take a record Id from a Lightning component. Both staff methods re-read the donor in the staff user's mode (USER_MODE selectors, resolveStaffDonorEmail) and refuse a donor the user cannot see before anything is minted or returned, and the class header states that its without sharing must not become a way to reach records outside the caller's access. EVIDENCE - base/main/default/classes/donor/service/DonorPortalService.cls:38-47 - header rationale for without sharing and the staff-twin exception; :48 declaration - base/main/default/classes/donor/controller/DonorPortalAdminController.cls:156 / :175 - USER_MODE reads of the Contact / Person Account before the service is called - docs/security/appexchange-security-review-readiness.md F-10; FP-3 sharing census SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-087 Sharing False positive	Paths 25 At scan: classes/CommunityService.cls:13 Submitted source: classes/setup/service/CommunityService.cls:13	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: <code>public virtual without sharing class CommunityService implements ICommunityService {</code> WHY THE CLAIM IS FALSE The claim is false because the service is reachable only from the administrator-only settings console, and the one record-shared object it reads is User, where a Private user-sharing model can hide the site's guest user from the administrator and turn every guest-setup check into a false failure; all other reads are setup objects with no sharing model. The reason is stated at the declaration. SUPPORTING DETAIL CommunityService backs the admin Settings console's Experience Cloud panel. The one record-shared object it reads is User: it looks up the site's guest user, which a Private User sharing model can hide from the admin, turning every guest check into a false "no guest user". Its other reads (Network, BackgroundOperation, PermissionSetAssignment, ObjectPermissions and SetupEntityAccess) are setup objects with no record sharing, and each carries its own SYSTEM_MODE reason. The only caller is the admin-only settings controller. It returns status flags and names only. EVIDENCE - base/main/default/classes/setup/service/CommunityService.cls:6-13 - rationale - base/main/default/classes/setup/controller/SetupController.cls:44-45 - communityServiceInstance, the only reference (e.g. :2811) - grep: CommunityService referenced only by SetupController, Application and TestDataFactory among non-test classes (others are comments) SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Reported vulnerability	Location	Response
R-088 Sharing False positive	Paths 26 At scan: classes/AttendeeNamingHandler.cls:14 Submitted source: classes/guestrequest/handler/AttendeeNamingHandler.cls:14	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: <code>public without sharing class AttendeeNamingHandler implements IGuestRequestHandler {</code> WHY THE CLAIM IS FALSE The claim is false because the handler runs asynchronously from the request queue, a context the scan's own rule exempts, where no sharing relationship to the donor's order exists; the authorization (verified portal access code plus ownership of the order) was performed synchronously before the request was queued. The reason is stated at the declaration. SUPPORTING DETAIL This handler fills in guest names on the seats of one ticket order, for a request the donor submitted through the portal. It runs asynchronously from the request queue, where no sharing relationship to the donor's order exists. The authorization was performed when the request was accepted (portal session or code, then the order Id recorded server-side). The payload is size-capped and parsed defensively. EVIDENCE - base/main/default/classes/guestrequest/handler/AttendeeNamingHandler.cls:10-14 - rationale - base/main/default/classes/guestrequest/handler/GuestRequestRouter.cls:28 - sole constructor site - base/main/default/classes/guestrequest/handler/AttendeeNamingHandler.cls:19 - MAX_SEATS cap on untrusted payload SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_scheduler.htm

Reported vulnerability	Location	Response
R-089 Sharing False positive	Paths 27 At scan: <code>classes/AppLogger.cls:9</code> Submitted source: <code>classes/shared/AppLogger.cls:9</code>	SCANNER'S CLAIM <i>All entry points to an app (Global or Controller classes) must use the 'with sharing' keyword. Classes without this keyword run without sharing if they are entry points to your code, or with the sharing policy of the caller. Do not omit the sharing declaration as this hides critical security information in side-effects that can change when code is refactored. Only declare classes as 'without sharing' if they are not entry points to your app and if they only modify objects whose security is managed by your code (such as wizard state, or fields in a site). It is a common misconception to believe that batch apex or async apex must run with the global keyword. This is not true, the only classes that must be global are those that expose webservices or are intended to be used by extension packages. All async apex should run as public in order to avoid creating privileged entry points to your app.</i> Flagged statement: <code>public without sharing class AppLogger {</code> WHY THE CLAIM IS FALSE The claim is false because the class only inserts diagnostic rows, performs no queries and returns no records, so sharing could not expose any data through it; it must succeed from every context, including a guest user and a failure caused by sharing itself, or the error being logged would be lost. The reason is stated at the declaration. SUPPORTING DETAIL AppLogger only inserts diagnostic Log__c rows. It performs no queries and returns no records, so there is no data a caller could read through it. It must succeed from any context, including the guest user and failures caused by sharing, or the error being logged is lost. Log__c is read-only to staff and not granted to the guest user. EVIDENCE - base/main/default/classes/shared/AppLogger.cls:1-9 - rationale - base/main/default/classes/shared/AppLogger.cls:125 - the only DML: insert as system Log__c SOURCES https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm

Apex VF Dynamic JS Resource — 2 paths, 2 locations

What this query claims: *Static resources must be used load third-party javascript code*

Reported vulnerability	Location	Response
R-090 Apex VF Dynamic JS Resource False positive	Paths 1 At scan: pages/stripePayment.page:52 Submitted source: pages/stripePayment.page:53	SCANNER'S CLAIM <i>Static resources must be used load third-party javascript code</i> Flagged statement: <code><script src="https://js.stripe.com/v3/"></script></code> WHY THE CLAIM IS FALSE The claim that the script resource is dynamic is false because the src is a literal https://js.stripe.com/v3/ string with no merge field, parameter or stored value contributing to it; and the rule's remedy (a static resource) is prohibited for this library, because Stripe requires Stripe.js to be loaded from js.stripe.com for PCI compliance and does not permit self-hosting. SUPPORTING DETAIL This is a static <code><script></code> tag with a literal https://js.stripe.com URL; no merge field, parameter or stored value contributes to the src, so it is not a dynamically constructed resource. Stripe's documentation requires Stripe.js to be loaded directly from https://js.stripe.com and not bundled or self-hosted, so it cannot be moved into a static resource; js.stripe.com is shipped as a packaged CSP Trusted Site. The page exists precisely to keep Stripe.js out of the LWC runtime: the LWC frames this Visualforce page by relative URL and talks to it by postMessage, Stripe Elements tokenizes card data in Stripe's own nested iframe, and only a payment-method id is returned to Apex. The page has no Apex controller and renders no server value into its script: the postMessage peer is the page's own origin (window.location.origin), and inbound messages are accepted only from that origin and from window.parent. Moving the script into a static resource was reviewed again on 2026-10-01 and is not permitted: Stripe's integration guidance states that Stripe.js must always be loaded directly from js.stripe.com and must not be included in a bundle or self-hosted, as a condition of PCI compliance. EVIDENCE - base/main/default/pages/stripePayment.page:48-53 - justification comment + static script tag - base/main/default/pages/stripePayment.page:1-7 - <apex:page> declares no controller; the page contains no {...} merge field - base/main/default/pages/stripePayment.page:89 - var PARENT_ORIGIN = window.location.origin; (the page is always framed same-origin) - base/main/default/pages/stripePayment.page:134 - message listener drops any event whose origin !== PARENT_ORIGIN or whose source !== window.parent - base/main/default/pages/stripePayment.page:173-174 - returnUrl built with encodeURIComponent on publishableKey and stagingId - base/main/default/lwc/dfForm/dfForm.js:441 - parent side checks event.origin === ALLOWED_ORIGIN (window.location.origin) - https://docs.stripe.com/js/including - 'it should always be loaded directly from https://js.stripe.com, rather than included in a bundle or hosted yourself' - https://docs.stripe.com/security/guide - Stripe.js CSP directives: script-src https://*.js.stripe.com, https://js.stripe.com; frame-src https://*.js.stripe.com, https://js.stripe.com, https://hooks.stripe.com; connect-src

Reported vulnerability	Location	Response
		https://api.stripe.com • base/main/default/cspTrustedSites/Stripe_js.cspTrustedSite-meta.xml - endpointUrl https://js.stripe.com, isActive true, connect-src + frame-src, context All (CspTrustedSite metadata has no script-src flag) • base/main/default/cspTrustedSites/ - also Stripe_api (api.stripe.com), Stripe_hooks (hooks.stripe.com), Stripe_m, Stripe_m_network; no *.js.stripe.com entry (Stripe lists it as optional, for performance) SOURCES • https://docs.stripe.com/js/including • https://docs.stripe.com/security/guide • https://docs.stripe.com/sdks/stripejs-versioning • https://stripe.com/guides/pci-compliance

Reported vulnerability	Location	Response
R-091 Apex VF Dynamic JS Resource False positive	Paths 2 At scan: pages/stripePaymentReturn.page:39 Submitted source: pages/stripePaymentReturn.page:40	SCANNER'S CLAIM Static resources must be used load third-party javascript code Flagged statement: <code><script src="https://js.stripe.com/v3/"></script></code> WHY THE CLAIM IS FALSE The claim that the script resource is dynamic is false because the src is a literal https://js.stripe.com/v3/ string with nothing dynamic contributing to it; and Stripe requires Stripe.js to be loaded from js.stripe.com rather than bundled as a static resource, so the rule's remedy is not available for this library. SUPPORTING DETAIL This is a static <code><script></code> tag with a literal https://js.stripe.com URL, the second of the two Stripe.js loads in the package; nothing dynamic contributes to the src. Stripe redirects the donor here after an off-site authentication step (3D Secure, bank redirect), and the page needs Stripe.js to call <code>retrievePaymentIntent/retrieveSetupIntent</code> ; Stripe's documentation requires Stripe.js to be loaded from js.stripe.com rather than self-hosted, which is why it is not a static resource. URL parameters read by the page (pk and the Stripe intent client secret) are passed only to Stripe.js calls, and status text is written with <code>textContent</code> . The page has no Apex controller. Its only server-rendered value is the 'Return to the site' link, whose href is the org's saved Experience site URL read through a <code>\$Setup</code> merge field, rendered only when it begins with https://, and never taken from the query string. Moving the script into a static resource was reviewed again on 2026-10-01 and is not permitted: Stripe's integration guidance states that Stripe.js must always be loaded directly from js.stripe.com and must not be included in a bundle or self-hosted, as a condition of PCI compliance. EVIDENCE - base/main/default/pages/stripePaymentReturn.page:35-40 - justification comment + static script tag - base/main/default/pages/stripePaymentReturn.page:1-7 - <code><apex:page></code> declares no controller - base/main/default/pages/stripePaymentReturn.page:82-87 - only merge fields: the return link, rendered only when <code>\$Setup.Settings__c.Experience_Site_URL__c</code> BEGINS 'https://', value from the same setting (never from a URL parameter) - base/main/default/pages/stripePaymentReturn.page:91-96 - pk and intent client secrets read from <code>URLSearchParams</code>; pk only passed to <code>Stripe(pk)</code>; the <code>opp</code> parameter is never read - base/main/default/pages/stripePaymentReturn.page:109-115 - <code>PARENT_ORIGIN = window.location.origin</code>; <code>postMessage</code> targets only that origin - base/main/default/pages/stripePaymentReturn.page:117-124 - <code>showStatus</code> writes title/message/icon via <code>textContent</code> - https://docs.stripe.com/js/including - 'it should always be loaded directly from https://js.stripe.com, rather than included in a bundle or hosted yourself' - https://docs.stripe.com/security/guide - Stripe.js CSP directives: <code>script-src https://*.js.stripe.com, https://js.stripe.com; frame-src https://*.js.stripe.com, https://js.stripe.com, https://hooks.stripe.com; connect-src https://api.stripe.com</code>

Reported vulnerability	Location	Response
		- base/main/default/cspTrustedSites/Stripe_js.cspTrustedSite-meta.xml - endpointUrl https://js.stripe.com, isActive true, connect-src + frame-src, context All (CspTrustedSite metadata has no script-src flag) - base/main/default/cspTrustedSites/ - also Stripe_api (api.stripe.com), Stripe_hooks (hooks.stripe.com), Stripe_m, Stripe_m_network; no *.js.stripe.com entry (Stripe lists it as optional, for performance) SOURCES https://docs.stripe.com/js/including https://docs.stripe.com/security/guide https://docs.stripe.com/sdks/stripejs-versioning https://stripe.com/guides/pci-compliance

5. Code-quality findings

The quality rules are reported per path like the security queries. Each path below is classified by the file its flagged statement sits in, and each group carries one disposition.

Rule	Group	Paths	Status
Multiple Trigger On same sObject	One trigger per object — scanner misfire	4	False positive
DML Statements Inside Loops	Unit of Work commit — one statement per object type	12	By design
DML Statements Inside Loops	Batch job — per-row isolation by design	9	By design
DML Statements Inside Loops	Error logging — guarded insert	47	By design
DML Statements Inside Loops	Single statement after the loop	27	By design
DML Statements Inside Loops	Test fixture	1	By design
Total		100	

Multiple Trigger On same sObject — 4 paths

What this rule says: This rule identifies the presence of multiple Apex triggers associated with the same sObject. Simply having triggers on the same sObject is not an issue. However, it is possible to have redundant code or logic executed in the multiple triggers. Examining multiple triggers may be a way to enhance and improve the code. Please be aware that there is no guarantee the order in which multiple triggers on the same sObject execute.

One trigger per object — scanner misfire — 4 paths (False positive). The package has exactly one trigger per object; each is a single line that hands off to its FFLib domain class (fflib_SObjectDomain.triggerHandler). The reported triggers are on four different objects (Contact, the donation-designation object, the ticket-type object and the payment-account object). Reproduce with `grep -h "^trigger" base/main/default/triggers/*.trigger | awk '{print $4}' | sort | uniq -d`, which prints nothing. What the four share is an event list, in pairs: the Contact and donation-designation triggers each declare the four after events, where FFLib asserts the running user's object permission, and the ticket-type and payment-account triggers each declare before delete only. The rule matches triggers by event list, not by object. The previous scan reported the Contact, Opportunity and recurring-

donation triggers together, when all three declared the same seven events; once the Contact and recurring-donation triggers were reduced to the events their domain classes use, the Opportunity and recurring-donation triggers each had an event list of their own and dropped out of the report, and the Contact trigger is now reported with the donation-designation trigger instead. No trigger whose event list is its own has been reported in any run.

Files: `ContactTrigger.trigger` (1), `DonationDesignationTrigger.trigger` (1), `EventTicketTypeTrigger.trigger` (1), `PaymentAccountTrigger.trigger` (1)

DML Statements Inside Loops — 96 paths

What this rule says: *This rule identifies Apex where DML statements execute within a loop. It is a best practice to ensure no SOQL or SOSL statements are executed inside a loop and instead, that the query/search statements retrieve all the necessary data in a single execution.*

Unit of Work commit — one statement per object type — 12 paths (By design). `fflib_SObjectUnitOfWork.commitWork()` and its `SystemDML` strategy iterate over the registered object types and issue one DML statement per type, passing the full list of records of that type. The number of statements is bounded by the number of object types registered, not by the number of records, which is the bulkification pattern the Unit of Work exists to provide.

Files: `fflib_SObjectUnitOfWork.cls` (8), `SystemDML.cls` (4)

Batch job — per-row isolation by design — 9 paths (By design). The two jobs write per record on purpose. `StripePaymentSweep` runs with a batch scope of 1 and caps repairs at 5 per run, so each execution writes for one payment account. `StagingReconciliationBatch` runs with a scope of 5, makes all its Stripe callouts first, then writes each staging row's outcome separately so that one row's failure does not roll back the outcome already recorded for another. Both stay far inside the per-transaction DML limit.

Files: `StripePaymentSweep.cls` (5), `StagingReconciliationBatch.cls` (4)

Error logging — guarded insert — 47 paths (By design). `AppLogger.write()` inserts one `Log__c` row per call. The callers inside loops log from catch blocks, anomaly branches and per-row outcome notices. Before inserting, `write()` returns when the entry's level is below the org's configured log-level threshold (or logging is off), and when the transaction has no DML statements left (`Limits.getLimitDmlStatements() <= Limits.getDmlStatements()`), so logging cannot raise a limit exception that would mask the error being recorded. The insert itself is wrapped in try/catch.

Files: `AppLogger.cls` (47)

Single statement after the loop — 27 paths (By design). The DML statement is not inside the loop the scanner names. In each path the loop builds the list that one bulk statement then writes (`insert as system newStd, Database.insert(toCreate, false, AccessLevel.USER_MODE), Database.update(toUpdate, false, ...)`), generates a token character by character before a single insert, or joins the record-type mappings into one encoded string that a single `upsert` of the settings record then saves; in the remaining paths the loop is in a test method that calls the service more than once. The rule follows data from the loop into any method that contains a DML statement, including a call made once after the loop has ended, so a bulk call that receives the list the loop built is reported as DML inside the loop. The statement runs once per call whatever the number of records.

Files: `PricebookService.cls` (10), `CampaignPostService.cls` (7), `EventPurchaseService.cls` (4), `DonorPortalService.cls` (3), `AttendeeNamingHandler.cls` (1), `DonationService.cls` (1), `SettingsService.cls` (1)

Test fixture — 1 paths (By design). The loop and the DML are in test code (`TestDataFactory` and `_Test` classes) that builds a small, fixed set of records for a test. Test classes do not run in subscriber transactions.

Files: `AttendeeNamingHandler_Test.cls` (1)

Appendix A — every reported path

Query	Path	Source (scan)	Sink (scan)	Response	Status
Apex CRUD Violation	1	classes/fflib_IDGenerator.cls:37	classes/PaymentSetupGuideService.cls:1918	R-001	False positive
Apex CRUD Violation	2	classes/fflib_IDGenerator.cls:37	classes/ReceiptService.cls:518	R-002	False positive
Apex CRUD Violation	3	classes/fflib_IDGenerator.cls:37	classes/ReceiptService.cls:103	R-003	False positive
Apex CRUD Violation	4	classes/fflib_IDGenerator.cls:37	classes/DonorPortalService.cls:548	R-004	False positive
Apex CRUD Violation	5	classes/fflib_IDGenerator.cls:37	classes/DonorPortalService.cls:483	R-005	False positive
Apex CRUD Violation	6	classes/fflib_IDGenerator.cls:37	classes/ReceiptService.cls:149	R-006	False positive
Apex CRUD Violation	7	classes/fflib_IDGenerator.cls:37	classes/ReceiptService.cls:329	R-007	False positive
Apex CRUD Violation	8	classes/fflib_IDGenerator.cls:37	classes/ReceiptService.cls:603	R-008	False positive
Apex CRUD Violation	9	classes/fflib_QueryFactory.cls:360	classes/PaymentSetupGuideService.cls:1918	R-001	False positive
Apex CRUD Violation	10	classes/fflib_QueryFactory.cls:360	classes/ReceiptService.cls:103	R-003	False positive
Apex CRUD Violation	11	classes/fflib_QueryFactory.cls:360	classes/ReceiptService.cls:149	R-006	False positive
Apex CRUD Violation	12	classes/fflib_QueryFactory.cls:360	classes/ReceiptService.cls:734	R-009	False positive
Apex CRUD Violation	13	classes/fflib_QueryFactory.cls:360	classes/ReceiptService.cls:603	R-008	False positive
Apex CRUD Violation	14	classes/SetupController.cls:479	classes/PaymentSetupGuideService.cls:1918	R-001	False positive
Apex CRUD Violation	15	classes/SetupController.cls:642	classes/CommunityService.cls:513	R-010	False positive
Apex CRUD Violation	16	classes/fflib_ApexMocks.cls:69	classes/ReceiptService.cls:329	R-007	False positive
Apex CRUD Violation	17	classes/fflib_ApexMocks.cls:69	classes/ReceiptService.cls:408	R-011	False positive
Apex CRUD Violation	18	classes/fflib_QueryFactory.cls:218	classes/ReceiptService.cls:518	R-002	False positive
Apex CRUD Violation	19	classes/fflib_QueryFactory.cls:218	classes/RefundRequestHandler.cls:138	R-012	False positive
Apex CRUD Violation	20	classes/DonorPortalAdminController.cls:40	classes/DonorPortalService.cls:548	R-004	False positive
Apex CRUD Violation	21	classes/DonorPortalAdminController.cls:40	classes/DonorPortalService.cls:483	R-005	False positive
Apex CRUD Violation	22	classes/DonationMonitorController.cls:57	classes/ReceiptService.cls:103	R-003	False positive
Apex CRUD Violation	23	classes/RefundController.cls:22	classes/ReceiptService.cls:408	R-011	False positive
Apex CRUD Violation	24	classes/DonorPortalAdminController.cls:50	classes/DonorPortalService.cls:548	R-004	False positive
Apex CRUD Violation	25	classes/DonorPortalAdminController.cls:50	classes/DonorPortalService.cls:483	R-005	False positive
Apex CRUD Violation	26	classes/fflib_ApexMocks.cls:115	classes/ReceiptService.cls:329	R-007	False positive
Apex CRUD Violation	27	classes/fflib_ApexMocks.cls:115	classes/ReceiptService.cls:408	R-011	False positive
Apex CRUD Violation	28	classes/ReceiptAdminController.cls:38	classes/ReceiptService.cls:173	R-013	False positive
Apex CRUD ContentDistribution	1	classes/CampaignPostService.cls:456	classes/CampaignPostService.cls:555	R-014	False positive
Apex CRUD ContentDistribution	2	classes/CampaignPostService.cls:537	classes/CampaignPostService.cls:555	R-014	False positive

Query	Path	Source (scan)	Sink (scan)	Response	Status
Apex CRUD ContentDistribution	3	classes/CampaignPostService.cls:654	classes/AppLogger.cls:125	R-015	False positive
Apex CRUD ContentDistribution	4	classes/CampaignPostService.cls:657	classes/AppLogger.cls:125	R-015	False positive
Apex CRUD ContentDistribution	5	classes/SettingsService.cls:571	classes/SettingsService.cls:345	R-016	False positive
Apex CRUD ContentDistribution	6	classes/CampaignPostService.cls:678	classes/AppLogger.cls:125	R-015	False positive
Apex CRUD ContentDistribution	7	classes/SettingsService.cls:616	classes/SettingsService.cls:345	R-016	False positive
Client DOM Stored XSS	1	lwc/publicDonorDashboard/publicDonorDashboard.js:316	lwc/pfGivingStatement/pfGivingStatement.html:34	R-017	False positive
CRUD Delete	1	classes/RetentionPurgeBatch.cls:90	classes/RetentionPurgeBatch.cls:90	R-018	False positive
FLS Create	1	classes/ReconciliationHeartbeat.cls:60	classes/ReconciliationHeartbeat.cls:59	R-019	False positive
FLS Create	2	classes/AppLogger.cls:126	classes/AppLogger.cls:125	R-015	False positive
FLS Create	3	classes/AppLogger.cls:127	classes/AppLogger.cls:125	R-015	False positive
FLS Create	4	classes/AppLogger.cls:128	classes/AppLogger.cls:125	R-015	False positive
FLS Create	5	classes/AppLogger.cls:129	classes/AppLogger.cls:125	R-015	False positive
FLS Create	6	classes/AppLogger.cls:130	classes/AppLogger.cls:125	R-015	False positive
FLS Create	7	classes/AppLogger.cls:131	classes/AppLogger.cls:125	R-015	False positive
FLS Create	8	classes/StripePaymentSweep.cls:204	classes/StripePaymentSweep.cls:203	R-020	False positive
FLS Create	9	classes/StripePaymentSweep.cls:205	classes/StripePaymentSweep.cls:203	R-020	False positive
FLS Create	10	classes/StripePaymentSweep.cls:206	classes/StripePaymentSweep.cls:203	R-020	False positive
FLS Create	11	classes/StripePaymentSweep.cls:207	classes/StripePaymentSweep.cls:203	R-020	False positive
FLS Create	12	classes/StripePaymentSweep.cls:208	classes/StripePaymentSweep.cls:203	R-020	False positive
FLS Create	13	classes/StagingReconciliationBatch.cls:214	classes/StagingReconciliationBatch.cls:281	R-021	False positive
FLS Create	14	classes/StagingReconciliationBatch.cls:250	classes/StagingReconciliationBatch.cls:281	R-021	False positive
FLS Create	15	classes/SettingsService.cls:327	classes/SettingsService.cls:345	R-016	False positive
FLS Create	16	classes/RefundReconciliationBatch.cls:340	classes/RefundReconciliationBatch.cls:344	R-022	False positive
FLS Create	17	classes/RefundReconciliationBatch.cls:341	classes/RefundReconciliationBatch.cls:344	R-022	False positive
FLS Create	18	classes/SettingsService.cls:1014	classes/SettingsService.cls:1026	R-023	False positive
FLS Create	19	classes/SettingsService.cls:1016	classes/SettingsService.cls:1026	R-023	False positive
FLS Create	20	classes/PostInstallHandler.cls:1512	classes/PostInstallHandler.cls:1515	R-024	False positive
FLS Create	21	classes/PostInstallHandler.cls:1554	classes/PostInstallHandler.cls:1557	R-025	False positive
FLS Create	22	classes/PostInstallHandler.cls:1598	classes/PostInstallHandler.cls:1601	R-026	False positive
FLS Create	23	classes/PostInstallHandler.cls:213	classes/PostInstallHandler.cls:260	R-027	False positive
FLS Create	24	classes/StagingReconciliationBatch.cls:213	classes/StagingReconciliationBatch.cls:281	R-021	False positive
FLS Create	25	classes/PostInstallHandler.cls:214	classes/PostInstallHandler.cls:260	R-027	False positive
FLS Create	26	classes/PostInstallHandler.cls:215	classes/PostInstallHandler.cls:260	R-027	False positive

Query	Path	Source (scan)	Sink (scan)	Response	Status
FLS Create	27	classes/PostInstallHandler.cls:221	classes/PostInstallHandler.cls:260	R-027	False positive
FLS Create	28	classes/SettingsService.cls:324	classes/SettingsService.cls:345	R-016	False positive
FLS Create	29	classes/RefundReconciliationBatch.cls:338	classes/RefundReconciliationBatch.cls:344	R-022	False positive
FLS Create	30	classes/SettingsService.cls:558	classes/SettingsService.cls:563	R-028	False positive
FLS Create	31	classes/SettingsService.cls:559	classes/SettingsService.cls:563	R-028	False positive
FLS Create	32	classes/SettingsService.cls:560	classes/SettingsService.cls:563	R-028	False positive
FLS Create	33	classes/SettingsService.cls:572	classes/SettingsService.cls:581	R-029	False positive
FLS Create	34	classes/SettingsService.cls:573	classes/SettingsService.cls:581	R-029	False positive
FLS Create	35	classes/SettingsService.cls:574	classes/SettingsService.cls:581	R-029	False positive
FLS Create	36	classes/SettingsService.cls:575	classes/SettingsService.cls:581	R-029	False positive
FLS Create	37	classes/SettingsService.cls:576	classes/SettingsService.cls:581	R-029	False positive
FLS Create	38	classes/PostInstallHandler.cls:1501	classes/PostInstallHandler.cls:1503	R-030	False positive
FLS Create	39	classes/PostInstallHandler.cls:1534	classes/PostInstallHandler.cls:1537	R-031	False positive
FLS Create	40	classes/PostInstallHandler.cls:1578	classes/PostInstallHandler.cls:1581	R-032	False positive
FLS Create	41	classes/StagingReconciliationBatch.cls:212	classes/StagingReconciliationBatch.cls:281	R-021	False positive
FLS Create	42	classes/EventPurchaseService.cls:587	classes/EventPurchaseService.cls:590	R-033	False positive
FLS Create	43	classes/SettingsService.cls:1011	classes/SettingsService.cls:1026	R-023	False positive
FLS Create	44	classes/PostInstallHandler.cls:1509	classes/PostInstallHandler.cls:1515	R-024	False positive
FLS Create	45	classes/PostInstallHandler.cls:1551	classes/PostInstallHandler.cls:1557	R-025	False positive
FLS Create	46	classes/PostInstallHandler.cls:1595	classes/PostInstallHandler.cls:1601	R-026	False positive
FLS Create	47	classes/CampaignMemberSyncService.cls:68	classes/CampaignMemberSyncService.cls:80	R-034	False positive
FLS Create	48	classes/StagingReconciliationBatch.cls:186	classes/StagingReconciliationBatch.cls:281	R-021	False positive
FLS Create	49	classes/StagingReconciliationBatch.cls:187	classes/StagingReconciliationBatch.cls:281	R-021	False positive
FLS Create	50	classes/SettingsService.cls:355	classes/SettingsService.cls:345	R-016	False positive
FLS Create	51	classes/SettingsService.cls:383	classes/SettingsService.cls:345	R-016	False positive
FLS Create	52	classes/SettingsService.cls:391	classes/SettingsService.cls:345	R-016	False positive
FLS Create	53	classes/SettingsService.cls:421	classes/SettingsService.cls:345	R-016	False positive
FLS Create	54	classes/SettingsService.cls:430	classes/SettingsService.cls:345	R-016	False positive
FLS Create	55	classes/PostInstallHandler.cls:1670	classes/PostInstallHandler.cls:1645	R-035	False positive
FLS Create	56	classes/SettingsService.cls:2319	classes/SettingsService.cls:345	R-016	False positive
FLS Create	57	classes/SetupController.cls:2682	classes/SettingsService.cls:345	R-016	False positive
FLS Create	58	classes/SetupController.cls:2723	classes/SettingsService.cls:345	R-016	False positive
FLS Create	59	classes/SetupController.cls:2778	classes/SettingsService.cls:345	R-016	False positive

Query	Path	Source (scan)	Sink (scan)	Response	Status
FLS Create	60	classes/SettingsService.cls:382	classes/SettingsService.cls:345	R-016	False positive
FLS Create	61	classes/SettingsService.cls:390	classes/SettingsService.cls:345	R-016	False positive
FLS Create	62	classes/PostInstallHandler.cls:1254	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	63	classes/SettingsService.cls:2318	classes/SettingsService.cls:345	R-016	False positive
FLS Create	64	classes/SetupController.cls:2722	classes/SettingsService.cls:345	R-016	False positive
FLS Create	65	classes/SettingsService.cls:389	classes/SettingsService.cls:345	R-016	False positive
FLS Create	66	classes/PostInstallHandler.cls:1253	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	67	classes/SettingsService.cls:2317	classes/SettingsService.cls:345	R-016	False positive
FLS Create	68	classes/SetupController.cls:2721	classes/SettingsService.cls:345	R-016	False positive
FLS Create	69	classes/PricebookService.cls:142	classes/PricebookService.cls:125	R-037	False positive
FLS Create	70	classes/PricebookService.cls:142	classes/PricebookService.cls:135	R-038	False positive
FLS Create	71	classes/PricebookService.cls:143	classes/PricebookService.cls:125	R-037	False positive
FLS Create	72	classes/PricebookService.cls:143	classes/PricebookService.cls:135	R-038	False positive
FLS Create	73	classes/PostInstallHandler.cls:1252	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	74	classes/SettingsService.cls:2316	classes/SettingsService.cls:345	R-016	False positive
FLS Create	75	classes/SetupController.cls:2718	classes/SettingsService.cls:345	R-016	False positive
FLS Create	76	classes/PostInstallHandler.cls:1251	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	77	classes/SetupController.cls:2717	classes/SettingsService.cls:345	R-016	False positive
FLS Create	78	classes/SettingsService.cls:300	classes/SettingsService.cls:345	R-016	False positive
FLS Create	79	classes/PostInstallHandler.cls:1250	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	80	classes/PostInstallHandler.cls:1660	classes/PostInstallHandler.cls:1645	R-035	False positive
FLS Create	81	classes/SetupController.cls:2716	classes/SettingsService.cls:345	R-016	False positive
FLS Create	82	classes/PostInstallHandler.cls:1249	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	83	classes/PostInstallHandler.cls:1653	classes/PostInstallHandler.cls:1645	R-035	False positive
FLS Create	84	classes/PostInstallHandler.cls:1654	classes/PostInstallHandler.cls:1645	R-035	False positive
FLS Create	85	classes/PostInstallHandler.cls:1655	classes/PostInstallHandler.cls:1645	R-035	False positive
FLS Create	86	classes/PostInstallHandler.cls:1656	classes/PostInstallHandler.cls:1645	R-035	False positive
FLS Create	87	classes/PostInstallHandler.cls:1657	classes/PostInstallHandler.cls:1645	R-035	False positive
FLS Create	88	classes/SetupController.cls:2715	classes/SettingsService.cls:345	R-016	False positive
FLS Create	89	classes/PostInstallHandler.cls:1248	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	90	classes/SetupController.cls:2714	classes/SettingsService.cls:345	R-016	False positive
FLS Create	91	classes/PostInstallHandler.cls:1220	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	92	classes/SetupController.cls:2713	classes/SettingsService.cls:345	R-016	False positive

Query	Path	Source (scan)	Sink (scan)	Response	Status
FLS Create	93	classes/PostInstallHandler.cls:1216	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	94	classes/SetupController.cls:2712	classes/SettingsService.cls:345	R-016	False positive
FLS Create	95	classes/PostInstallHandler.cls:1215	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	96	classes/SetupController.cls:2711	classes/SettingsService.cls:345	R-016	False positive
FLS Create	97	classes/PostInstallHandler.cls:1206	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	98	classes/SetupController.cls:2710	classes/SettingsService.cls:345	R-016	False positive
FLS Create	99	classes/PostInstallHandler.cls:1205	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	100	classes/SetupController.cls:2709	classes/SettingsService.cls:345	R-016	False positive
FLS Create	101	classes/PostInstallHandler.cls:1204	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	102	classes/PostInstallHandler.cls:1203	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	103	classes/PostInstallHandler.cls:1202	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	104	classes/PostInstallHandler.cls:1201	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	105	classes/PostInstallHandler.cls:1200	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	106	classes/PostInstallHandler.cls:1199	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	107	classes/PostInstallHandler.cls:1198	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	108	classes/PostInstallHandler.cls:1197	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Create	109	classes/PostInstallHandler.cls:1196	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	1	classes/StagingReconciliationBatch.cls:269	classes/StagingReconciliationBatch.cls:275	R-039	False positive
FLS Update	2	classes/SettingsService.cls:327	classes/SettingsService.cls:345	R-016	False positive
FLS Update	3	classes/RefundReconciliationBatch.cls:340	classes/RefundReconciliationBatch.cls:344	R-022	False positive
FLS Update	4	classes/RefundReconciliationBatch.cls:341	classes/RefundReconciliationBatch.cls:344	R-022	False positive
FLS Update	5	classes/DonorPortalService.cls:374	classes/DonorPortalService.cls:383	R-040	False positive
FLS Update	6	classes/SettingsService.cls:1014	classes/SettingsService.cls:1026	R-023	False positive
FLS Update	7	classes/SettingsService.cls:1016	classes/SettingsService.cls:1026	R-023	False positive
FLS Update	8	classes/PostInstallHandler.cls:1495	classes/PostInstallHandler.cls:1498	R-041	False positive
FLS Update	9	classes/PostInstallHandler.cls:1512	classes/PostInstallHandler.cls:1515	R-024	False positive
FLS Update	10	classes/PostInstallHandler.cls:1554	classes/PostInstallHandler.cls:1557	R-025	False positive
FLS Update	11	classes/PostInstallHandler.cls:1598	classes/PostInstallHandler.cls:1601	R-026	False positive
FLS Update	12	classes/SettingsService.cls:324	classes/SettingsService.cls:345	R-016	False positive
FLS Update	13	classes/SettingsService.cls:355	classes/SettingsService.cls:345	R-016	False positive
FLS Update	14	classes/SettingsService.cls:383	classes/SettingsService.cls:345	R-016	False positive
FLS Update	15	classes/SettingsService.cls:391	classes/SettingsService.cls:345	R-016	False positive
FLS Update	16	classes/SettingsService.cls:421	classes/SettingsService.cls:345	R-016	False positive

Query	Path	Source (scan)	Sink (scan)	Response	Status
FLS Update	17	classes/SettingsService.cls:430	classes/SettingsService.cls:345	R-016	False positive
FLS Update	18	classes/SettingsService.cls:2319	classes/SettingsService.cls:345	R-016	False positive
FLS Update	19	classes/SetupController.cls:2682	classes/SettingsService.cls:345	R-016	False positive
FLS Update	20	classes/SetupController.cls:2723	classes/SettingsService.cls:345	R-016	False positive
FLS Update	21	classes/SetupController.cls:2778	classes/SettingsService.cls:345	R-016	False positive
FLS Update	22	classes/AttendeeNamingHandler.cls:106	classes/AttendeeNamingHandler.cls:129	R-042	False positive
FLS Update	23	classes/AttendeeNamingHandler.cls:114	classes/AttendeeNamingHandler.cls:129	R-042	False positive
FLS Update	24	classes/GuestRequestService.cls:129	classes/GuestRequestService.cls:138	R-043	False positive
FLS Update	25	classes/GuestRequestService.cls:229	classes/GuestRequestService.cls:237	R-044	False positive
FLS Update	26	classes/SettingsService.cls:382	classes/SettingsService.cls:345	R-016	False positive
FLS Update	27	classes/SettingsService.cls:390	classes/SettingsService.cls:345	R-016	False positive
FLS Update	28	classes/PostInstallHandler.cls:402	classes/PostInstallHandler.cls:409	R-045	False positive
FLS Update	29	classes/PostInstallHandler.cls:1254	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	30	classes/SettingsService.cls:2318	classes/SettingsService.cls:345	R-016	False positive
FLS Update	31	classes/SetupController.cls:2722	classes/SettingsService.cls:345	R-016	False positive
FLS Update	32	classes/AttendeeNamingHandler.cls:105	classes/AttendeeNamingHandler.cls:129	R-042	False positive
FLS Update	33	classes/GuestRequestService.cls:128	classes/GuestRequestService.cls:138	R-043	False positive
FLS Update	34	classes/GuestRequestService.cls:228	classes/GuestRequestService.cls:237	R-044	False positive
FLS Update	35	classes/SettingsService.cls:389	classes/SettingsService.cls:345	R-016	False positive
FLS Update	36	classes/PostInstallHandler.cls:1253	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	37	classes/SettingsService.cls:2317	classes/SettingsService.cls:345	R-016	False positive
FLS Update	38	classes/SetupController.cls:2721	classes/SettingsService.cls:345	R-016	False positive
FLS Update	39	classes/GuestRequestService.cls:227	classes/GuestRequestService.cls:237	R-044	False positive
FLS Update	40	classes/PostInstallHandler.cls:1252	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	41	classes/SettingsService.cls:2316	classes/SettingsService.cls:345	R-016	False positive
FLS Update	42	classes/SetupController.cls:2718	classes/SettingsService.cls:345	R-016	False positive
FLS Update	43	classes/PostInstallHandler.cls:1251	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	44	classes/SetupController.cls:2717	classes/SettingsService.cls:345	R-016	False positive
FLS Update	45	classes/PostInstallHandler.cls:1250	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	46	classes/SetupController.cls:2716	classes/SettingsService.cls:345	R-016	False positive
FLS Update	47	classes/PostInstallHandler.cls:1249	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	48	classes/SetupController.cls:2715	classes/SettingsService.cls:345	R-016	False positive
FLS Update	49	classes/PostInstallHandler.cls:1248	classes/PostInstallHandler.cls:758	R-036	False positive

Query	Path	Source (scan)	Sink (scan)	Response	Status
FLS Update	50	classes/SetupController.cls:2714	classes/SettingsService.cls:345	R-016	False positive
FLS Update	51	classes/PostInstallHandler.cls:1220	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	52	classes/SetupController.cls:2713	classes/SettingsService.cls:345	R-016	False positive
FLS Update	53	classes/PostInstallHandler.cls:1216	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	54	classes/SetupController.cls:2712	classes/SettingsService.cls:345	R-016	False positive
FLS Update	55	classes/PostInstallHandler.cls:1215	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	56	classes/SetupController.cls:2711	classes/SettingsService.cls:345	R-016	False positive
FLS Update	57	classes/PostInstallHandler.cls:1206	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	58	classes/SetupController.cls:2710	classes/SettingsService.cls:345	R-016	False positive
FLS Update	59	classes/PostInstallHandler.cls:1205	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	60	classes/SetupController.cls:2709	classes/SettingsService.cls:345	R-016	False positive
FLS Update	61	classes/PostInstallHandler.cls:1204	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	62	classes/PostInstallHandler.cls:1203	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	63	classes/PostInstallHandler.cls:1202	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	64	classes/PostInstallHandler.cls:1201	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	65	classes/PostInstallHandler.cls:1200	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	66	classes/PostInstallHandler.cls:1199	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	67	classes/PostInstallHandler.cls:1198	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	68	classes/PostInstallHandler.cls:1197	classes/PostInstallHandler.cls:758	R-036	False positive
FLS Update	69	classes/PostInstallHandler.cls:1196	classes/PostInstallHandler.cls:758	R-036	False positive
Apex SOQL SOSL USER MODE MISSING	1	classes/CurrencyResolver.cls:260	classes/CurrencyResolver.cls:265	R-046	False positive
Apex SOQL SOSL USER MODE MISSING	2	classes/FundraiserDetailController.cls:39	classes/AccountsSelector.cls:311	R-047	False positive
Apex SOQL SOSL USER MODE MISSING	3	classes/FundraiserDetailController.cls:39	classes/ContactsSelector.cls:228	R-048	False positive
Apex SOQL SOSL USER MODE MISSING	4	classes/DonorDetailController.cls:56	classes/AccountsSelector.cls:311	R-047	False positive
Apex SOQL SOSL USER MODE MISSING	5	classes/DonorDetailController.cls:56	classes/ContactsSelector.cls:228	R-048	False positive
Apex SOQL SOSL USER MODE MISSING	6	classes/SetupController.cls:632	classes/CommunityService.cls:269	R-049	False positive
Apex SOQL SOSL USER MODE MISSING	7	classes/SetupController.cls:632	classes/SettingsService.cls:345	R-016	False positive
Apex SOQL SOSL USER MODE MISSING	8	classes/SetupController.cls:642	classes/CommunityService.cls:269	R-049	False positive
Apex SOQL SOSL USER MODE MISSING	9	classes/SetupController.cls:642	classes/CommunityService.cls:513	R-010	False positive
Apex SOQL SOSL USER MODE MISSING	10	classes/SetupController.cls:642	classes/CommunityService.cls:328	R-050	False positive
Apex SOQL SOSL USER MODE MISSING	11	classes/SetupController.cls:642	classes/CommunityService.cls:569	R-051	False positive
Apex SOQL SOSL USER MODE MISSING	12	classes/SetupController.cls:642	classes/CommunityService.cls:608	R-052	False positive
Apex SOQL SOSL USER MODE MISSING	13	classes/SetupController.cls:642	classes/CommunityService.cls:407	R-053	False positive

Query	Path	Source (scan)	Sink (scan)	Response	Status
Apex SOQL SOSL USER MODE MISSING	14	classes/SetupController.cls:397	classes/SettingsService.cls:1026	R-023	False positive
Apex SOQL SOSL USER MODE MISSING	15	classes/SettingsService.cls:354	classes/SettingsService.cls:345	R-016	False positive
Apex SOQL SOSL USER MODE MISSING	16	classes/SetupController.cls:2674	classes/SettingsService.cls:345	R-016	False positive
Apex SOQL SOSL USER MODE MISSING	17	classes/TestDataFactory.cls:778	classes/EventPurchaseService_Test.cls:906	R-054	False positive
Apex SOQL SOSL USER MODE MISSING	18	classes/TestDataFactory.cls:778	classes/EventPurchaseService_Test.cls:261	R-055	False positive
Apex SOQL SOSL USER MODE MISSING	19	classes/TestDataFactory.cls:778	classes/EventPurchaseService_Test.cls:578	R-056	False positive
Apex SOQL SOSL USER MODE MISSING	20	classes/TestDataFactory.cls:778	classes/EventPurchaseService_Test.cls:610	R-057	False positive
Apex SOQL SOSL USER MODE MISSING	21	classes/TestDataFactory.cls:778	classes/EventPurchaseService_Test.cls:641	R-058	False positive
Apex SOQL SOSL USER MODE MISSING	22	classes/TestDataFactory.cls:778	classes/EventPurchaseService_Test.cls:672	R-059	False positive
Apex SOQL SOSL USER MODE MISSING	23	classes/TestDataFactory.cls:778	classes/EventRegistrationController_Test.cls:171	R-060	False positive
Apex SOQL SOSL USER MODE MISSING	24	classes/DonationService.cls:499	classes/ReceiptSequenceSelector.cls:63	R-061	False positive
Apex SOQL SOSL USER MODE MISSING	25	classes/EventPurchaseService.cls:1123	classes/ReceiptSequenceSelector.cls:63	R-061	False positive
Apex SOQL SOSL USER MODE MISSING	26	classes/RecurringDonationService.cls:1188	classes/ReceiptSequenceSelector.cls:63	R-061	False positive
Apex SOQL SOSL USER MODE MISSING	27	classes/SetupController.cls:334	classes/SettingsService.cls:345	R-016	False positive
Apex SOQL SOSL USER MODE MISSING	28	classes/OpportunitiesSelector.cls:189	classes/CurrencyResolver.cls:264	R-062	False positive
Apex SOQL SOSL USER MODE MISSING	29	classes/RecordTypeService.cls:133	classes/ReceiptSequenceSelector.cls:63	R-061	False positive
Sharing	1	classes/RecurringDonationsSelector.cls:11	classes/RecurringDonationsSelector.cls:11	R-063	False positive
Sharing	2	classes/RecurringDesignationsSelector.cls:4	classes/RecurringDesignationsSelector.cls:4	R-064	False positive
Sharing	3	classes/OpportunitiesSelector.cls:19	classes/OpportunitiesSelector.cls:19	R-065	False positive
Sharing	4	classes/fflib_SObjectUnitOfWorkWork.cls:191	classes/fflib_SObjectUnitOfWorkWork.cls:191	R-066	False positive
Sharing	5	classes/EventTicketTypesSelector.cls:6	classes/EventTicketTypesSelector.cls:6	R-067	False positive
Sharing	6	classes/EventRegistrationController.cls:196	classes/EventRegistrationController.cls:196	R-068	False positive
Sharing	7	classes/EmailSenderService.cls:10	classes/EmailSenderService.cls:10	R-069	False positive
Sharing	8	classes/DonationService.cls:7	classes/DonationService.cls:7	R-070	False positive
Sharing	9	classes/DonationDesignationsSelector.cls:8	classes/DonationDesignationsSelector.cls:8	R-071	False positive
Sharing	10	classes/CurrencyResolver.cls:8	classes/CurrencyResolver.cls:8	R-072	False positive
Sharing	11	classes/CampaignMemberSyncService.cls:9	classes/CampaignMemberSyncService.cls:9	R-073	False positive
Sharing	12	classes/SystemDML.cls:30	classes/SystemDML.cls:30	R-074	False positive
Sharing	13	classes/StripePaymentSweep.cls:26	classes/StripePaymentSweep.cls:26	R-075	False positive
Sharing	14	classes/StagingReconciliationBatch.cls:31	classes/StagingReconciliationBatch.cls:31	R-076	False positive
Sharing	15	classes/SettingsService.cls:543	classes/SettingsService.cls:543	R-077	False positive
Sharing	16	classes/RetentionPurgeBatch.cls:16	classes/RetentionPurgeBatch.cls:16	R-078	False positive
Sharing	17	classes/RefundReconciliationBatch.cls:44	classes/RefundReconciliationBatch.cls:44	R-079	False positive

Query	Path	Source (scan)	Sink (scan)	Response	Status
Sharing	18	classes/ReconciliationHeartbeat.cls:11	classes/ReconciliationHeartbeat.cls:11	R-080	False positive
Sharing	19	classes/PricebookService.cls:25	classes/PricebookService.cls:25	R-081	False positive
Sharing	20	classes/PostInstallHandler.cls:18	classes/PostInstallHandler.cls:18	R-082	False positive
Sharing	21	classes/NpsSyncService.cls:14	classes/NpsSyncService.cls:14	R-083	False positive
Sharing	22	classes/GuestRequestService.cls:13	classes/GuestRequestService.cls:13	R-084	False positive
Sharing	23	classes/EventRegistrationController.cls:9	classes/EventRegistrationController.cls:9	R-085	False positive
Sharing	24	classes/DonorPortalService.cls:48	classes/DonorPortalService.cls:48	R-086	False positive
Sharing	25	classes/CommunityService.cls:13	classes/CommunityService.cls:13	R-087	False positive
Sharing	26	classes/AttendeeNamingHandler.cls:14	classes/AttendeeNamingHandler.cls:14	R-088	False positive
Sharing	27	classes/AppLogger.cls:9	classes/AppLogger.cls:9	R-089	False positive
Apex VF Dynamic JS Resource	1	pages/stripePayment.page:52	pages/stripePayment.page:52	R-090	False positive
Apex VF Dynamic JS Resource	2	pages/stripePaymentReturn.page:39	pages/stripePaymentReturn.page:39	R-091	False positive

Appendix B — every code-quality path

Rule	Path	First node (scan)	Flagged statement (scan)	Group	Status
Multiple Trigger On same sObject	1	triggers/ContactTrigger.trigger:2	triggers/ContactTrigger.trigger:2	One trigger per object — scanner misfire	False positive
Multiple Trigger On same sObject	2	triggers/DonationDesignationTrigger.trigger:2	triggers/DonationDesignationTrigger.trigger:2	One trigger per object — scanner misfire	False positive
Multiple Trigger On same sObject	3	triggers/EventTicketTypeTrigger.trigger:1	triggers/EventTicketTypeTrigger.trigger:1	One trigger per object — scanner misfire	False positive
Multiple Trigger On same sObject	4	triggers/PaymentAccountTrigger.trigger:1	triggers/PaymentAccountTrigger.trigger:1	One trigger per object — scanner misfire	False positive
DML Statements Inside Loops	1	classes/fflib_SObjectUnitOfWork.cls:1037	classes/SystemDML.cls:35	Unit of Work commit — one statement per object type	By design
DML Statements Inside Loops	2	classes/fflib_SObjectUnitOfWork.cls:1037	classes/fflib_SObjectUnitOfWork.cls:208	Unit of Work commit — one statement per object type	By design
DML Statements Inside Loops	3	classes/fflib_SObjectUnitOfWork.cls:1037	classes/fflib_SObjectUnitOfWork.cls:286	Unit of Work commit — one statement per object type	By design
DML Statements Inside Loops	4	classes/fflib_SObjectUnitOfWork.cls:1046	classes/SystemDML.cls:40	Unit of Work commit — one statement per object type	By design
DML Statements Inside Loops	5	classes/fflib_SObjectUnitOfWork.cls:1046	classes/fflib_SObjectUnitOfWork.cls:218	Unit of Work commit — one statement per object type	By design
DML Statements Inside Loops	6	classes/fflib_SObjectUnitOfWork.cls:1046	classes/fflib_SObjectUnitOfWork.cls:290	Unit of Work commit — one statement per object type	By design
DML Statements Inside Loops	7	classes/fflib_SObjectUnitOfWork.cls:1056	classes/SystemDML.cls:45	Unit of Work commit — one statement per object type	By design
DML Statements Inside Loops	8	classes/fflib_SObjectUnitOfWork.cls:1056	classes/fflib_SObjectUnitOfWork.cls:229	Unit of Work commit — one statement per object type	By design
DML Statements Inside Loops	9	classes/fflib_SObjectUnitOfWork.cls:1056	classes/fflib_SObjectUnitOfWork.cls:299	Unit of Work commit — one statement per object type	By design
DML Statements Inside Loops	10	classes/fflib_SObjectUnitOfWork.cls:1070	classes/SystemDML.cls:50	Unit of Work commit — one statement per object type	By design
DML Statements Inside Loops	11	classes/fflib_SObjectUnitOfWork.cls:1070	classes/fflib_SObjectUnitOfWork.cls:240	Unit of Work commit — one statement per object type	By design
DML Statements Inside Loops	12	classes/fflib_SObjectUnitOfWork.cls:1070	classes/fflib_SObjectUnitOfWork.cls:294	Unit of Work commit — one statement per object type	By design
DML Statements Inside Loops	13	classes/StripePaymentSweep.cls:149	classes/StripePaymentSweep.cls:203	Batch job — per-row isolation by design	By design
DML Statements Inside Loops	14	classes/StripePaymentSweep.cls:149	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	15	classes/StripePaymentSweep.cls:149	classes/EventPurchaseService.cls:590	Single statement after the loop	By design

Rule	Path	First node (scan)	Flagged statement (scan)	Group	Status
DML Statements Inside Loops	16	classes/StagingReconciliationBatch.cls:92	classes/StagingReconciliationBatch.cls:234	Batch job — per-row isolation by design	By design
DML Statements Inside Loops	17	classes/StagingReconciliationBatch.cls:92	classes/StagingReconciliationBatch.cls:240	Batch job — per-row isolation by design	By design
DML Statements Inside Loops	18	classes/StagingReconciliationBatch.cls:92	classes/StagingReconciliationBatch.cls:275	Batch job — per-row isolation by design	By design
DML Statements Inside Loops	19	classes/StagingReconciliationBatch.cls:92	classes/StagingReconciliationBatch.cls:281	Batch job — per-row isolation by design	By design
DML Statements Inside Loops	20	classes/StagingReconciliationBatch.cls:92	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	21	classes/StagingReconciliationBatch.cls:92	classes/EventPurchaseService.cls:590	Single statement after the loop	By design
DML Statements Inside Loops	22	classes/GuestRequestService.cls:140	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	23	classes/RecurringRenewalBatch.cls:153	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	24	classes/StripePaymentSweep.cls:86	classes/StripePaymentSweep.cls:191	Batch job — per-row isolation by design	By design
DML Statements Inside Loops	25	classes/StripePaymentSweep.cls:86	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	26	classes/NpsSyncService.cls:82	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	27	classes/NpsSyncService.cls:148	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	28	classes/CampaignMemberSyncService.cls:150	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	29	classes/CampaignMemberSyncService.cls:152	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	30	classes/CampaignDesignationService.cls:171	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	31	classes/AttendeeNamingHandler.cls:193	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	32	classes/RefundReconciliationBatch.cls:217	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	33	classes/ReceiptService.cls:313	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	34	classes/ReceiptService.cls:392	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	35	classes/FundraiserAdminController.cls:519	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	36	classes/CommunityService.cls:698	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	37	classes/EventPurchaseService.cls:911	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	38	classes/EventPurchaseService.cls:917	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	39	classes/RecurringRenewalBatch.cls:80	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	40	classes/StripePaymentSweep_Test.cls:207	classes/StripePaymentSweep.cls:191	Batch job — per-row isolation by design	By design
DML Statements Inside Loops	41	classes/StripePaymentSweep_Test.cls:207	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	42	classes/RecurringRenewalBatch.cls:66	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	43	classes/EventTicketAdminController.cls:258	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	44	classes/EventTicketAdminController.cls:258	classes/PricebookService.cls:125	Single statement after the loop	By design
DML Statements Inside Loops	45	classes/EventTicketAdminController.cls:258	classes/PricebookService.cls:135	Single statement after the loop	By design
DML Statements Inside Loops	46	classes/RefundReconciliationBatch.cls:283	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	47	classes/EventRegistrationController.cls:591	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	48	classes/EventPurchaseService.cls:1195	classes/AppLogger.cls:125	Error logging — guarded insert	By design

Rule	Path	First node (scan)	Flagged statement (scan)	Group	Status
DML Statements Inside Loops	49	classes/NpspSyncService.cls:114	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	50	classes/GuestRequestService.cls:124	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	51	classes/AttendeeNamingHandler_Test.cls:237	classes/AttendeeNamingHandler_Test.cls:40	Test fixture	By design
DML Statements Inside Loops	52	classes/AttendeeNamingHandler_Test.cls:237	classes/AttendeeNamingHandler.cls:129	Single statement after the loop	By design
DML Statements Inside Loops	53	classes/SecureTokens.cls:30	classes/DonationService.cls:890	Single statement after the loop	By design
DML Statements Inside Loops	54	classes/CancelGoalReachedRecurringQueueable.cls:89	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	55	classes/RefundReconciliationBatch.cls:177	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	56	classes/ReceiptService.cls:514	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	57	classes/CampaignPostService.cls:679	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	58	classes/PostInstallHandler.cls:804	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	59	classes/AttendeeNamingHandler.cls:133	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	60	classes/CampaignPostService.cls:332	classes/CampaignPostService.cls:678	Single statement after the loop	By design
DML Statements Inside Loops	61	classes/CommunityService.cls:728	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	62	classes/CampaignMemberSyncService.cls:65	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	63	classes/StripePaymentSweep.cls:113	classes/StripePaymentSweep.cls:203	Batch job — per-row isolation by design	By design
DML Statements Inside Loops	64	classes/StripePaymentSweep.cls:113	classes/EventPurchaseService.cls:590	Single statement after the loop	By design
DML Statements Inside Loops	65	classes/AttendeeNamingHandler.cls:185	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	66	classes/CompanyMatchService.cls:47	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	67	classes/EventPurchaseService.cls:419	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	68	classes/DonorPortalService.cls:452	classes/DonorPortalService.cls:443	Single statement after the loop	By design
DML Statements Inside Loops	69	classes/CommunityService.cls:793	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	70	classes/PostInstallHandler.cls:1754	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	71	classes/CampaignPostService.cls:613	classes/CampaignPostService.cls:678	Single statement after the loop	By design
DML Statements Inside Loops	72	classes/EventPurchaseService.cls:1049	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	73	classes/CampaignPostService.cls:286	classes/CampaignPostService.cls:678	Single statement after the loop	By design
DML Statements Inside Loops	74	classes/CampaignPostService.cls:302	classes/CampaignPostService.cls:678	Single statement after the loop	By design
DML Statements Inside Loops	75	classes/EventPurchaseService.cls:832	classes/PricebookService.cls:125	Single statement after the loop	By design
DML Statements Inside Loops	76	classes/EventPurchaseService.cls:832	classes/PricebookService.cls:135	Single statement after the loop	By design
DML Statements Inside Loops	77	classes/RecurringRenewalBatch.cls:426	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	78	classes/NpspSyncService.cls:56	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	79	classes/RecordTypeMappingService.cls:112	classes/SettingsService.cls:345	Single statement after the loop	By design
DML Statements Inside Loops	80	classes/StripeGateway.cls:533	classes/StripePaymentSweep.cls:203	Batch job — per-row isolation by design	By design

Rule	Path	First node (scan)	Flagged statement (scan)	Group	Status
DML Statements Inside Loops	81	classes/StripeGateway.cls:533	classes/EventPurchaseService.cls:590	Single statement after the loop	By design
DML Statements Inside Loops	82	classes/RecurringRenewalBatch_Test.cls:722	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	83	classes/PricebookService_Test.cls:182	classes/PricebookService.cls:125	Single statement after the loop	By design
DML Statements Inside Loops	84	classes/PricebookService_Test.cls:182	classes/PricebookService.cls:135	Single statement after the loop	By design
DML Statements Inside Loops	85	classes/CampaignPostService.cls:589	classes/CampaignPostService.cls:678	Single statement after the loop	By design
DML Statements Inside Loops	86	classes/RecurringDonationService_Test.cls:1399	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	87	classes/RefundReconciliationBatch.cls:628	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	88	classes/CampaignPostService.cls:605	classes/CampaignPostService.cls:678	Single statement after the loop	By design
DML Statements Inside Loops	89	classes/AttendeeNamingHandler.cls:92	classes/AppLogger.cls:125	Error logging — guarded insert	By design
DML Statements Inside Loops	90	classes/CampaignPostService.cls:256	classes/CampaignPostService.cls:678	Single statement after the loop	By design
DML Statements Inside Loops	91	classes/EventTicketAdminController.cls:210	classes/PricebookService.cls:125	Single statement after the loop	By design
DML Statements Inside Loops	92	classes/EventTicketAdminController.cls:210	classes/PricebookService.cls:135	Single statement after the loop	By design
DML Statements Inside Loops	93	classes/DonorPortalService_Test.cls:456	classes/DonorPortalService.cls:392	Single statement after the loop	By design
DML Statements Inside Loops	94	classes/DonorPortalService_Test.cls:456	classes/DonorPortalService.cls:383	Single statement after the loop	By design
DML Statements Inside Loops	95	classes/EventPurchaseService.cls:772	classes/PricebookService.cls:125	Single statement after the loop	By design
DML Statements Inside Loops	96	classes/EventPurchaseService.cls:772	classes/PricebookService.cls:135	Single statement after the loop	By design